NetGuard AI: Malware Detection Using AI-Based Network Traffic Analysis

BCSE308P – COURSE PROJECT

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Computer Science and Engineering (Artificial Intelligence and Data Engineering) / Bachelor of Computer Science and Engineering (Cyber Security)

in

Department of Cybersecurity and Intelligent Systems

by

<Student Names>

<Register Numbers>



School of Computer Science Engineering and Information Systems [SCORE]

VIT, Vellore

November, 2026

DECLARATION

I hereby declare that the BCSE308P – Computer Networks Course Project entitled "NetGuard AI: Malware Detection Using AI-Based Network Traffic Analysis" submitted by me, for the award of the degree of Bachelor of Computer Science and Engineering (Artificial Intelligence and Data Engineering) / Bachelor of Computer Science and Engineering (Cyber Security), School of Computer Science Engineering and Information Systems to VIT is a record of bonafide work carried out by me under the supervision of <Faculty Guide Name, Designation, SCORE, VIT, Vellore>.

I further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place: Vellore

Date:

Signature of the Candidate

CERTIFICATE

This is to certify that the BCSE308P – Computer Networks Course Project entitled "NetGuard AI: Malware Detection Using AI-Based Network Traffic Analysis" submitted by <Student Name & Reg. No.>, SCORE, VIT, for the award of the degree of Bachelor of Computer Science and Engineering (Artificial Intelligence and Data Engineering) / Bachelor of Computer Science and Engineering (Cyber Security), School of Computer Science Engineering and Information Systems, is a record of bonafide work carried out under my supervision during the period <start date> to <end date>, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place: Vellore

Date:

Signature of the VIT-SCORE - Guide

ACKNOWLEDGEMENT

I express my sincere gratitude to my guide, <Faculty Guide Name>, School of Computer Science Engineering and Information Systems, VIT, Vellore, for the guidance, encouragement and constructive feedback given throughout this project.

I thank the Head of the Department of Cybersecurity and Intelligent Systems and the Dean of SCORE for providing the academic environment and facilities that made this work possible.

I am grateful to the Canadian Institute for Cybersecurity, University of New Brunswick, for publishing the CIC-IDS2017 dataset, and to the developers of the open-source tools used in this project, including pandas, scikit-learn, FastAPI, React and Vite.

Finally, I thank my family and friends for their constant support.

<Student Name>

Executive Summary

NetGuard AI is a web application that detects malicious network traffic by applying machine learning to network flow records from the public CIC-IDS2017 dataset, whose 2,830,743 flows were analysed in detail. The analysis found 331,200 exact duplicates, infinite and missing values caused by zero-duration flows, protocol sentinel values, constant features and an extremely uneven distribution of attack types. After cleaning, a stratified 500,000-flow sample was split into training, validation, calibration and test sets. Random Forest, Decision Tree and Logistic Regression were compared; the selected Random Forest reached an F1-score of 99.49% on 100,000 unseen flows. A reliability layer calibrates its probabilities, keeps false alarms within a 1% budget while raising recall to 99.92%, routes uncertain flows to review and warns when an uploaded batch differs from the training traffic. A temporal experiment showed that recall falls to 30.3% on capture days with attack types absent from training, the key limitation of random-split evaluation. The model is served through a FastAPI backend and a React dashboard.

CONTENTS

Page No.

Acknowledgement ..... **iv**
Executive Summary ..... **v**
Table of Contents ..... **vi**
List of Figures ..... **ix**
List of Tables ..... **xi**
Abbreviations ..... **xiii**
Symbols and Notations ..... **xv**
1 INTRODUCTION **1**
1.1 Objective 2
1.2 Motivation 3
1.3 Background 3
1.3.1 Network traffic and the TCP/IP model 4
1.3.2 Network flows 4
1.3.3 Intrusion detection systems 5
1.3.4 Machine learning for intrusion detection 6
1.3.5 Intrusion detection datasets 7
1.3.6 Attack types in CIC-IDS2017 7
2 PROJECT DESCRIPTION AND GOALS **9**
2.1 Problem statement 9
2.2 Project description 9
2.3 Goals 10
2.4 Scope 11
2.5 Deliverables 11
2.6 Users and use cases 12
3 TECHNICAL SPECIFICATION **13**
3.1 Hardware specification 13
3.2 Software specification 13
3.3 Dataset specification 14
3.4 Functional requirements 14
3.5 Non-functional requirements 14
3.6 API specification 15
4 DESIGN APPROACH AND DETAILS **17**
4.1 Design Approach / Materials & Methods 17
4.1.1 Methodology 17

4.1.2 System architecture 18
4.1.3 The CIC-IDS2017 dataset 19
4.1.4 Flow features 21
4.1.5 Exploratory data analysis 25
4.1.6 Data preprocessing pipeline 34
4.1.7 Sampling and data splitting 36
4.1.8 Evaluation protocols: random versus temporal 38
4.1.9 Machine learning models 38
4.1.10 Evaluation metrics and model selection 40
4.1.11 Reliability layer 41
4.1.12 Backend design 42
4.1.13 Frontend design 43
4.2 Codes and Standards 44
4.3 Constraints, Alternatives and Tradeoffs 45
5 SCHEDULE, TASKS AND MILESTONES 47
5.1 Tasks 47
5.2 Gantt chart 47
5.3 Milestones 48
5.4 Risks and mitigation 48
6 PROJECT DEMONSTRATION 50
6.1 Sample Codes 50
6.1.1 Shared preprocessing (ml/preprocess.py) 50
6.1.2 Splitting protocols (ml/train.py) 51
6.1.3 Model definitions and selection (ml/train.py) 52
6.1.4 Risk-controlled threshold and three-way decision (ml/reliability.py) 53
6.1.5 Batch shift score (ml/reliability.py) 54
6.1.6 Loading the model and predicting (backend/app/model_store.py) 55
6.1.7 Upload validation and the prediction endpoint (backend/app/main.py) 56
6.1.8 Frontend API client (frontend/src/api.ts) 58
6.1.9 Automated tests (backend/tests/) 59
6.1.10 Per-attack evaluation (ml/analyze_dataset.py) 60
6.1.11 Running the project 60
6.2 Sample Screen Shots 61
7 RESULT & DISCUSSION 69
7.1 Overall model comparison 69
7.2 Confusion matrices 70
7.3 Threshold-independent evaluation 71

7.4 Detection rate per attack type 72
7.5 Effect of the reliability layer 74
7.6 Generalisation to unseen days 75
7.7 Batch shift gate 77
7.8 Feature importance 78
7.9 Results on the demonstration files 79
7.10 Limitations and threats to validity 79
7.11 Cost analysis 80
8 SUMMARY 81
9 REFERENCES 83
APPENDIX A 87
A.1 Repository structure 87
A.2 Example API response 87
A.3 Features required by the model 88
A.4 Demonstration and test files 89

List of Figures

Figure No.	Title	Page No.
1.1	Forward and Backward Packets, Flow Duration and Inter-Arrival Time in One TCP Flow	5
4.1	System Architecture of NetGuard AI	19
4.2	Generation of the CIC-IDS2017 Flow Records	20
4.3	Benign and Malicious Flows in Each CIC-IDS2017 File	21
4.4	Feature Groups of the CICFlowMeter Feature Set	22
4.5	Number of Flows per Label in the Raw Dataset (Log Scale)	25
4.6	Share of Each Label Removed as Exact Duplicates	27
4.7	Infinite, Missing and Negative Values in the Raw Files (Log Scale)	28
4.8	Distribution of Six Flow Features for Benign and Malicious Flows	29
4.9	Cumulative Distribution of Flow Duration by Class	30
4.10	Ten Most Frequent Destination Ports for Benign and Malicious Flows	32
4.11	Pearson Correlation Among the Fifteen Most Important Features	33
4.12	Preprocessing Pipeline With Row Counts at Each Step	34
4.13	Share of Each Label in the Cleaned Dataset and in the Training Sample	36
5.1	Project Gantt Chart	48
6.1	Dashboard on Start-Up: Navigation, Model Status and Upload Panel	61
6.2	Analysis Summary for the 2,000-Flow Demonstration File	61
6.3	Shift Verdict for an In-Distribution Batch	62
6.4	Prediction Results Table With Decisions and Probabilities	62
6.5	Results Filtered to the Misclassified Flows	63
6.6	Warning for a Batch Outside the Training Distribution	63
6.7	Validation Error for a File With Missing Feature Columns	63
6.8	Model Comparison Table and Chart in the Dashboard	64
6.9	Confusion Matrix in the Dashboard	65
6.10	Training Data Summary in the Dashboard	66
6.11	Most Important Features in the Dashboard	66
6.12	Interactive API Documentation Generated by FastAPI	67
6.13	Dashboard on a 390-Pixel-Wide Mobile Screen	68
7.1	Accuracy, Precision, Recall and F1-Score of the Three Models	69
7.2	Test-Set Confusion Matrices With Row Percentages	70
7.3	ROC and Precision–Recall Curves on the Test Set	71
7.4	Distribution of the Random Forest Vote Fraction by Actual Class	72
7.5	Detection Rate per Attack Label on the Test Set (Default Threshold)	73

7.6 Random Forest Performance Under the Three Evaluation Protocols (Raw Vote at 0.5) 75
7.7 Detection of Each Later-Day Attack: Random Split Versus Unseen Days 76
7.8 Batch Shift Score of Each Capture Day in the Test Set 77
7.9 Fifteen Most Important Features of the Random Forest 78

List of Tables

Table No.	Title	Page No.
1.1	TCP Header Flags Counted in CIC-IDS2017 Flow Features	4
1.2	Classification of Intrusion Detection Systems	6
1.3	Public Network Intrusion Detection Datasets	7
1.4	Attack Types in CIC-IDS2017 and Their Network Behaviour	7
2.1	Project Scope	11
2.2	Project Deliverables and Their Location in the Repository	11
3.1	Hardware Used for Development and Training	13
3.2	Software Stack and Versions	13
3.3	Dataset Specification	14
3.4	Functional Requirements	14
3.5	Non-Functional Requirements	14
3.6	REST API Endpoints	15
3.7	API Error Responses	15
4.1	CIC-IDS2017 Files, Flow Counts and Attacks	20
4.2	Feature Dictionary of the 77 CIC-IDS2017 Flow Features	22
4.3	Flows per Label Before and After Deduplication and in the Training Sample	26
4.4	Invalid and Sentinel Values in the Raw Dataset	28
4.5	Median and Mean of Key Features by Class (Unique Flows)	30
4.6	Median Traffic Profile of Each Label	31
4.7	Most Strongly Correlated Feature Pairs	32
4.8	Composition of the Training, Validation, Calibration and Test Sets	37
4.9	Rare Attack Labels in Each Set	37
4.10	Evaluation Protocols	38
4.11	Model Configurations	39
4.12	Input Validation Rules	43
4.13	Frontend Components	44
4.14	Standards Followed in the Project	44
4.15	Design Alternatives and the Chosen Trade-Offs	45
5.1	Project Tasks and Planned Weeks	47
5.2	Project Milestones	48
5.3	Project Risks and Mitigation	48
7.1	Test-Set Performance of the Three Models (Positive Class = Malicious)	69
7.2	Confusion Matrix Counts and Error Rates	70
7.3	Area Under the ROC Curve and Average Precision	71

7.4 Detection Rate per Label on the Test Set 73
7.5 Random Forest With Progressively More of the Reliability Layer (Test Set) 74
7.6 Results Under the Three Protocols 75
7.7 Temporal Protocol: Results per Label on Thursday Afternoon and Friday 76
7.8 Shift Gate on In-Distribution and Synthetic Shifted Batches 77
7.9 Project Cost 80
A.1 Model Input Features 88
A.2 Demonstration and Test Files 89

List of Abbreviations

ACK	Acknowledgement (TCP flag)
CDF	Cumulative Distribution Function
CRC	Conformal Risk Control
ECE	Expected Calibration Error
KS	Kolmogorov–Smirnov (test)
API	Application Programming Interface
AUC	Area Under the Curve
AP	Average Precision
CIC	Canadian Institute for Cybersecurity
CIC-IDS2017	CIC Intrusion Detection Evaluation Dataset 2017
CORS	Cross-Origin Resource Sharing
CSV	Comma-Separated Values
CWR / CWE	Congestion Window Reduced (TCP flag; CWE in the dataset)
DDoS	Distributed Denial of Service
DNS	Domain Name System
DoS	Denial of Service
DT	Decision Tree
ECE	ECN-Echo (TCP flag)
ECN	Explicit Congestion Notification
FIN	Finish (TCP flag)
FN	False Negative
FP	False Positive
FPR	False Positive Rate
FTP	File Transfer Protocol
HIDS	Host-based Intrusion Detection System
HTTP(S)	Hypertext Transfer Protocol (Secure)
IAT	Inter-Arrival Time
IDS	Intrusion Detection System
IETF	Internet Engineering Task Force
IP	Internet Protocol
IPFIX	IP Flow Information Export
IR	Imbalance Ratio
JSON	JavaScript Object Notation

LR	Logistic Regression
MDI	Mean Decrease in Impurity
ML	Machine Learning
NaN	Not a Number (missing value)
NIDS	Network-based Intrusion Detection System
NTP	Network Time Protocol
OAS	OpenAPI Specification
PCAP	Packet Capture (file format)
PSH	Push (TCP flag)
REST	Representational State Transfer
RF	Random Forest
RFC	Request for Comments
ROC	Receiver Operating Characteristic
RST	Reset (TCP flag)
SMOTE	Synthetic Minority Over-sampling Technique
SQL	Structured Query Language
SSH	Secure Shell
SYN	Synchronise (TCP flag)
TCP	Transmission Control Protocol
TLS	Transport Layer Security
TN	True Negative
TP	True Positive
UDP	User Datagram Protocol
UI	User Interface
UNB	University of New Brunswick
URG	Urgent (TCP flag)
WCAG	Web Content Accessibility Guidelines
XSS	Cross-Site Scripting

Symbols and Notations

x, x_i	Feature vector of a flow / of the i-th flow
x_{ij}	Value of feature j for flow i
y, y_i	Class label (0 = benign, 1 = malicious)
$\hat{y}$	Predicted label
n, N	Number of flows
N_c	Number of flows of class c
p	Number of features (69)
T	Number of trees in the Random Forest (100)
m	Features considered at each split ($\lfloor\sqrt{p}\rfloor = 8$)
$h_t(x)$	Prediction of tree t
p_k	Share of class k at a tree node
G	Gini impurity
ΔG	Impurity decrease of a split
Imp_j	Importance of feature j
$\sigma(\cdot)$	Sigmoid function
w, b	Weights and bias of Logistic Regression
μ_j, σ_j	Mean and standard deviation of feature j (training set)
z_{ij}	Standardised feature value
r	Pearson correlation coefficient
TP, TN, FP, FN	Confusion-matrix counts
F_1	F1-score
µs	Microsecond (unit of Flow Duration)
α	False-positive budget of the alert threshold (0.01)
$g(\cdot)$	Isotonic calibration map
t_alert, t_benign	Alert threshold and lower edge of the review band
n_0	Number of benign flows in the threshold half of the calibration set
S_j	Scaled Kolmogorov–Smirnov statistic of feature j in the shift gate

CHAPTER 1

INTRODUCTION

Computer networks carry almost every activity of a modern organisation, from e-mail and file sharing to banking, cloud services and remote administration. The same connectivity that makes these services possible also exposes them to attack. Malicious software rarely acts in isolation: a worm spreads by scanning for vulnerable hosts, a botnet client contacts its command-and-control server, ransomware downloads keys and exfiltrates data, and denial-of-service tools flood a victim with requests. Every one of these activities leaves a trace in the network traffic, so analysing that traffic is one of the most effective ways of detecting malware and intrusions.

Traditional intrusion detection systems (IDS) compare traffic with a database of known attack signatures. They are precise for attacks they already know but cannot recognise new variants, and maintaining the signature database requires constant expert effort. Machine learning (ML) offers a complementary approach: instead of writing rules by hand, a model learns the statistical difference between normal and malicious traffic from labelled examples. Because ML works on summary statistics of network flows, such as how long a connection lasted, how many packets were exchanged and how large they were, it can also analyse encrypted traffic whose payload is unreadable.

This project, **NetGuard AI**, builds a complete and working malware detection system based on AI-driven analysis of network traffic. It uses the public CIC-IDS2017 dataset published by the Canadian Institute for Cybersecurity, which contains 2,830,743 labelled network flows recorded over five working days in a realistic testbed. The dataset is analysed in depth, cleaned and prepared; three classifiers (Random Forest, Decision Tree and Logistic Regression) are trained and compared on held-out data; and the best model is served through a FastAPI backend and a React dashboard where a user can upload a CSV file of flows and receive a benign or malicious verdict for every flow. A reliability layer calibrates the model's probabilities, sets the alert threshold for a chosen false-positive budget, routes uncertain flows to human review and warns when an uploaded batch does not resemble the training traffic. Particular attention is given to the

dataset itself, because the quality, balance and structure of the data determine what any model can learn, and a temporal experiment tests how the model performs on days it has never seen.

1.1 Objective

The primary objective of this project is to develop a web-based system that uses machine learning and data analysis techniques to classify network flows as benign or malicious, and to understand and document the dataset on which such a system depends. The specific objectives are:

- ✓ To study the CIC-IDS2017 dataset in detail: how it was generated, what each of its 78 flow features means in networking terms, how its classes are distributed and what data-quality problems it contains.
- ✓ To design a cleaning and preprocessing pipeline that handles missing and infinite values, duplicate records and uninformative features, and that is applied identically during training and prediction.
- ✓ To split the data correctly into training, validation and test sets so that the reported performance reflects traffic the model has never seen.
- ✓ To train Random Forest, Decision Tree and Logistic Regression classifiers and compare their accuracy, precision, recall and F1-score using genuine evaluation results.
- ✓ To analyse detection performance separately for each attack type, linking strengths and weaknesses of the model back to properties of the dataset.
- ✓ To test whether the model generalises to traffic recorded on days it was not trained on, by comparing a random split with a temporal (day-based) split.
- ✓ To add a reliability layer: calibrated probabilities, an alert threshold that keeps the false positive rate within a chosen budget, a review band for uncertain flows and a batch-level distribution-shift warning.
- ✓ To deploy the selected model behind a validated REST API and a responsive dashboard named NetGuard AI that shows predictions, evaluation metrics, a

confusion matrix and a model comparison chart, and lets the user download the results.

1.2 Motivation

Network traffic grows faster than the number of analysts who can inspect it. A single office network in the CIC-IDS2017 testbed produced more than half a million flows on an ordinary Monday with no attacks at all. Manually separating the few malicious flows from this background is impossible, and signature-based tools only flag what they have been told to look for. A model that learns from flow statistics can screen every flow automatically and draw the analyst's attention to the suspicious ones.

A second motivation is the gap between research results and working systems. Many published studies report very high accuracy on CIC-IDS2017, but few explain how the data were cleaned, whether duplicate records leaked between training and testing, or how well rare attack types are detected. This project therefore treats the dataset as a first-class subject of study. Every number in this report is produced by scripts in the project repository, and every chart in the dashboard is read from files written by the training pipeline, so the results can be reproduced and questioned.

Finally, the project connects the theory of the Computer Networks course with a practical security problem. The features used by the model, such as TCP flag counts, initial window sizes, packet-length statistics, inter-arrival times and destination ports, are direct measurements of the protocols studied in the course. Understanding why a SYN-heavy, single-packet flow to a random port looks like a port scan, or why a connection that stays open for 100 seconds while sending almost no data looks like a slow denial-of-service attack, is exactly the kind of reasoning the course aims to develop.

1.3 Background

This section introduces the networking and machine-learning concepts on which the project is built: the TCP/IP protocol suite, the notion of a network flow, the main families of intrusion detection systems, the use of machine learning for intrusion detection, the public datasets available for this task and the attack types contained in CIC-IDS2017.

1.3.1 Network traffic and the TCP/IP model

Communication on the Internet follows the layered TCP/IP model. The application layer (HTTP, DNS, FTP, SSH, SMTP) produces messages; the transport layer (TCP or UDP) splits them into segments and identifies the communicating programs with 16-bit port numbers; the network layer (IP) addresses and routes packets between hosts; and the link layer delivers frames on each physical network [1], [2]. Every packet therefore carries a header for each layer, and an observer on the network can measure its size, its timing, its addresses and ports and, for TCP, its control flags.

TCP is connection-oriented. A connection starts with a three-way handshake (SYN, SYN-ACK, ACK), transfers data in acknowledged segments and ends with FIN or RST segments [3]. The flags carried in the TCP header describe what each segment is doing, and attacks often misuse them: a port scan sends SYN segments without completing the handshake, and a SYN flood leaves thousands of half-open connections. UDP, used by DNS and NTP, has no handshake and no flags [4]. Table 1.1 lists the TCP flags that CIC-IDS2017 counts for every flow.

Table 1.1 TCP Header Flags Counted in CIC-IDS2017 Flow Features

Flag	Name	Meaning	Dataset feature
SYN	Synchronise	Opens a connection and synchronises sequence numbers	SYN Flag Count
ACK	Acknowledge	Acknowledges received data; set on almost every segment after the handshake	ACK Flag Count
FIN	Finish	Sender has no more data; graceful connection close	FIN Flag Count
RST	Reset	Aborts the connection immediately	RST Flag Count
PSH	Push	Asks the receiver to pass data to the application without waiting	PSH Flag Count, Fwd/Bwd PSH Flags
URG	Urgent	Urgent pointer field is significant	URG Flag Count, Fwd/Bwd URG Flags
ECE	ECN-Echo	Explicit Congestion Notification echo [5]	ECE Flag Count
CWR	Congestion Window Reduced	Sender reduced its congestion window [5]; called CWE in the dataset	CWE Flag Count

1.3.2 Network flows

Inspecting every packet of a busy network is expensive and, for encrypted traffic, reveals little. Network monitoring therefore usually works with flows. A flow is a sequence of packets that share the same five-tuple (source IP address, destination IP

address, source port, destination port and transport protocol) within a limited time window. Router-based standards such as NetFlow version 9 [6] and IPFIX [7] export one record per flow with counters such as packets and bytes.

CIC-IDS2017 uses bidirectional flows produced by CICFlowMeter [8], [9]. The host that sends the first packet defines the forward direction and the reply traffic forms the backward direction, so a single record describes both sides of a conversation. A TCP flow ends when the connection is closed with FIN; any flow also ends when it exceeds the flow timeout. In the published files no flow lasts longer than 120,000,000 µs, i.e. 120 seconds, which is the flow timeout used by the dataset authors. Fig. 1.1 shows how the main timing and size features are measured on one TCP connection.

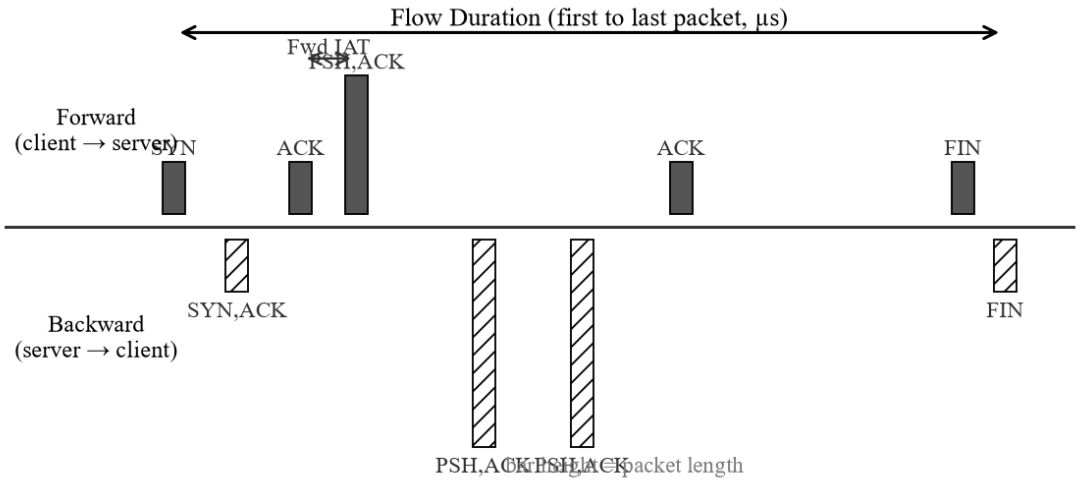


Fig. 1.1 Forward and Backward Packets, Flow Duration and Inter-Arrival Time in One TCP Flow

For each flow CICFlowMeter computes more than 80 statistics: counts and sizes of packets in each direction, rates in bytes and packets per second, inter-arrival times (IAT) between consecutive packets, counts of each TCP flag, header lengths, initial TCP window sizes, bulk-transfer and sub-flow statistics, and the durations of active and idle periods. These features are the input of the NetGuard model and are described one by one in Chapter 4.

1.3.3 Intrusion detection systems

An intrusion detection system monitors a host or network and raises alerts when it observes behaviour that indicates an attack. Denning's intrusion-detection model [10]

introduced the idea that attacks can be recognised as deviations from normal behaviour. Today IDS are classified by where they collect data and how they decide, as summarised in Table 1.2.

Table 1.2 Classification of Intrusion Detection Systems

Criterion	Type	Description	Example
Data source	Network-based (NIDS)	Analyses packets or flows observed on a network link	Snort, Zeek, NetGuard AI
Data source	Host-based (HIDS)	Analyses logs, system calls and files on one host	OSSEC, Wazuh
Detection method	Signature (misuse)	Matches traffic against patterns of known attacks	Snort rules
Detection method	Anomaly	Learns normal behaviour and flags deviations	Statistical profiles
Detection method	Supervised ML	Learns from labelled benign and attack examples	Random Forest on flows

NetGuard AI is a network-based system that uses supervised machine learning on flow records. It does not capture packets itself; it analyses flow files produced by a flow meter, which keeps the system simple and lets it be evaluated on a standard dataset.

1.3.4 Machine learning for intrusion detection

In supervised classification a model is trained on examples (x, y), where x is a vector of features describing a flow and y is its label, and then predicts the label of new flows. Surveys of the field [11] show that tree-based models, support vector machines, k-nearest neighbours and neural networks have all been applied to intrusion detection, with decision-tree ensembles such as Random Forest performing strongly on tabular flow features while remaining fast and interpretable [12], [13].

Applying ML to security also has well-documented pitfalls. Sommer and Paxson [14] warned that intrusion detection differs from other ML problems because errors are costly, traffic is diverse and attacks are rare. Arp et al. [15] listed common mistakes in security ML research, including sampling bias, data snooping (using test data for decisions) and inappropriate baselines. This project addresses these points explicitly: duplicates are removed before splitting, all decisions use a separate validation set, simple baselines are compared and results are broken down by attack type.

1.3.5 Intrusion detection datasets

Supervised models need labelled traffic, and collecting real labelled attack traffic is difficult because of privacy and the rarity of attacks. Researchers therefore rely on public datasets [16]. Table 1.3 compares the most widely used ones.

Table 1.3 Public Network Intrusion Detection Datasets

Dataset	Year	Records / features	Remarks
KDD Cup 99	1999	≈4.9 million connections, 41 features	Derived from DARPA 1998; many duplicates and dated attacks [17]
NSL-KDD	2009	125,973 train / 22,544 test, 41 features	Cleaned KDD 99 without duplicates [17]
UNSW-NB15	2015	2,540,044 records, 49 features	Nine attack families, modern traffic generator [18]
CIC-IDS2017	2017	2,830,743 flows, 78 features	Realistic benign profiles, 14 attack labels, CICFlowMeter features [19]
CSE-CIC-IDS2018	2018	Larger capture on AWS, same feature set	Successor of CIC-IDS2017 [16]

CIC-IDS2017 was chosen because it is recent compared with KDD-based data, it contains a broad set of attacks that are still relevant (denial of service, brute force, web attacks, botnets, port scans), it provides flow features computed by a documented open-source tool, and it is freely available. Its known weaknesses, such as labelling errors and flow-meter artefacts reported by Engelen et al. [20] and Lanvin et al. [21], are examined in Chapters 4 and 7.

1.3.6 Attack types in CIC-IDS2017

The dataset labels each flow either BENIGN or with one of 14 attack names. Table 1.4 explains how each attack works at the network level and which service it targets in the dataset; the target ports were measured from the data.

Table 1.4 Attack Types in CIC-IDS2017 and Their Network Behaviour

Label	Category	Network-level behaviour	Main port
DoS Hulk	Denial of service	HTTP GET flood with randomised URLs and headers to exhaust the web server	80
DoS GoldenEye	Denial of service	HTTP flood that keeps connections alive to occupy server workers	80
DoS slowloris	Denial of service	Opens many connections and sends partial HTTP headers very slowly	80
DoS Slowhttptest	Denial of service	Slow HTTP attacks that hold connections open with minimal traffic	80

Label	Category	Network-level behaviour	Main port
DDoS	Distributed DoS	Several machines flood the web server at the same time (LOIC tool)	80
Heartbleed	Vulnerability exploit	Malformed TLS heartbeat requests leak server memory (CVE-2014-0160) [22]	444
PortScan	Reconnaissance	Probes thousands of ports with short connection attempts (Nmap)	many ports
FTP-Patator	Brute force	Repeated FTP login attempts with a password list	21
SSH-Patator	Brute force	Repeated SSH login attempts with a password list	22
Web Attack – Brute Force	Web application	Automated login attempts against a vulnerable web application	80
Web Attack – XSS	Web application	Injects scripts into web pages (cross-site scripting)	80
Web Attack – Sql Injection	Web application	Injects SQL statements through web form parameters	80
Bot	Botnet	Infected hosts contact a command-and-control server (Ares botnet)	8080
Infiltration	Internal compromise	A victim host is compromised and then used to attack the internal network	444

Apart from PortScan, whose flows are spread over thousands of destination ports, each attack targets one service. Port 80 (HTTP) dominates because the denial-of-service, DDoS and web attacks all aim at the testbed's web server, the brute-force tools target FTP (21) and SSH (22), the botnet uses port 8080 for its command channel, and the Heartbleed and Infiltration flows use port 444.

CHAPTER 2

PROJECT DESCRIPTION AND GOALS

This chapter defines the problem addressed by NetGuard AI, describes what the system does from the user's point of view, states its goals in measurable terms and sets the boundaries of the project.

2.1 Problem statement

Given a file of network flow records in CICFlowMeter format, decide for every flow whether it is benign or malicious, and present the decision together with an honest account of how reliable the deciding model is. Formally this is a binary classification problem: each flow is described by a feature vector x containing numeric flow statistics, and the system must predict a label $y \in \{0, 1\}$, where 0 means benign and 1 means malicious (any of the 14 attack types).

Three difficulties make the problem harder than it first appears. The classes are imbalanced, since most traffic is benign. The dataset contains duplicate, missing, infinite and constant values that must be handled without leaking information from the test set. And some attack types are extremely rare, so a model can achieve a high overall score while still missing them. A good solution must handle all three, and its evaluation must expose them.

2.2 Project description

NetGuard AI consists of three parts that work together. The **machine learning pipeline** reads the eight CIC-IDS2017 CSV files, cleans and deduplicates them, builds a stratified sample, splits it into training, validation and test sets, trains three classifiers, evaluates them and saves the selected model together with an evaluation report. A separate **dataset analysis** script measures the properties of the data and produces the figures used in this report.

The **FastAPI backend** loads the saved model and exposes endpoints that accept a CSV upload, validate it, apply the same preprocessing as training, and return a prediction and a malicious probability for every flow, or the uploaded file with the predictions

appended as new columns. Every flow receives one of three decisions (Benign, Review or Malicious) and every batch receives a distribution-shift verdict. The **React dashboard** lets the user upload a file, start the analysis, read the benign and malicious counts, inspect and filter a results table, download the results and view the evaluation of all three models, including a confusion matrix and a comparison chart.

A typical session proceeds as follows. The user opens the dashboard, which reports that the Random Forest model is loaded. The user selects a CSV file of flows and presses *Analyze traffic*. Within a few seconds the dashboard shows how many flows were analysed and how many were classified as benign, malicious or sent for review, and a banner states whether the batch resembles the training traffic; if the file contains a Label column, the agreement between the predictions and the labels is also shown. The user can filter the table to see only malicious flows or only misclassified ones, and can download the full result as a CSV file.

2.3 Goals

The project goals, with the criterion used to verify each one, are:

1. **Dataset understanding:** document the origin, features, class distribution and data-quality issues of CIC-IDS2017 using measurements on the real files (verified in Chapter 4).
2. **Correct methodology:** remove duplicates before splitting, use a stratified split with a fixed random seed, and never use the test set for any decision (verified by the reproducibility check in Section 4.1.7).
3. **Detection quality:** reach an F1-score above 99% on the test set with the primary model; achieved 99.49% (Chapter 7).
4. **Honest comparison:** report accuracy, precision, recall and F1-score for Random Forest, Decision Tree and Logistic Regression from the same test set, plus per-attack detection rates.
5. **Generalisation:** measure performance on unseen capture days with a temporal protocol and report the result even when it is poor (Section 7.6).

6. **Controlled alerting:** keep the false positive rate of the deployed alert threshold below $\alpha = 1\%$ on held-out data and flag batches that differ from the training distribution.
7. **Working application:** serve real predictions from the saved model through a validated API and a responsive dashboard, with automated backend tests and a successful production build.
8. **Reproducibility:** every number in the dashboard and in this report is generated by scripts in the repository from the public dataset.

2.4 Scope

Table 2.1 lists what is included in the project and what is deliberately left out to keep the system focused and explainable.

Table 2.1 Project Scope

In scope	Out of scope
Binary classification of CICFlowMeter flow records (benign / malicious)	Classifying the exact attack type (multi-class)
Offline training on CIC-IDS2017 with a documented pipeline	Live packet capture or reading .pcap files
CSV upload, validation, prediction and CSV download	User accounts, authentication and a database
Random Forest, Decision Tree, Logistic Regression	Deep learning models
Evaluation dashboard with real metrics	Automatic blocking of traffic (intrusion prevention)
Calibration, review band and batch shift warning	Automatic retraining when shift is detected
Random and temporal (day-based) evaluation protocols	Online learning from analyst feedback
Detailed dataset analysis and reporting	Testing on other datasets or production networks

2.5 Deliverables

Table 2.2 Project Deliverables and Their Location in the Repository

Deliverable	Location
Shared preprocessing used by training and API	ml/preprocess.py
Training and model comparison pipeline	ml/train.py
Reliability layer and shift gate	ml/reliability.py
Protocol comparison experiments	ml/experiments.py, reports/experiments.json
Dataset analysis and report figures	ml/analyze_dataset.py, reports/figures/
Trained model bundle	models/netguard_model.joblib

Deliverable	Location
Evaluation report	reports/metrics.json, reports/model_report.md
Dataset analysis results	reports/dataset_analysis.json
FastAPI backend and tests	backend/app/, backend/tests/
React dashboard	frontend/src/
Demo CSV files from the held-out test set	dataset/samples/
Test files for evaluators (valid, invalid and shifted inputs)	dataset/faculty_tests/
One-command start and stop scripts	start.sh, stop.sh
README and this report	README.md, docs/report/

2.6 Users and use cases

The primary users are students and security analysts who want to screen a set of flow records. The main use cases are: checking whether a captured set of flows contains suspicious traffic; demonstrating how a machine learning IDS works, including its evaluation; and studying how the properties of a dataset affect what a model learns. Because the dashboard shows the probability behind every prediction and the per-model evaluation, users can judge how much to trust a result rather than receiving an unexplained verdict.

CHAPTER 3

TECHNICAL SPECIFICATION

This chapter specifies the hardware and software used, the dataset, the functional and non-functional requirements, and the interface of the backend API.

3.1 Hardware specification

Table 3.1 Hardware Used for Development and Training

Component	Specification
Processor	Intel Core i5-1135G7 (11th Gen) @ 2.40 GHz, 6 logical CPUs available to Linux
Memory	7 GB available to the Linux environment (WSL 2)
Storage	1 TB virtual disk; the dataset needs about 844 MB, the project about 100 MB plus dependencies
Peak memory during training	About 3 GB (2.8 GB measured with /usr/bin/time)
Training time	About 4 minutes to read and clean the CSV files; under 30 s to fit the three models

3.2 Software specification

Table 3.2 Software Stack and Versions

Layer	Software	Version	Purpose
Operating system	Linux on WSL 2	kernel 6.18	Development and execution environment
Language	Python	3.14.7	ML pipeline, analysis and backend
Data processing	pandas / NumPy	3.0.6 / 2.5.4	Loading and cleaning CSV data
Machine learning	scikit-learn	1.9.1	Models, pipelines, calibration, metrics
Statistics	SciPy	1.18.1	Kolmogorov–Smirnov test in the shift gate
Model storage	joblib	1.6.0	Saving and loading the trained pipeline
Web API	FastAPI / Uvicorn	0.157.0 / 0.54.0	REST endpoints and ASGI server
File upload	python-multipart	0.0.32	Parsing multipart/form-data uploads
Testing	pytest / httpx2	9.1.1 / 2.13.1	Automated backend tests
Figures and report	matplotlib / python-docx	3.11.2 / 1.2.0	Dataset figures and this document
Runtime	Node.js / npm	24.21.0 / 11.19.0	Frontend tooling
UI library	React	19.3.0	Dashboard components
Build tool	Vite	8.1.6	Development server and production build
Language	TypeScript	5.9.3	Type-checked frontend code

Layer	Software	Version	Purpose
Styling	Tailwind CSS	4.3.3	Utility-first CSS, dark navy theme
Charts / icons	Recharts / lucide-react	3.10.1 / 1.52.0	Comparison chart, feature importance, icons

3.3 Dataset specification

Table 3.3 Dataset Specification

Property	Value
Name	CIC-IDS2017 (MachineLearningCSV / MachineLearningCVE version)
Publisher	Canadian Institute for Cybersecurity, University of New Brunswick [19]
Capture period	Monday 3 July to Friday 7 July 2017 (five working days)
Files	8 CSV files, one or more per day, about 844 MB in total
Records	2,830,743 flows (2,273,097 benign, 557,646 attack)
Columns	78 numeric flow features + 1 Label column (79 in total)
Labels	BENIGN and 14 attack labels
Encoding	Latin-1 / Windows-1252 (web attack labels contain the dash byte 0x96)
Feature extractor	CICFlowMeter, bidirectional flows, ≈120 s flow timeout
Licence / access	Free for research; official download after a short form, mirrored on Kaggle

3.4 Functional requirements

Table 3.4 Functional Requirements

ID	Requirement
FR1	The system shall clean the dataset and convert its labels to Benign (BENIGN) and Malicious (all attacks).
FR2	The system shall split the data into training, validation and test sets using stratified sampling.
FR3	The system shall train Random Forest, Decision Tree and Logistic Regression and report accuracy, precision, recall and F1-score on the test set.
FR4	The system shall save the selected model with its feature list and reuse it for every prediction.
FR5	The API shall accept a CSV upload and return a prediction and malicious probability for every row.
FR6	The API shall reject files that are not CSV, are empty, exceed 100 MB, miss required features or contain non-numeric feature values.
FR7	The API shall return HTTP 503 with an explanation when no trained model is available.
FR8	The dashboard shall show total, benign and malicious counts and, when labels are present, the agreement with them.
FR9	The dashboard shall show a filterable, paginated results table and allow the results to be downloaded as CSV.
FR10	The dashboard shall show the model comparison table and chart, the confusion matrix, the dataset split and feature importances from the saved evaluation report.

3.5 Non-functional requirements

Table 3.5 Non-Functional Requirements

Quality	Requirement and how it is met

Quality	Requirement and how it is met
Correctness	No hard-coded predictions or statistics; all values come from the model or from files written by training.
Consistency	Training and inference share ml/preprocess.py, and imputation is stored inside the saved scikit-learn Pipeline.
Reproducibility	Fixed random seed (42); re-running training reproduces identical metrics and the analysis script verifies this.
Performance	A 2,000-flow file is uploaded, classified, checked for shift and returned in about 1–3 s.
Security	Text cells that a spreadsheet would run as formulas are neutralised in downloaded CSV files.
Usability	Responsive layout from 390 px phones to desktops, clear error messages, colour-blind-safe chart colours.
Maintainability	Small modules with one responsibility each; code is commented for explanation in a viva.
Testability	38 automated tests (API, preprocessing, reliability layer) run in about four seconds on synthetic data.

3.6 API specification

Table 3.6 REST API Endpoints

Method	Endpoint	Input	Output
GET	/api/health	–	Model status, feature count, and whether the reliability layer and shift gate are active
GET	/api/model/features	–	List of the 69 feature columns an upload must contain
GET	/api/metrics	–	The evaluation report written by training (metrics.json)
POST	/api/predict	multipart file (CSV)	Benign / review / malicious counts, shift verdict and warning, agreement with labels, first 5,000 rows
POST	/api/predict/download	multipart file (CSV)	The uploaded CSV with prediction, decision and probability columns added

Table 3.7 API Error Responses

HTTP status	Condition	Example message
400 Bad Request	Wrong extension, empty file, header only, unparseable CSV	Please upload a file with a .csv extension.
413 Payload Too Large	File larger than 100 MB	File is larger than 100 MB.
422 Unprocessable Entity	Required feature columns missing or non-numeric	CSV is missing 3 required feature column(s).
503 Service Unavailable	No trained model found or model file unreadable	No trained model found. Run python -m ml.train first.

Uploaded files may contain extra columns (for example source and destination IP addresses or the original Label); these are ignored for prediction. If a Label column is

present, the API compares it with the predictions and reports the agreement, which is how the demonstration in Chapter 6 shows that the predictions are genuine.

CHAPTER 4

DESIGN APPROACH AND DETAILS

This chapter explains how NetGuard AI was designed. It begins with the overall methodology and system architecture, then studies the CIC-IDS2017 dataset in depth: how it was generated, what its features measure, how its classes are distributed and what quality problems it has. The findings of that analysis drive the preprocessing, sampling and splitting design that follows. The chapter then describes the three models, the evaluation metrics, the backend and frontend, the standards followed and the constraints and trade-offs.

4.1 Design Approach / Materials & Methods

4.1.1 Methodology

The project follows a data-centred methodology in which every stage produces files that the next stage reads:

1. **Data acquisition:** download the eight MachineLearningCSV files of CIC-IDS2017 and verify the row count (2,830,743 flows) against the official figure.
2. **Exploratory data analysis:** measure class and attack distributions, missing, infinite and negative values, duplicates, constant and correlated features, and per-attack traffic profiles (ml/analyze_dataset.py).
3. **Preprocessing:** clean headers and values, remove duplicates, map labels to two classes.
4. **Sampling and splitting:** stratified sample, then a stratified train / validation / calibration / test split, plus a temporal (day-based) protocol for comparison.
5. **Modelling:** train Random Forest, Decision Tree and Logistic Regression inside scikit-learn Pipelines.
6. **Evaluation and selection:** choose the model on the validation set and report all metrics on the test set, overall and per attack type.

7. **Reliability layer:** calibrate the probabilities, set the alert threshold and review band on the calibration set, and fit a batch distribution-shift gate on benign training traffic.
8. **Deployment:** serve the selected Pipeline through FastAPI and visualise it in the React dashboard.

4.1.2 System architecture

Fig. 4.1 shows the architecture. The lower half runs once, offline: the training script reads the dataset and writes two artefacts, the model bundle (models/netguard_model.joblib, which also holds the calibrator, the decision thresholds and the shift-gate reference profile) and the evaluation report (reports/metrics.json). The upper half runs for every user request: the browser uploads a CSV file to the FastAPI backend, which validates it, prepares the features with the same preprocessing module used in training, runs the saved Pipeline, applies the reliability layer and the shift gate, and returns JSON (or a CSV file) to the browser. The backend never retrains the model, and the dashboard never computes metrics itself; it only displays what the backend returns.

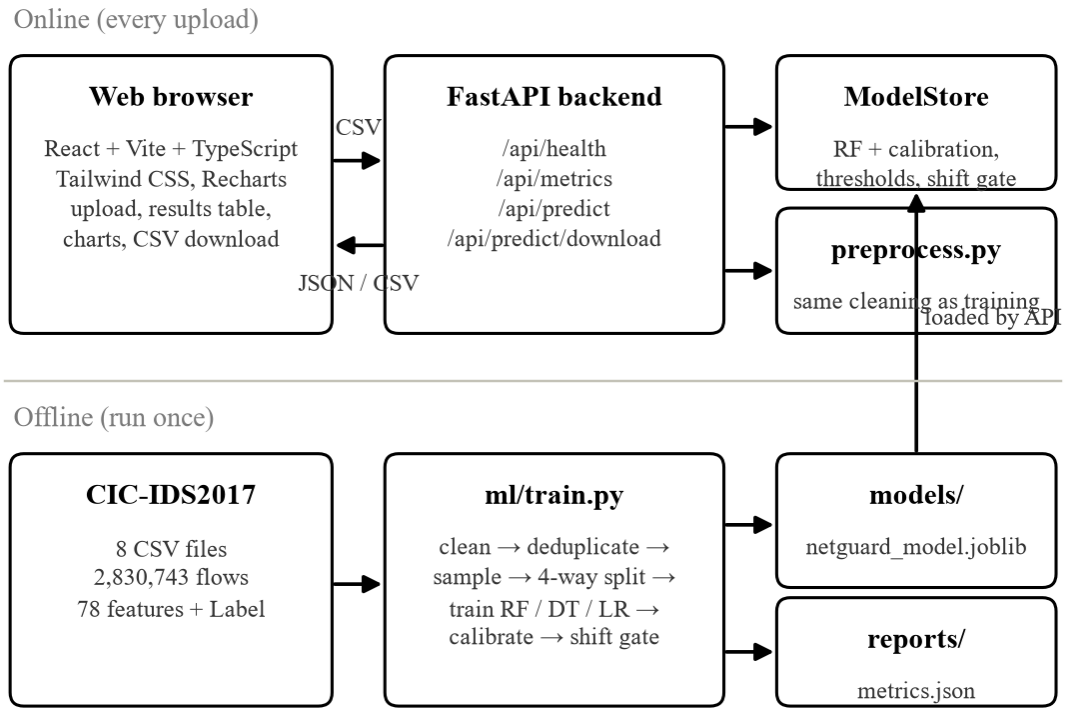


Fig. 4.1 System Architecture of NetGuard AI

Separating the offline and online parts keeps each part simple. Training can take minutes and several gigabytes of memory, whereas prediction must be fast and light. Because the saved bundle contains the fitted imputer, the optional scaler, the classifier and the exact list of feature names, the online part cannot drift from the offline part.

4.1.3 The CIC-IDS2017 dataset

CIC-IDS2017 was created by Sharafaldin, Habibi Lashkari and Ghorbani at the Canadian Institute for Cybersecurity [19]. The authors built a testbed with a victim network of Windows, Linux and macOS machines and servers protected by a firewall, and a separate attack network running Kali Linux. To produce realistic benign traffic they profiled the behaviour of 25 users over protocols such as HTTP, HTTPS, FTP, SSH and e-mail and replayed that behaviour with agents (the B-Profile system). Traffic was captured for five working days, from Monday 3 July to Friday 7 July 2017, while attacks were launched according to a schedule. Fig. 4.2 summarises how the published CSV files were produced.

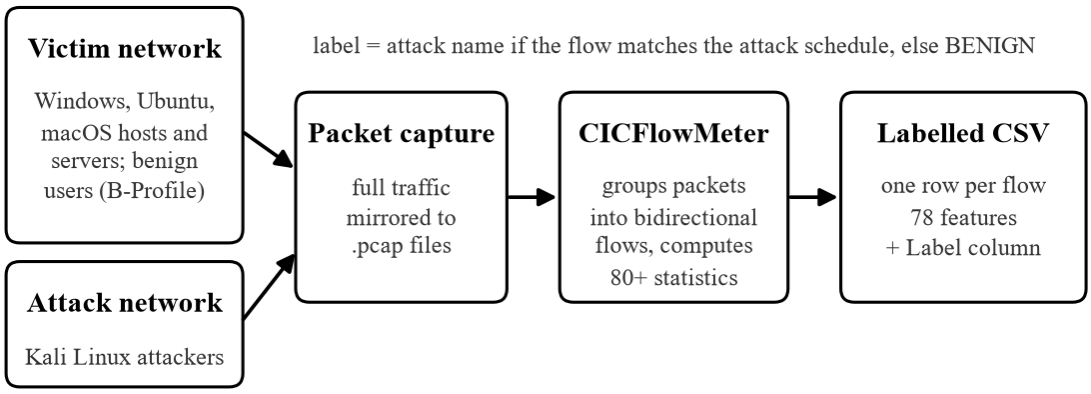


Fig. 4.2 Generation of the CIC-IDS2017 Flow Records

The full packet captures were processed with CICFlowMeter, which grouped packets into bidirectional flows and computed the flow statistics. Each flow was then labelled BENIGN or with the name of the attack running at that time between the attacker and victim addresses. The MachineLearningCSV version used here removes the identifying columns (flow ID, IP addresses, source port, protocol and timestamp) and keeps the destination port, 77 statistical features and the label. One feature, Fwd Header Length, appears twice, so the files have 79 columns in total. Table 4.1 lists the eight files, measured directly from the data.

Table 4.1 CIC-IDS2017 Files, Flow Counts and Attacks

File (day and period)	Flows	Benign	Attack labels (flows)
Monday-WorkingHours	529,918	529,918	none
Tuesday-WorkingHours	445,909	432,074	FTP-Patator (7,938), SSH-Patator (5,897)
Wednesday-workingHours	692,703	440,031	DoS Hulk (231,073), DoS GoldenEye (10,293), DoS slowloris (5,796), DoS Slowhttptest (5,499), Heartbleed (11)
Thursday-WorkingHours-Afternoon-Infilteration	288,602	288,566	Infiltration (36)
Thursday-WorkingHours-Morning-WebAttacks	170,366	168,186	Web Attack – Brute Force (1,507), Web Attack – XSS (652), Web Attack – Sql Injection (21)
Friday-WorkingHours-Afternoon-DDos	225,745	97,718	DDoS (128,027)
Friday-WorkingHours-Afternoon-PortScan	286,467	127,537	PortScan (158,930)
Friday-WorkingHours-Morning	191,033	189,067	Bot (1,966)
Total	2,830,743	2,273,097	557,646 attack flows

Monday contains only benign traffic and serves as a clean baseline. Attacks start on Tuesday with FTP and SSH brute force, continue on Wednesday with the denial-of-

service tools and Heartbleed, on Thursday with web attacks in the morning and infiltration in the afternoon, and end on Friday with the botnet, the port scan and the DDoS attack. Fig. 4.3 shows the volume of each file. Wednesday is the largest file because DoS Hulk alone generated 231,073 flows, whereas the Friday afternoon DDoS and PortScan files are more than half malicious.

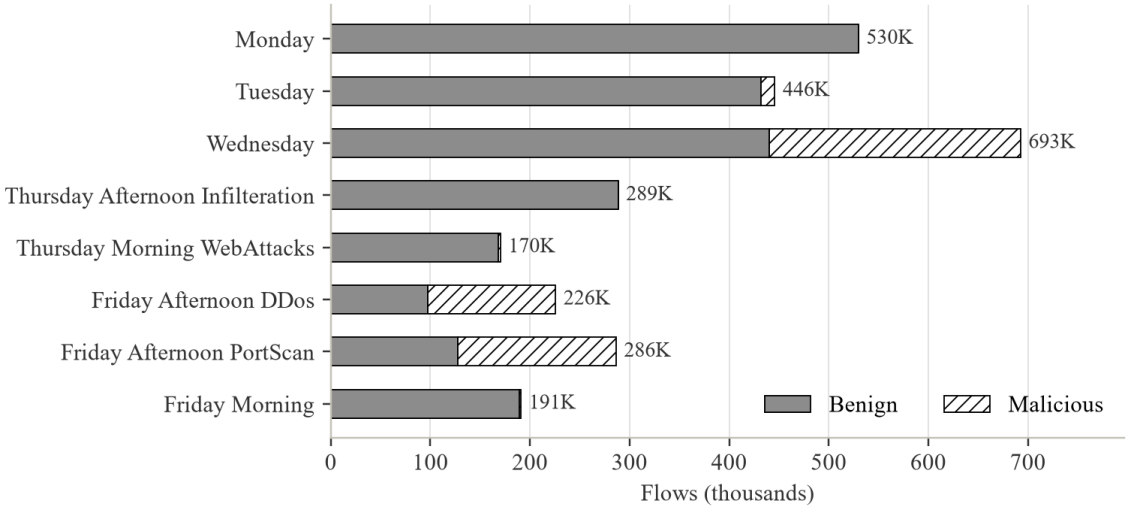


Fig. 4.3 Benign and Malicious Flows in Each CIC-IDS2017 File

4.1.4 Flow features

The 77 distinct features can be grouped by what they measure. Fig. 4.4 shows the size of each group and how many features of each group were dropped because they never change in the training data (Section 4.1.5). The groups are:

- ✓ **Basic flow:** destination port, duration and the number and total size of packets in each direction.
- ✓ **Packet length:** minimum, maximum, mean and spread of packet sizes, per direction and overall.
- ✓ **Flow rate:** bytes and packets per second.
- ✓ **Inter-arrival time (IAT):** timing between consecutive packets, overall and per direction.
- ✓ **TCP flags:** how many packets carried each TCP flag.
- ✓ **Header and window:** header bytes, TCP initial window sizes, data-carrying packets and the download/upload ratio.

- ✓ **Bulk transfer and sub-flow:** statistics of bulk data transfers and of sub-flows within a flow.
- ✓ **Active / idle:** how long the flow alternated between activity and silence.

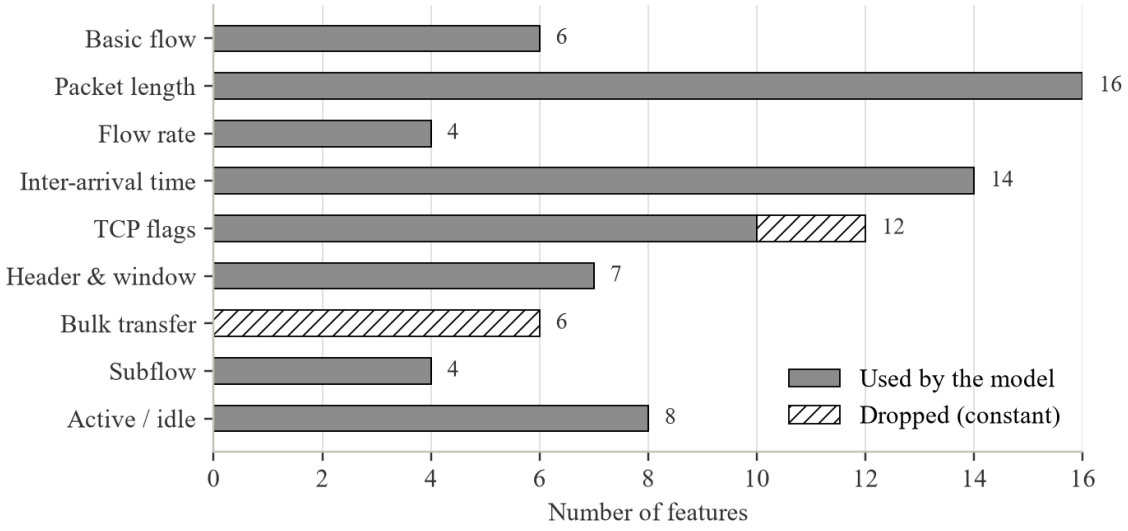


Fig. 4.4 Feature Groups of the CICFlowMeter Feature Set

Two of the features are derived quantities whose definitions matter for the data-quality analysis. The flow byte rate and packet rate are computed from the totals and the duration, which is measured in microseconds, as shown in Eqn. 4.1 and Eqn. 4.2:

$$\text{Flow Bytes/s} = \frac{\text{Total Length of Fwd Packets} + \text{Total Length of Bwd Packets}}{\text{Flow Duration}} \times 10^6 \quad (4.1)$$

$$\text{Flow Packets/s} = \frac{\text{Total Fwd Packets} + \text{Total Backward Packets}}{\text{Flow Duration}} \times 10^6 \quad (4.2)$$

These definitions were checked against the files: on all 529,466 Monday flows with a positive duration, the published Flow Bytes/s and Flow Packets/s agree with Eqn. 4.1 and Eqn. 4.2 to within 0.1%. When the duration is zero, the division in these equations is undefined, which explains the infinite and missing values described below. Table 4.2 gives the meaning of every feature; features marked as constant or identical to another feature were confirmed by measurement.

Table 4.2 Feature Dictionary of the 77 CIC-IDS2017 Flow Features

Feature	Group	Description	Model
Destination Port	Basic flow	Destination TCP/UDP port of the first packet; identifies the targeted service	used
Flow Duration	Basic flow	Time between the first and the last packet of the flow,	used

Feature	Group	Description	Model
		in microseconds	
Total Fwd Packets	Basic flow	Number of packets sent in the forward direction (initiator to responder)	used
Total Backward Packets	Basic flow	Number of packets sent in the backward direction (responder to initiator)	used
Total Length of Fwd Packets	Basic flow	Total size of the forward packets in bytes	used
Total Length of Bwd Packets	Basic flow	Total size of the backward packets in bytes	used
Fwd Packet Length Max	Packet length	Largest forward packet (bytes)	used
Fwd Packet Length Min	Packet length	Smallest forward packet (bytes)	used
Fwd Packet Length Mean	Packet length	Mean forward packet size (bytes)	used
Fwd Packet Length Std	Packet length	Standard deviation of forward packet sizes	used
Bwd Packet Length Max	Packet length	Largest backward packet (bytes)	used
Bwd Packet Length Min	Packet length	Smallest backward packet (bytes)	used
Bwd Packet Length Mean	Packet length	Mean backward packet size (bytes)	used
Bwd Packet Length Std	Packet length	Standard deviation of backward packet sizes	used
Min Packet Length	Packet length	Smallest packet in either direction	used
Max Packet Length	Packet length	Largest packet in either direction	used
Packet Length Mean	Packet length	Mean packet size in either direction	used
Packet Length Std	Packet length	Standard deviation of packet sizes	used
Packet Length Variance	Packet length	Variance of packet sizes (square of the standard deviation)	used
Average Packet Size	Packet length	Mean size of all packets in the flow	used
Avg Fwd Segment Size	Packet length	Mean forward segment size (equals Fwd Packet Length Mean)	used
Avg Bwd Segment Size	Packet length	Mean backward segment size (equals Bwd Packet Length Mean)	used
Flow Bytes/s	Flow rate	Bytes per second over the whole flow (Eqn. 4.1)	used
Flow Packets/s	Flow rate	Packets per second over the whole flow (Eqn. 4.2)	used
Fwd Packets/s	Flow rate	Forward packets per second	used
Bwd Packets/s	Flow rate	Backward packets per second	used
Flow IAT Mean	Inter-arrival time	Mean time between two consecutive packets of the flow (either direction)	used
Flow IAT Std	Inter-arrival time	Standard deviation of the flow inter-arrival time	used
Flow IAT Max	Inter-arrival time	Largest gap between consecutive packets	used
Flow IAT Min	Inter-arrival time	Smallest gap between consecutive packets	used
Fwd IAT Total	Inter-arrival time	Sum of gaps between consecutive forward packets	used
Fwd IAT Mean	Inter-arrival time	Mean gap between consecutive forward packets	used
Fwd IAT Std	Inter-arrival time	Standard deviation of forward inter-arrival times	used
Fwd IAT Max	Inter-arrival time	Largest gap between forward packets	used
Fwd IAT Min	Inter-arrival time	Smallest gap between forward packets	used
Bwd IAT Total	Inter-arrival time	Sum of gaps between consecutive backward packets	used
Bwd IAT Mean	Inter-arrival time	Mean gap between consecutive backward packets	used
Bwd IAT Std	Inter-arrival time	Standard deviation of backward inter-arrival times	used
Bwd IAT Max	Inter-arrival	Largest gap between backward packets	used

Feature	Group	Description	Model
	time		
Bwd IAT Min	Inter-arrival time	Smallest gap between backward packets	used
Fwd PSH Flags	TCP flags	Forward packets with PSH set (identical to SYN Flag Count in the files)	used
Bwd PSH Flags	TCP flags	Backward packets with PSH set (always 0 in the files)	dropped (constant)
Fwd URG Flags	TCP flags	Forward packets with URG set	used
Bwd URG Flags	TCP flags	Backward packets with URG set (always 0 in the files)	dropped (constant)
FIN Flag Count	TCP flags	Packets with the FIN flag	used
SYN Flag Count	TCP flags	Packets with the SYN flag	used
RST Flag Count	TCP flags	Packets with the RST flag	used
PSH Flag Count	TCP flags	Packets with the PSH flag	used
ACK Flag Count	TCP flags	Packets with the ACK flag	used
URG Flag Count	TCP flags	Packets with the URG flag	used
CWE Flag Count	TCP flags	Packets with the CWR (congestion window reduced) flag	used
ECE Flag Count	TCP flags	Packets with the ECE (ECN-echo) flag	used
Fwd Header Length	Header & window	Total bytes of headers in forward packets	used
Bwd Header Length	Header & window	Total bytes of headers in backward packets	used
Init_Win_bytes_forward	Header & window	TCP initial window size in the first forward packet (-1 if not TCP / not seen)	used
Init_Win_bytes_backward	Header & window	TCP initial window size in the first backward packet (-1 if not seen)	used
act_data_pkt_fwd	Header & window	Forward packets carrying at least one byte of TCP payload	used
min_seg_size_forward	Header & window	Minimum segment (header) size observed in the forward direction	used
Down/Up Ratio	Header & window	Ratio of download (backward) to upload (forward) traffic	used
Fwd Avg Bytes/Bulk	Bulk transfer	Mean bytes per bulk transfer, forward (always 0)	dropped (constant)
Fwd Avg Packets/Bulk	Bulk transfer	Mean packets per bulk transfer, forward (always 0)	dropped (constant)
Fwd Avg Bulk Rate	Bulk transfer	Mean bulk transfer rate, forward (always 0)	dropped (constant)
Bwd Avg Bytes/Bulk	Bulk transfer	Mean bytes per bulk transfer, backward (always 0)	dropped (constant)
Bwd Avg Packets/Bulk	Bulk transfer	Mean packets per bulk transfer, backward (always 0)	dropped (constant)
Bwd Avg Bulk Rate	Bulk transfer	Mean bulk transfer rate, backward (always 0)	dropped (constant)
Subflow Fwd Packets	Subflow	Mean forward packets per sub-flow (equals Total Fwd Packets)	used
Subflow Fwd Bytes	Subflow	Mean forward bytes per sub-flow (equals Total Length of Fwd Packets)	used
Subflow Bwd Packets	Subflow	Mean backward packets per sub-flow (equals Total Backward Packets)	used
Subflow Bwd Bytes	Subflow	Mean backward bytes per sub-flow	used
Active Mean	Active / idle	Mean time the flow was active before becoming idle	used
Active Std	Active / idle	Standard deviation of active periods	used
Active Max	Active / idle	Longest active period	used
Active Min	Active / idle	Shortest active period	used

Feature	Group	Description	Model
Idle Mean	Active / idle	Mean time the flow was idle before becoming active again	used
Idle Std	Active / idle	Standard deviation of idle periods	used
Idle Max	Active / idle	Longest idle period	used
Idle Min	Active / idle	Shortest idle period	used

4.1.5 Exploratory data analysis

Before designing the preprocessing, the raw files were analysed with ml/analyze_dataset.py. The script reads all eight files, measures them before and after cleaning, and saves every number to reports/dataset_analysis.json, from which the tables and figures in this section are generated.

Class and attack distribution. Fig. 4.5 shows the number of flows per label on a logarithmic scale. The distribution is extremely skewed: BENIGN accounts for 80.30% of all flows, the three largest attack labels (DoS Hulk, PortScan and DDoS) for 18.30%, and the four smallest labels together contain only 720 flows. Heartbleed has just 11 flows and SQL injection 21.

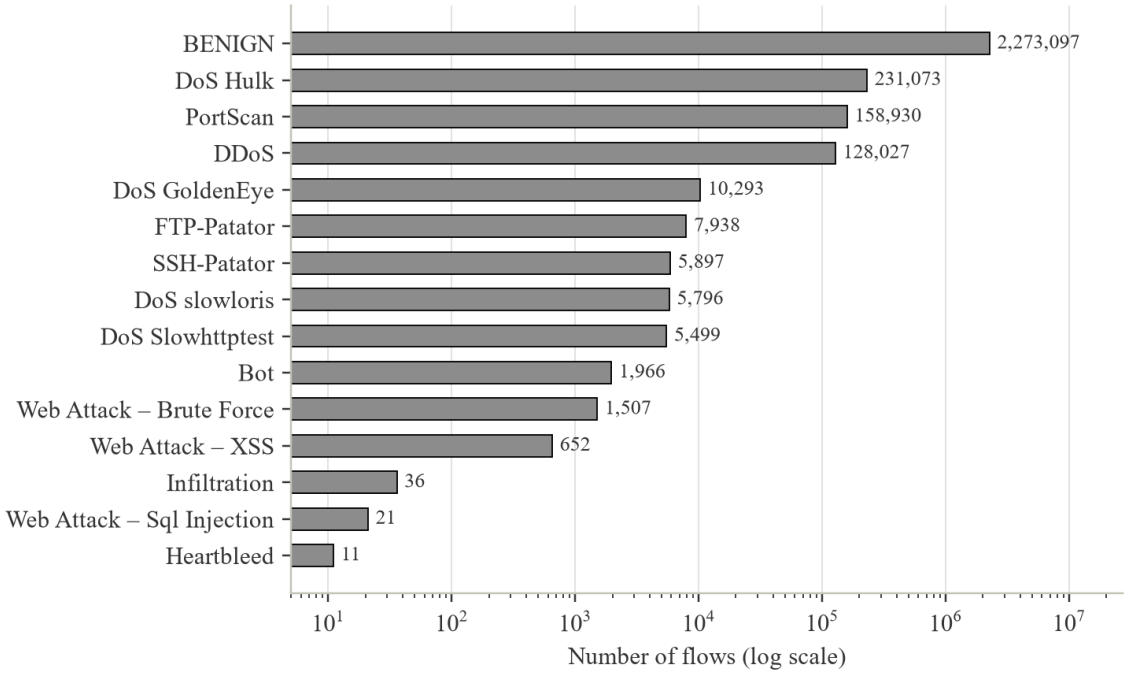


Fig. 4.5 Number of Flows per Label in the Raw Dataset (Log Scale)

For the binary problem the imbalance ratio, defined in Eqn. 4.3, is moderate: after deduplication there are 2,073,680 benign and 425,863 malicious flows, a ratio of 4.87 to

1. Within the malicious class, however, the ratio between the largest and smallest attack label exceeds 15,000 to 1, which is the root of the per-attack differences reported in Chapter 7 [23].

$$\mathrm{IR} = \frac{N_{\mathrm{benign}}}{N_{\mathrm{malicious}}} \tag{4.3}$$

Table 4.3 Flows per Label Before and After Deduplication and in the Training Sample

Label	Raw flows	Unique flows	Duplicates	In 500k sample
BENIGN	2,273,097	2,073,680	8.8%	414,812
DoS Hulk	231,073	172,849	25.2%	34,475
PortScan	158,930	90,819	42.9%	18,151
DDoS	128,027	128,016	0.0%	25,587
DoS GoldenEye	10,293	10,282	0.1%	2,092
FTP-Patator	7,938	5,933	25.3%	1,204
SSH-Patator	5,897	3,219	45.4%	648
DoS slowloris	5,796	5,374	7.3%	1,078
DoS Slowhttptest	5,499	5,228	4.9%	1,095
Bot	1,966	1,953	0.7%	405
Web Attack – Brute Force	1,507	1,470	2.5%	298
Web Attack – XSS	652	652	0.0%	141
Infiltration	36	36	0.0%	9
Web Attack – Sql Injection	21	21	0.0%	1
Heartbleed	11	11	0.0%	4
Total	2,830,743	2,499,543	11.7%	500,000

Duplicate records. 331,200 rows (11.70%) are exact copies of another row, with identical values in all 78 features and the same label. Fig. 4.6 shows that duplication is very uneven. SSH-Patator (45.4%) and PortScan (42.9%) are the most duplicated, because these tools repeat the same short exchange thousands of times: a scan probe or a failed login produces a flow with the same packet counts, sizes and flags every time. Benign traffic is far more varied, and only 8.8% of it is duplicated.

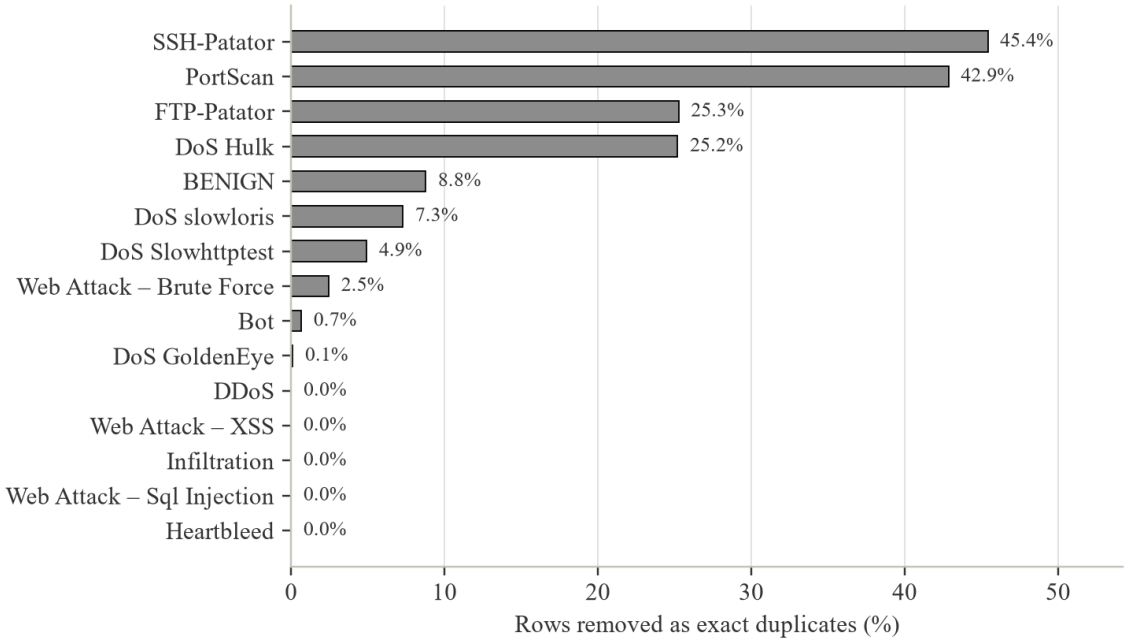


Fig. 4.6 Share of Each Label Removed as Exact Duplicates

Duplicates matter for evaluation. If one copy of a record falls in the training set and another in the test set, the model is tested on a flow it has already memorised, and the test score overstates how well it generalises. Because the most duplicated labels are attacks, keeping duplicates would also inflate the apparent detection rate of exactly those attacks. NetGuard therefore removes duplicates before any sampling or splitting.

Missing, infinite and negative values. Fig. 4.7 and Table 4.4 summarise the invalid values found in the raw files. Flow Bytes/s is missing (NaN) in 1,358 rows and infinite in 1,509 rows, and Flow Packets/s is infinite in 2,867 rows. All of them come from flows whose measured duration is zero: when both totals are also zero, Eqn. 4.1 evaluates 0/0 (NaN); otherwise it divides a positive number by zero (infinity).

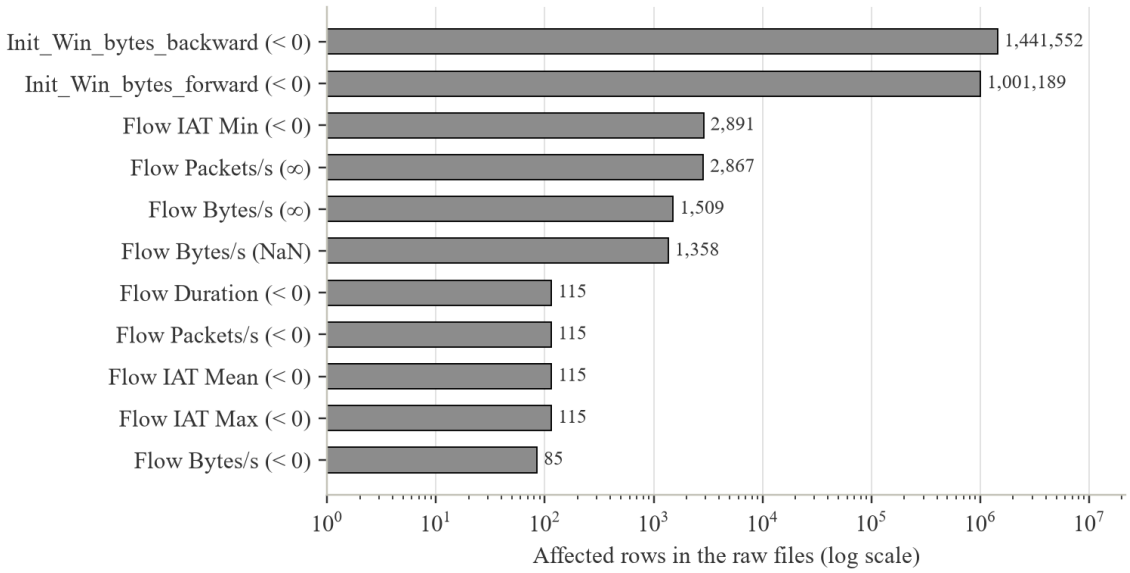


Fig. 4.7 Infinite, Missing and Negative Values in the Raw Files (Log Scale)

Table 4.4 Invalid and Sentinel Values in the Raw Dataset

Feature	Issue	Rows	Explanation and handling
Flow Bytes/s	NaN	1,358	0/0 for zero-duration flows → median imputation
Flow Bytes/s	+∞	1,509	Division by zero duration → treated as missing
Flow Packets/s	+∞	2,867	Division by zero duration → treated as missing
Init_Win_bytes_backward	-1	1,441,552	Sentinel: no TCP window seen (UDP or no reply) → kept as a value
Init_Win_bytes_forward	-1	1,001,189	Sentinel: no TCP window seen → kept as a value
Flow IAT Min	< 0	2,891	Timestamp ordering artefact of the flow meter → kept
Flow Duration	< 0	115	Clock artefact, physically impossible → kept (rare)

The value -1 in the initial window features is not an error but a sentinel. In the Monday file, every flow to port 53 (DNS) and port 123 (NTP), both UDP services, has Init_Win_bytes_forward equal to -1, whereas only 0.6% of flows to port 443 do. UDP has no window field [4], so the flow meter records -1. Because the sentinel carries protocol information that tree-based models can use, it is kept as an ordinary value rather than treated as missing. The small number of negative durations and inter-arrival times are physically impossible and point to timestamp problems in the flow meter, which Engelen et al. [20] also reported; they affect fewer than 0.01% of rows and are left unchanged.

Constant features. Eight features have the same value in every one of the 2,499,543 unique flows: Bwd PSH Flags, Bwd URG Flags, Fwd Avg Bytes/Bulk, Fwd

Avg Packets/Bulk, Fwd Avg Bulk Rate, Bwd Avg Bytes/Bulk, Bwd Avg Packets/Bulk, Bwd Avg Bulk Rate. All six bulk-transfer statistics are always zero, which suggests that the bulk-detection logic of the flow-meter version used did not trigger, and the backward PSH and URG counters are never set. A feature that never changes cannot separate classes, so these features are removed. The training script detects constant features on the training split only and finds the same eight, leaving 69 features for the model.

Feature distributions. Fig. 4.8 compares benign and malicious flows on six features. Values span many orders of magnitude, so they are plotted as log10(value + 1). Malicious flows last longer at the median, because slow DoS attacks and brute-force sessions keep connections open, and they have much larger variation in backward packet size, because attacked web servers return a mix of large pages and small error responses. The boxes overlap substantially, which shows that no single feature separates the classes; the model has to combine many features.

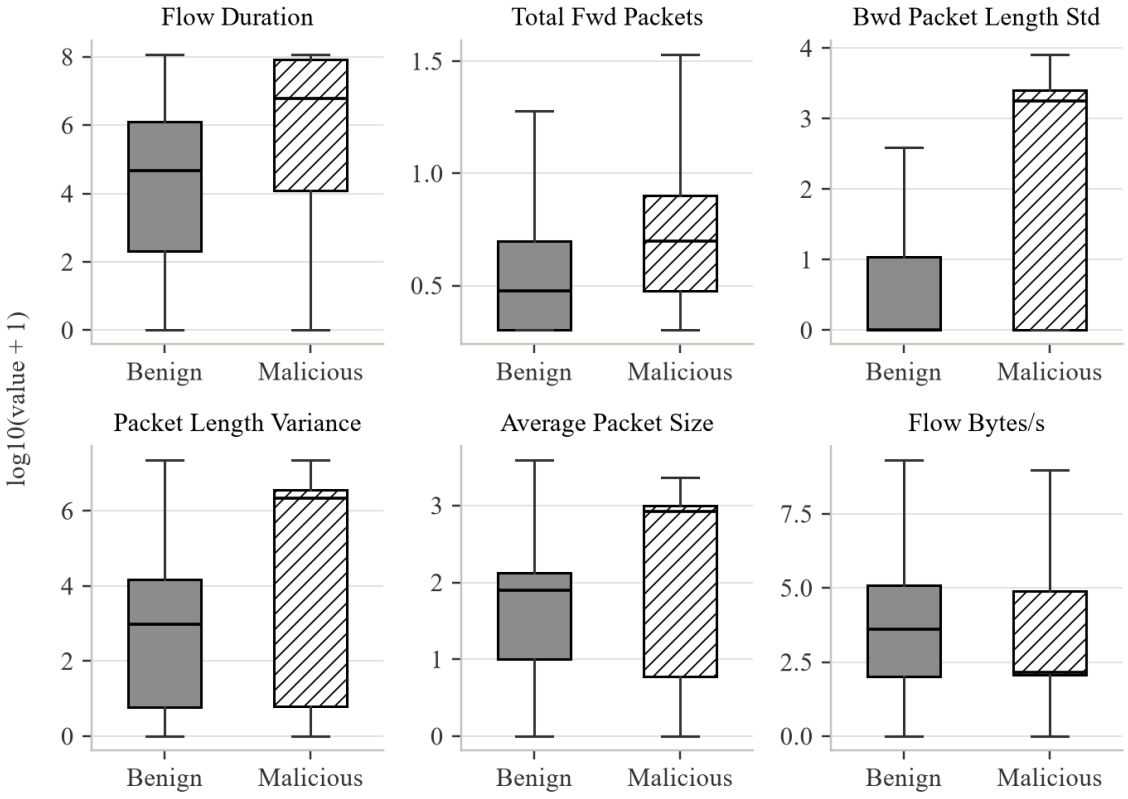


Fig. 4.8 Distribution of Six Flow Features for Benign and Malicious Flows

Table 4.5 Median and Mean of Key Features by Class (Unique Flows)

Feature	Benign median	Benign mean	Malicious median	Malicious mean
Flow Duration	46,172.0	12,279,842.3	6,027,848.0	38,383,902.3
Total Fwd Packets	2.0	11.5	4.0	4.9
Total Backward Packets	2.0	13.2	5.0	4.1
Bwd Packet Length Std	0.0	135.0	1,757.6	1,571.7
Average Packet Size	78.5	131.9	829.5	612.0
Flow Bytes/s	4,147.3	1,681,239.4	143.3	106,220.9
Flow Packets/s	84.5	53,833.0	1.1	16,459.7
Init_Win_bytes_forward	123.0	7,344.5	274.0	7,177.3

The large gap between median and mean in Table 4.5 shows heavily skewed distributions. For benign flows the median duration is 46.2 ms but the mean is 12.3 s, because most flows are short DNS lookups while a few long sessions dominate the mean. Fig. 4.9 shows the same effect as a cumulative distribution: more than half of the benign flows finish within 50 ms, whereas more than half of the malicious flows last longer than five seconds. Skewed, heavy-tailed features favour tree-based models, which split on thresholds and are unaffected by scale, over linear models that assume roughly linear effects.

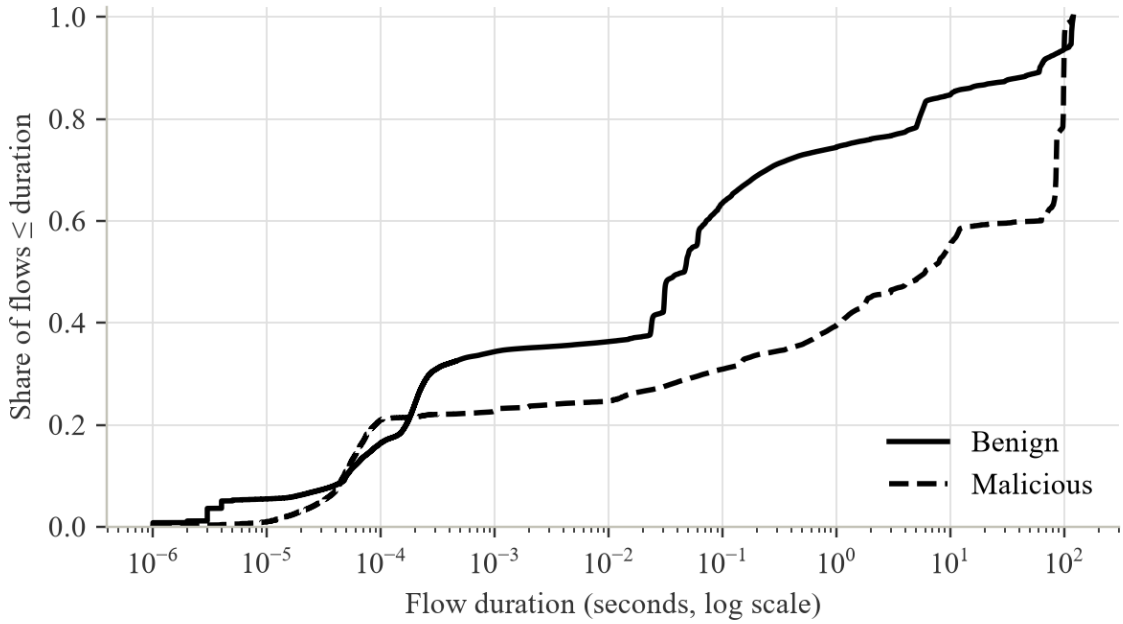


Fig. 4.9 Cumulative Distribution of Flow Duration by Class

Per-attack traffic profiles. Each attack leaves a distinctive network signature. Table 4.6 lists median values per label, computed on unique flows.

Table 4.6 Median Traffic Profile of Each Label

Label	Duration (s)	Fwd pkts	Bwd pkts	Bytes/s	Top port (share)
BENIGN	0.046	2	2	4,147.3	53 (41%)
DoS Hulk	86.401	6	6	123.8	80 (100%)
DDoS	1.879	4	4	160.1	80 (100%)
PortScan	0.000	1	1	137,931.0	many
DoS GoldenEye	11.588	7	5	651.0	80 (100%)
FTP-Patator	8.686	9	15	33.9	21 (100%)
DoS slowloris	99.999	3	2	0.2	80 (100%)
DoS Slowhttptest	63.120	7	0	0.0	80 (100%)
SSH-Patator	12.030	21	32	389.4	22 (100%)
Bot	0.071	3	3	5,013.1	8080 (64%)
Web Attack – Brute Force	5.568	3	1	0.0	80 (100%)
Web Attack – XSS	5.399	3	1	0.0	80 (100%)
Infiltration	93.189	26	26	173.4	444 (100%)
Web Attack – Sql Injection	5.006	4	3	210.2	80 (100%)
Heartbleed	119.261	2,792	2,069	66,172.2	444 (100%)

The profiles match how the tools work. PortScan flows last 50 µs and contain one packet in each direction (a probe and a reset), spread over thousands of ports, so no port holds even 5% of them. DoS slowloris holds connections open for about 100 seconds at 0.16 bytes per second, and DoS Slowhttptest flows typically receive no backward packet at all, which is the essence of a slow attack: occupying server resources while sending almost nothing. Brute-force sessions exchange tens of packets with the FTP or SSH server, and Heartbleed flows are long TLS sessions with thousands of packets in both directions. These distinct signatures explain why most attacks are easy to detect, and the labels whose profiles resemble benign traffic, such as Bot, are where errors concentrate.

Destination ports. The unique flows use 53,805 distinct destination ports. Fig. 4.10 shows the ten most frequent ports for each class.

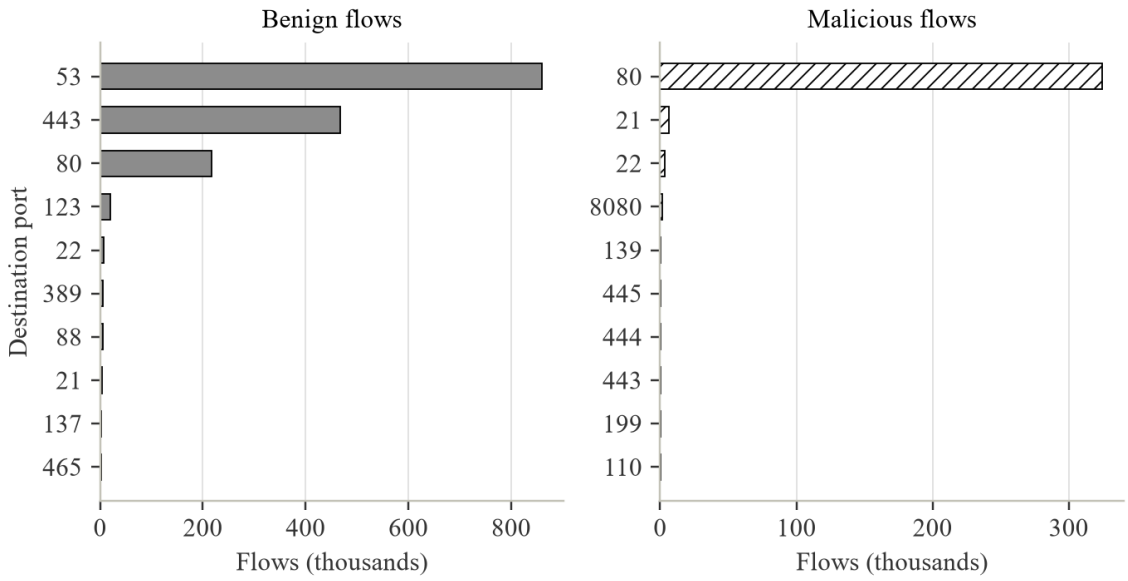


Fig. 4.10 Ten Most Frequent Destination Ports for Benign and Malicious Flows

Benign traffic is dominated by DNS (port 53, 859,553 flows), HTTPS (443) and HTTP (80), followed by NTP (123), SSH (22), LDAP (389) and Kerberos (88), which is typical of a Windows domain network. Malicious traffic is concentrated on port 80 (324,262 flows) because the DoS, DDoS and web attacks all target the web server, followed by FTP (21), SSH (22) and the botnet port 8080. Destination port is therefore informative, but it is not sufficient: benign HTTP traffic also uses port 80, so the model must look at how the connection behaves, not only where it goes.

Correlation and redundancy. Linear dependence between features was measured with the Pearson correlation coefficient (Eqn. 4.4) on a random sample of 200,000 unique flows, after removing constant features and filling missing values with medians.

$$r_{ab} = \frac{\sum_{i=1}^{n}(a_i - \bar{a})(b_i - \bar{b})}{\sqrt{\sum_{i=1}^{n}(a_i - \bar{a})^2}\sqrt{\sum_{i=1}^{n}(b_i - \bar{b})^2}} \tag{4.4}$$

44 feature pairs have $|r| \geq 0.95$, and 8 pairs are perfectly correlated ($|r| = 1$). Table 4.7 lists the strongest pairs and Fig. 4.11 shows the correlation among the fifteen features the model finds most important.

Table 4.7 Most Strongly Correlated Feature Pairs

Feature A	Feature B	r

Feature A	Feature B	r
Total Fwd Packets	Subflow Fwd Packets	1.0000
Total Backward Packets	Subflow Bwd Packets	1.0000
Total Length of Fwd Packets	Subflow Fwd Bytes	1.0000
Fwd Packet Length Mean	Avg Fwd Segment Size	1.0000
Bwd Packet Length Mean	Avg Bwd Segment Size	1.0000
Fwd PSH Flags	SYN Flag Count	1.0000
Fwd URG Flags	CWE Flag Count	1.0000
Total Length of Bwd Packets	Subflow Bwd Bytes	1.0000
Total Fwd Packets	Total Backward Packets	0.9987
Total Fwd Packets	Subflow Bwd Packets	0.9987
Total Backward Packets	Subflow Fwd Packets	0.9987
Subflow Fwd Packets	Subflow Bwd Packets	0.9987

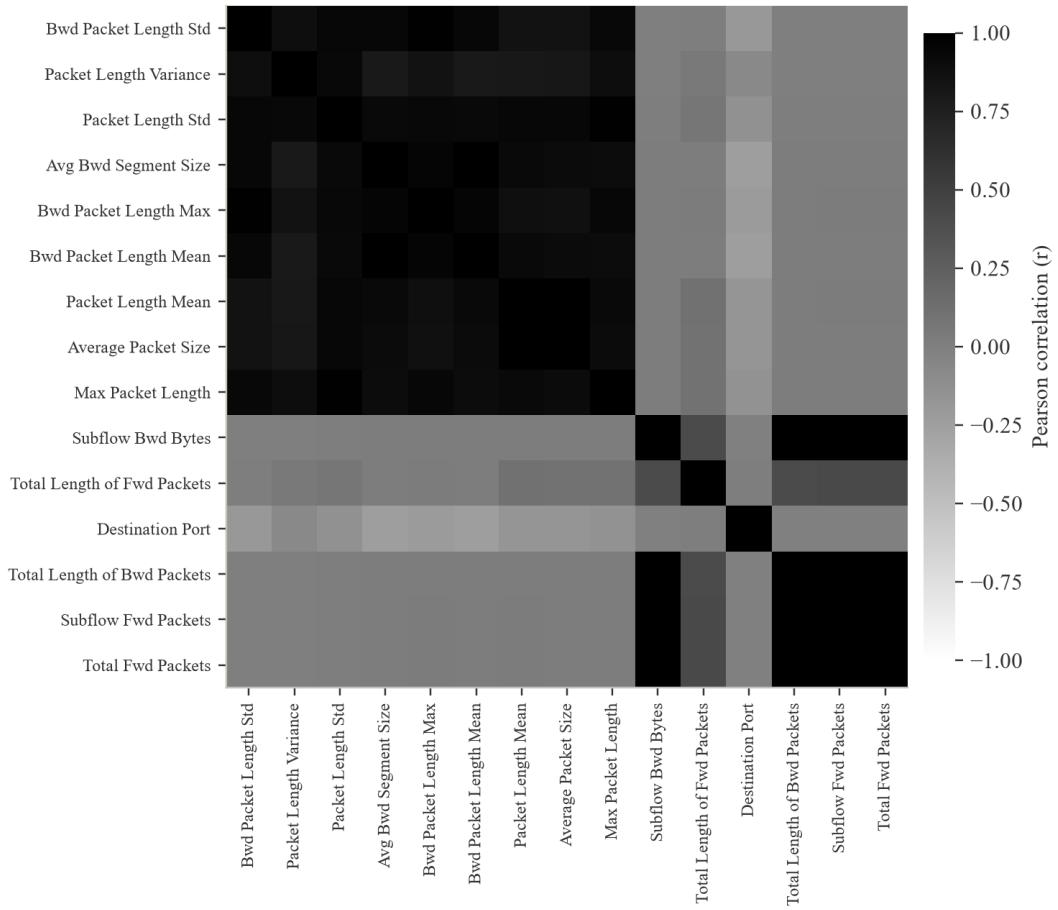


Fig. 4.11 Pearson Correlation Among the Fifteen Most Important Features

The perfect correlations reveal redundancy built into the feature set. The sub-flow features equal the corresponding totals because CICFlowMeter found one sub-flow per flow; the average segment sizes equal the mean packet lengths; and, surprisingly, Fwd PSH Flags equals SYN Flag Count in every row, which indicates that the flow meter

stored the wrong counter in one of these columns. Redundant features do not harm a Random Forest's predictions, but they split the importance score between copies, so the importance ranking in Chapter 7 is read at the level of feature groups. They were kept to preserve compatibility with standard CIC-IDS2017 files.

4.1.6 Data preprocessing pipeline

The findings above define the preprocessing steps shown in Fig. 4.12. Steps 1 to 4 run once on the full dataset in ml/train.py; steps 5 to 7 build the training data; step 8 lives inside the saved scikit-learn Pipeline and is therefore applied automatically to every uploaded file.

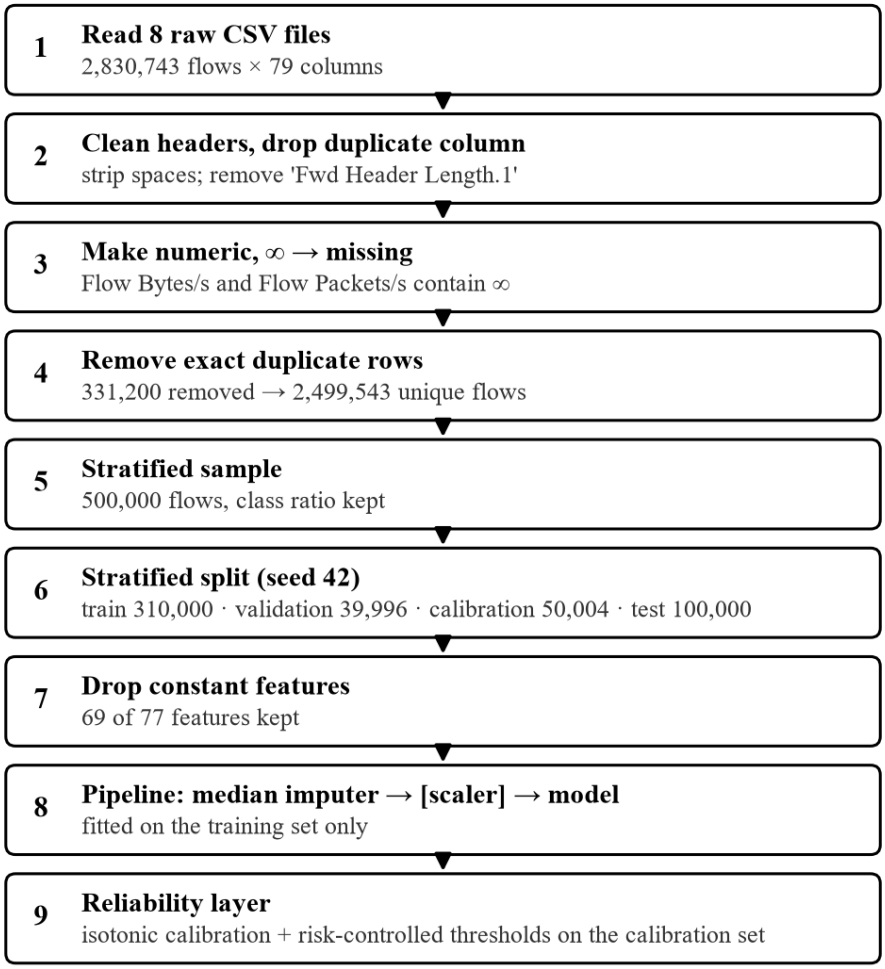


Fig. 4.12 Preprocessing Pipeline With Row Counts at Each Step

- ✓ **Header cleaning.** Column names in the files start with a space (" Flow Duration"), so names are stripped. The duplicated Fwd Header Length column, which pandas renames to Fwd Header Length.1, is dropped.
- ✓ **Type conversion.** Every feature is converted to a number; text that cannot be converted becomes missing, and +∞ and -∞ become missing. Features are stored as 32-bit floats, halving memory use.
- ✓ **Label mapping.** Labels are stripped and mapped to two classes with Eqn. 4.5.
- ✓ **Deduplication.** Each row (all features and the label) is hashed to a 64-bit value with pandas.util.hash_pandas_object and only the first row of each hash is kept. Hashing 2.8 million rows takes seconds, whereas pandas' column-by-column drop_duplicates had not finished after nine minutes in an early version of the pipeline. With 2.8 million rows, the probability of two different rows sharing a 64-bit hash is about 2×10^{-7}, which is negligible.

$$y_i = \{0 \text{ if label}_i = \text{BENIGN};\ 1 \text{ otherwise} \tag{4.5}$$

Missing values that remain after cleaning are replaced by the median of the training data (Eqn. 4.6). The median is used instead of the mean because the features are heavily skewed (Table 4.5), so the mean would be pulled towards rare extreme values. For Logistic Regression the features are additionally standardised (Eqn. 4.7), because that model is sensitive to the very different scales of, for example, Destination Port and Flow Duration.

$$x'_{ij} = \{\text{median}(x_j \mid \text{train}) \text{ if } x_{ij} \text{ is missing};\ x_{ij} \text{ otherwise} \tag{4.6}$$

$$z_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j} \tag{4.7}$$

The median, mean and standard deviation in these equations are learned with fit() on the training split only and then applied with transform() to the validation set, the test set and every uploaded file. Fitting them on all data would leak information about the test set into the model.

4.1.7 Sampling and data splitting

Training on all 2,499,543 unique flows would need far more than the 7 GB of memory available, so the pipeline draws a stratified random sample of 500,000 flows. Stratification keeps the benign/malicious ratio of the sample equal to that of the full data, as expressed in Eqn. 4.8, where N_c is the number of flows of class c.

$$\frac{N_c^{\text{sample}}}{N^{\text{sample}}} \approx \frac{N_c}{N}, \quad c \in \{benign, malicious\} \tag{4.8}$$

Fig. 4.13 compares the share of every label in the sample with its share in the cleaned dataset. For the large labels the two agree closely, but the sample is stratified on the binary class, not on the attack label, so very rare labels are kept only by chance: the sample contains 4 of the 11 Heartbleed flows, 9 of the 36 Infiltration flows and 1 of the 21 SQL injection flows.

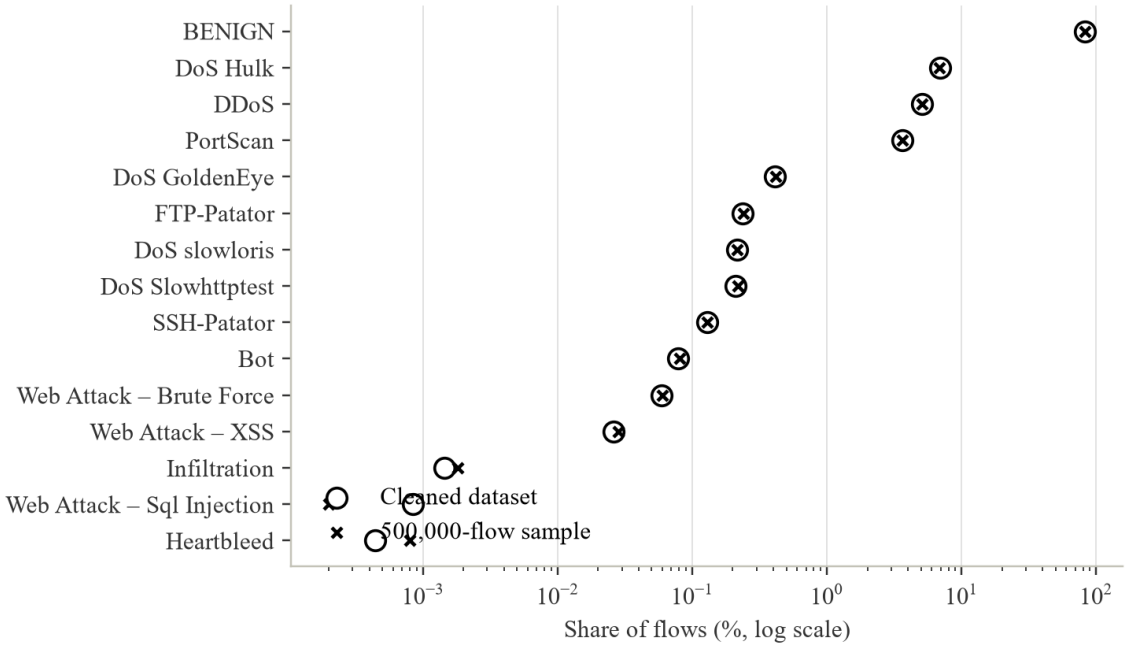


Fig. 4.13 Share of Each Label in the Cleaned Dataset and in the Training Sample

The sample is then split in three stratified stages with random_state = 42: 20% is set aside as the test set; 22.5% of the remainder is set aside and divided almost equally into a validation set and a calibration set. The result is 310,000 training, 39,996 validation, 50,004 calibration and 100,000 test flows (62% / 8% / 10% / 20%). Table 4.8 shows the class balance of each part.

Table 4.8 Composition of the Training, Validation, Calibration and Test Sets

Set	Flows	Benign	Malicious	Malicious share
Training	310,000	257,184	52,816	17.04%
Validation	39,996	33,182	6,814	17.04%
Calibration	50,004	41,484	8,520	17.04%
Test	100,000	82,962	17,038	17.04%

All four sets have the same malicious share to three decimal places, which confirms the stratification. Their roles are strictly separated. The training set fits the models. The validation set is used for one decision, choosing which model to deploy. The calibration set fits the reliability layer (Section 4.1.11). The test set is used only to report the final scores, so it estimates the performance on unseen traffic. As a reproducibility check, the dataset analysis script rebuilds the sample and split independently and runs the saved model on the rebuilt test set. It reproduces the stored confusion matrix of the deployed Random Forest exactly. The Decision Tree and Logistic Regression refitted on the rebuilt training set differ from the stored results by at most six flows, which is attributable to a different library build; the per-attack figures for those two models in Chapter 7 therefore come from the refitted models.

Table 4.9 shows how the rare attack labels were distributed over the four sets.

Table 4.9 Rare Attack Labels in Each Set

Label	Train	Validation	Calibration	Test
Bot	260	27	44	74
Web Attack – Brute Force	192	23	29	54
Web Attack – XSS	89	10	14	28
Infiltration	4	0	0	5
Heartbleed	0	1	0	3
Web Attack – Sql Injection	1	0	0	0

The table exposes an important property of the data. The training set contains 0 Heartbleed, 4 Infiltration and 1 SQL injection flows, and SQL injection does not appear in the test set at all. No classifier can learn a reliable pattern from a handful of examples, or from none, so detection of these labels in Chapter 7 must be interpreted with this in mind. Stratifying on the attack label, or keeping every flow of the rarest labels, would reduce the problem; this is discussed as a recommendation in Chapter 8.

4.1.8 Evaluation protocols: random versus temporal

A random split mixes flows from all five days, so training and test flows come from the same hours of the same attacks. That answers the question "can the model recognise more flows of attacks it has seen?" but not "will it work tomorrow?". To answer the second question, ml/experiments.py repeats the training under three protocols, defined in Table 4.10.

Table 4.10 Evaluation Protocols

Protocol	Training data	Calibration data	Test data	Test flows
Random (all days)	Random 62% of the sample, all days	Random 10%	Random 20%	100,000
Random (Mon–Wed)	Random 62% of Mon–Wed flows	Random 10%	Random 20% of Mon–Wed	100,000
Temporal	Monday–Wednesday	Thursday morning	Thursday afternoon and Friday	61,989

The temporal protocol is the realistic one: the model is trained on the first three days and then faces later days, which contain attack types that never occurred during training (PortScan, Bot, Infiltration, DDoS). Its test set is capped at 20,000 flows per label to keep the large DDoS and PortScan classes from dominating, while every flow of the rare labels is kept. The deployed model uses the random protocol so that the dashboard reflects the standard benchmark; the temporal results are reported in Section 7.6.

4.1.9 Machine learning models

Three classifiers of increasing complexity were compared, all implemented with scikit-learn [24]. Each is wrapped in a Pipeline together with its preprocessing, so the whole chain is saved and reused as one object.

Decision Tree. A decision tree [13], [25] splits the data recursively. At each node it chooses the feature and threshold that make the two resulting groups as pure as possible. Purity is measured with the Gini impurity (Eqn. 4.9), where p_k is the share of class k among the N samples at a node; a node with only one class has G = 0. The chosen split maximises the impurity decrease (Eqn. 4.10), where L and R are the left and right children. The tree grows until its leaves are pure, which makes it accurate on training data but prone to overfitting.

$$G = 1 - \sum_{k=1}^{K} {p_k}^2 \tag{4.9}$$

$$\Delta G = G_{\text{parent}} - \frac{N_L}{N} G_L - \frac{N_R}{N} G_R \tag{4.10}$$

Random Forest. A Random Forest [12] trains T decision trees, each on a bootstrap sample of the training data (rows drawn with replacement), and at every split considers only a random subset of m features (Eqn. 4.12). The randomness makes the trees different from each other, so their errors partly cancel. The forest's malicious probability is the average of the trees' votes (Eqn. 4.11), and a flow is classified as malicious when this probability is at least 0.5.

$$P(malicious \mid x) = \frac{1}{T} \sum_{t=1}^{T} h_t(x) \tag{4.11}$$

$$m = \lfloor \sqrt{p} \rfloor = \lfloor \sqrt{69} \rfloor = 8 \tag{4.12}$$

Here $h_t(x)$ is the prediction of tree t (strictly, scikit-learn averages each tree's class probability), and p = 69 is the number of features. The forest also measures feature importance as the mean decrease in impurity (Eqn. 4.13): the total Gini decrease contributed by splits on a feature, averaged over all trees and normalised to sum to one.

$$\text{Imp}_j = \frac{1}{T} \sum_{t=1}^{T} \sum_{s \in t:\, feature(s) = j} \frac{N_s}{N} \Delta G_s \tag{4.13}$$

Logistic Regression. Logistic Regression [26] is a linear model. It computes a weighted sum of the standardised features and passes it through the sigmoid function to obtain a probability (Eqn. 4.14). The weights are learned by minimising the log-loss (Eqn. 4.15). Because its decision boundary is a hyperplane, it serves as a baseline that shows how much the non-linear tree models gain.

$$P(y = 1 \mid x) = \sigma(w^T x + b) = \frac{1}{1 + e^{-(w^T x + b)}} \tag{4.14}$$

$$L(w, b) = -\frac{1}{n} \sum_{i=1}^{n} [y_i \, log \, p_i + (1 - y_i) \, log(1 - p_i)] \tag{4.15}$$

Table 4.11 Model Configurations

Model	Pipeline steps	Main hyper-parameters
Random Forest	SimpleImputer(median) → RandomForestClassifier	100 trees, max_features = √p, unlimited depth, n_jobs = −1, random_state = 42
Decision Tree	SimpleImputer(median) →	Gini criterion, unlimited depth,

Model	Pipeline steps	Main hyper-parameters
	DecisionTreeClassifier	random_state = 42
Logistic Regression	SimpleImputer(median) → StandardScaler → LogisticRegression	L2 penalty, C = 1.0, lbfgs solver, max_iter = 1000

Default hyper-parameters were used deliberately. The aim is a fair, explainable comparison rather than a tuned competition; tuning would also require additional validation data, and the models already reach very high scores. Class weights were not used because the binary imbalance (4.9 to 1) is moderate.

4.1.10 Evaluation metrics and model selection

Malicious is treated as the positive class. From the test-set confusion matrix, with TP (attacks correctly flagged), TN (benign flows correctly passed), FP (false alarms) and FN (missed attacks), the following metrics are computed:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \tag{4.16}$$

$$\text{Precision} = \frac{TP}{TP + FP} \tag{4.17}$$

$$\text{Recall} = \frac{TP}{TP + FN} \tag{4.18}$$

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}} \tag{4.19}$$

$$\text{FPR} = \frac{FP}{FP + TN} \tag{4.20}$$

Accuracy alone is misleading for imbalanced data: a model that labels every flow benign would reach 82.96% accuracy on this dataset while detecting nothing. Precision tells an analyst how many alerts are real, recall (the detection rate) how many attacks are caught, and F1 balances the two. The false positive rate matters operationally because even a small FPR produces many false alarms on a large benign volume. The ROC curve [27] and the precision–recall curve [28] summarise performance over all decision thresholds.

The model to deploy is chosen on the validation set using F1-score. Random Forest is the primary model of the project, so it is kept unless another model's validation F1 is higher by more than 0.001. This margin prevents a difference of one or two flows,

which is statistical noise, from changing the deployed model. The rule was added after the first training run showed a near-tie between Random Forest and Decision Tree, and it uses validation data only, so the test set plays no part in the choice.

4.1.11 Reliability layer

A classifier that outputs "malicious" whenever its vote fraction reaches 0.5 gives the operator no control over how many false alarms to expect, treats a 0.51 vote like a 0.99 vote, and has no way of saying that a whole batch looks unlike anything it was trained on. The reliability layer (ml/reliability.py) adds these controls on top of the selected Random Forest. All of it is fitted on the calibration set of 50,004 flows, which the model never trained on, and it is split into two disjoint halves of 25,002 flows so that the calibration map and the thresholds are not fitted on the same examples.

Calibration. The raw output of a Random Forest is a vote fraction, not a probability. Isotonic regression [29] learns a non-decreasing function g that maps the vote fraction s to the observed frequency of malicious flows on the first half of the calibration set (Eqn. 4.21). Because g is monotone it never changes the ranking of flows, only the scale of the scores.

$$g \;=\; \arg\min_{g\ \text{non-decreasing}} \sum_i \big(y_i \;-\; g(s_i)\big)^2 \tag{4.21}$$

Risk-controlled alert threshold. The operator chooses a false-positive budget α, the largest acceptable share of benign flows that may raise an alert; NetGuard uses α = 0.01. Following the Conformal Risk Control idea [30], the threshold t_{alert} is the lowest score at which the false positive rate on the n_0 benign flows of the second calibration half, corrected for finite samples, stays within α (Eqn. 4.22).

$$\frac{n_0}{n_0+1}\widehat{\mathrm{FPR}}\;\;(t_{\mathrm{alert}}) \;+\; \frac{1}{n_0+1} \;\le\; \alpha \tag{4.22}$$

With n_0 = 20,794 benign calibration flows, the chosen threshold is t_{alert} = 0.0182 on the calibrated scale, with an empirical false positive rate of 0.606% on that half. The guarantee holds only if future traffic resembles the calibration traffic, which is exactly what the temporal experiment in Section 7.6 tests.

Review band. Flows just below the alert threshold are the ones the model is least sure about. Instead of silently calling them benign, NetGuard sends flows with a calibrated score in $[t_{benign}, t_{alert})$ to a human review queue (Eqn. 4.23). The lower edge t_{benign} is the widest band whose share of traffic stays within the analyst's review capacity of 5%; the fitted value is $t_{benign} = 0.0116$.

$$\text{decision}(x) = \begin{cases} \text{Malicious if } g(s) \geq t_{alert}; \\ \text{Review if } t_{benign} \leq g(s) < t_{alert}; \\ \text{Benign otherwise} \end{cases} \quad (4.23)$$

Batch distribution-shift gate. Calibration and thresholds assume that new traffic comes from the same distribution as the training traffic. The shift gate checks this assumption for every uploaded batch. For each feature j it computes the two-sample Kolmogorov–Smirnov statistic [31] D_j, the largest gap between the cumulative distribution of the batch (n flows) and that of a reference sample of benign training flows (m flows), scaled so that its typical value does not depend on batch size (Eqn. 4.24). The batch score is the 90th percentile of the scaled statistics over all features, so a few strongly shifted features are enough to raise it.

$$S_j = \sup_v \left| \hat{F}_{batch}(v) - \hat{F}_{ref}(v) \right| \times \sqrt{\frac{n\,m}{n+m}} \quad (4.24)$$

The gate compares only the flows the model did not flag as malicious, because attacks are supposed to look different; it asks whether the *normal* traffic in the batch resembles the normal traffic the model learned. Its thresholds are set by bootstrapping 200 batches of 2,000 benign flows from single capture days: the 99th percentile (4.24) separates in-distribution from shifted batches and the 99.9th percentile (4.28) marks severe shift. When a batch is shifted the dashboard shows a warning and names the most affected features.

4.1.12 Backend design

The backend is a FastAPI [32] application (backend/app/main.py) created by a factory function, create_app(), which receives the model and metrics paths. The factory makes the application easy to test: the tests create an app that points at a small synthetic model in a temporary folder. A ModelStore object (backend/app/model_store.py) loads

the joblib bundle on first use, checks that it contains a pipeline, a feature list and a model name, and reloads it automatically if the file changes after retraining.

Every prediction request follows the same path. The upload is checked for a .csv extension, a size limit of 100 MB and non-empty content; it is parsed with pandas; column names are stripped; the prepare_features() function from ml/preprocess.py checks that all 69 model features are present and numeric and orders them as in training; the Pipeline returns raw vote fractions, which the reliability layer calibrates and turns into decisions; and the shift gate scores the batch. Table 4.12 lists the validation rules.

Table 4.12 Input Validation Rules

Check	Rule	Response
File type	Name must end in .csv	400
Size	At most 100 MB (the server stops reading after the limit)	413
Content	Not empty, parseable as CSV, at least one data row	400
Columns	All 69 required features present (extra columns ignored)	422 + list of missing columns
Values	Feature values numeric; blank, NaN and Infinity accepted as missing	422 + list of bad columns
Model	Saved model file present and loadable	503

The JSON response contains benign, review and malicious counts computed over every row, the shift verdict with the most affected features and a warning text when the batch is shifted, the first 5,000 rows for the dashboard table (five readable columns, the decision, the calibrated probability, the raw vote fraction and the actual label when present) and, if the file has a Label column, the agreement between predictions and labels together with the precision and recall of the alerts. The download endpoint returns the whole file with the prediction, decision and probability columns added, and neutralises text cells beginning with =, +, - or @ so that a spreadsheet cannot execute them as formulas (CSV injection). No row is lost however large the upload. Cross-origin requests from the Vite development server are allowed, and when the frontend has been built the backend also serves it, so the complete application can run on one port.

4.1.13 Frontend design

The dashboard is a single-page React [33] application written in TypeScript and styled with Tailwind CSS. It is laid out as a console with a navigation sidebar, a dark theme and Lucide icons. Table 4.13 lists its components.

Table 4.13 Frontend Components

Component	Responsibility
ConsoleSidebar	Navigation between traffic analysis, results and model evaluation
Header	Live model status, reliability and shift-gate state from /api/health, link to the API reference
UploadPanel	File picker with drag and drop, input requirements, Analyze and Clear buttons, validation errors
ShiftWarning	Banner with the batch shift verdict, score, limits and most affected features
SummaryCards	Flows analysed, benign, malicious and review counts, agreement with labels, composition bar
ResultsTable	Filter (all / benign / malicious / review / misclassified), pagination, probabilities, CSV download
ModelComparison	Test-set metrics table and grouped bar chart for the three models
ConfusionMatrix	2 × 2 matrix with counts and row percentages, model selector
DatasetInfo	Raw and used flow counts, duplicates removed, train / validation / test split
FeatureImportance	Top ten Random Forest features as a horizontal bar chart
api.ts	Typed API client with readable error messages for network and validation failures

Each decision (benign, review, malicious) has its own colour in the dashboard, but every coloured mark is accompanied by a text label or an icon, so the meaning never depends on colour alone. Each model keeps the same colour in every chart, bar charts start at zero so that bar lengths are honest, and exact values are always available in an adjacent table.

4.2 Codes and Standards

The project follows the protocol, data-format and software standards in Table 4.14.

Table 4.14 Standards Followed in the Project

Standard	Title	Where it applies
RFC 791	Internet Protocol [34]	Addressing and packet structure behind every flow record
RFC 9293	Transmission Control Protocol [3]	Handshake, flags and window fields used as features
RFC 768	User Datagram Protocol [4]	Explains the −1 window sentinel for UDP flows
RFC 3168	Explicit Congestion Notification [5]	ECE and CWR flag features
RFC 3954 / RFC 7011	NetFlow v9 / IPFIX [6], [7]	Flow-record concept used by CICFlowMeter
RFC 9110	HTTP Semantics [35]	REST API methods and status codes (400, 413, 422, 503)
RFC 7578	multipart/form-data [36]	CSV file upload
RFC 4180	CSV format [37]	Input files and downloaded results

Standard	Title	Where it applies
RFC 8259	JSON [38]	API responses and saved reports
OpenAPI 3.1	OpenAPI Specification [39]	Interactive API documentation generated at /docs
PEP 8	Style Guide for Python Code [40]	Python code layout and naming
WCAG 2.2	Web Content Accessibility Guidelines [41]	Contrast, non-colour cues, keyboard focus

4.3 Constraints, Alternatives and Tradeoffs

The design had to respect the following constraints:

- ✓ **Memory:** about 7 GB of RAM, which rules out loading and training on all 2.5 million unique flows at once; hence float32 storage and the 500,000-flow stratified sample.
- ✓ **Explainability:** the system must be explainable in a 20-minute viva, which favours classical models and a small, readable code base over deep learning.
- ✓ **Data access:** the official download requires a form, so a byte-identical Kaggle mirror was used and the row count was verified.
- ✓ **Scope:** no database, authentication or live capture, as required by the project brief.

Table 4.15 Design Alternatives and the Chosen Trade-Offs

Decision	Chosen	Alternative	Reason
Problem type	Binary (benign / malicious)	Multi-class attack type	Matches the project goal; rare classes make multi-class unreliable
Training data size	500k stratified sample	All 2.5M unique flows	Fits in memory and trains in minutes; results stable
Duplicates	Removed before splitting	Kept	Prevents train/test leakage and inflated attack scores
Missing values	Median imputation inside the Pipeline	Drop rows	Keeps every uploaded row predictable; robust to skew
Primary model	Random Forest	Gradient boosting, neural networks	Strong, fast, needs no scaling, gives feature importance
Model choice	Validation F1 with 0.001 margin	Best test score	Test set must not influence decisions
Alert threshold	Risk-controlled at $\alpha = 1\%$	Fixed 0.5 vote	Operator controls the false-alarm budget
Uncertain flows	Review band ($\leq 5\%$ of traffic)	Forced benign / malicious	Doubtful flows reach an analyst
Shift detection	Per-feature KS gate on benign traffic	None	Warns when the model is used outside its data
Deduplication method	64-bit row hashing	pandas drop_duplicates	Seconds instead of more than nine minutes
Serving	FastAPI + saved joblib	Retrain per request /	Fast, consistent, automatically

Decision	Chosen	Alternative	Reason
	Pipeline	notebook	documented
Frontend	React + Vite SPA	Server-rendered templates	Interactive table and charts, clean API boundary

The main trade-off is between completeness and practicality. Training on a sample loses some of the rare attack flows, and binary labels hide which attack was detected. In return the system is fast, fits on a laptop and is simple enough to explain completely, while the dataset analysis makes the cost of these choices visible instead of hiding it.

CHAPTER 5

SCHEDULE, TASKS AND MILESTONES

The project was planned as a twelve-week course project. Work was organised so that each stage produced a concrete artefact that the next stage depended on: the verified dataset, the analysis results, the trained model and report, the API, the dashboard and finally this document.

5.1 Tasks

Table 5.1 Project Tasks and Planned Weeks

No.	Task	Weeks	Output
1	Literature survey and dataset selection	W1–W2	Comparison of IDS datasets (Table 1.3)
2	Dataset download and verification	W2	8 CSV files, row count verified
3	Exploratory data analysis	W3–W4	Distributions, quality issues, duplicates
4	Preprocessing and deduplication	W4–W5	ml/preprocess.py, cleaning functions
5	Sampling, splitting and model training	W5–W6	ml/train.py, three trained models
6	Evaluation and model selection	W7	metrics.json, model_report.md
7	FastAPI backend and input validation	W7–W8	backend/app/
8	React dashboard	W8–W9	frontend/src/
9	Reliability layer and shift gate	W9–W10	ml/reliability.py
10	Temporal protocol experiments	W10	ml/experiments.py, experiments.json
11	Tests, integration and test files	W10–W11	38 passing tests, dataset/faculty_tests/
12	Dataset analysis and report	W11–W12	dataset_analysis.json, this report

5.2 Gantt chart

Fig. 5.1 shows the same plan as a Gantt chart. Several tasks overlap where they were independent.

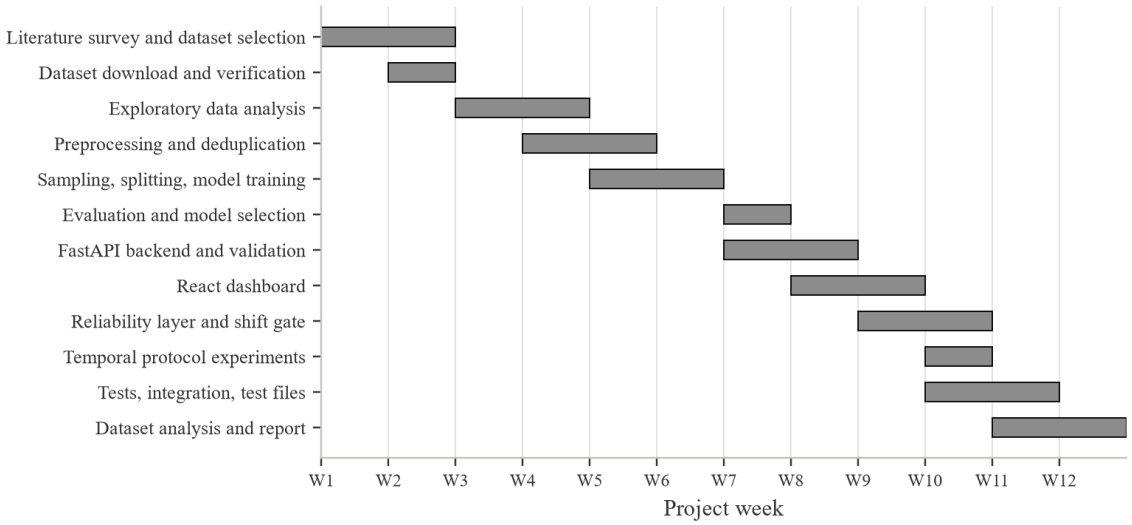


Fig. 5.1 Project Gantt Chart

5.3 Milestones

Table 5.2 Project Milestones

Milestone	Week	Completion criterion
M1 Dataset ready	W2	2,830,743 flows loaded, matching the official count
M2 Data understood	W4	Quality issues, duplicates and distributions documented
M3 Models trained	W7	Three models evaluated on the test set; Random Forest F1 > 99%
M4 API working	W8	Real predictions returned; invalid files rejected with clear errors
M5 Dashboard working	W9	Upload, results, download and evaluation views working end to end
M6 Reliable	W10	Alert threshold meets $\alpha = 1\%$ on the test set; shift gate flags synthetic shift
M7 Verified	W11	38 backend tests pass; frontend production build succeeds; browser test passes
M8 Submitted	W12	Report and demonstration complete

5.4 Risks and mitigation

Table 5.3 Project Risks and Mitigation

Risk	Impact	Mitigation
Official dataset download requires a form	Delay in obtaining data	Byte-identical Kaggle mirror; row count verified
Training exceeds available memory	Pipeline cannot run	float32 storage, 500k stratified sample; peak 2.8 GB measured
Slow deduplication	Long iteration cycle	Replaced column-wise comparison with 64-bit row hashing
Data leakage between train and test	Over-optimistic results	Deduplicate first; fit preprocessing on training data only
Training/inference mismatch	Wrong predictions in the app	Shared preprocess.py and a saved Pipeline with feature list

Risk	Impact	Mitigation
Results not reproducible	Report cannot be verified	Fixed seeds; analysis script re-checks the saved confusion matrices

CHAPTER 6

PROJECT DEMONSTRATION

This chapter shows the most important parts of the implementation and the working application. The code excerpts are taken verbatim from the project repository; the screenshots were captured from the running application analysing real CIC-IDS2017 flows from the held-out test set.

6.1 Sample Codes

6.1.1 Shared preprocessing (ml/preprocess.py)

prepare_features() is called both by the training pipeline's callers and by the API. It strips the column names, rejects files with missing or non-numeric feature columns and returns the features in the exact order used in training, with infinities converted to missing values.

```
def clean_column_names(df: pd.DataFrame) -> pd.DataFrame:
    """Strip spaces from headers and drop the known duplicate column."""
    df = df.rename(columns=lambda c: str(c).strip())
    return df.drop(columns=[c for c in DUPLICATE_COLUMNS if c in df.columns])


def to_binary_labels(labels: pd.Series) -> np.ndarray:
    """Map 'BENIGN' -> 0 and every attack type -> 1 (Malicious)."""
    cleaned = labels.astype(str).str.strip().str.upper()
    return (cleaned != BENIGN_LABEL).astype(np.int8).to_numpy()


def prepare_features(df: pd.DataFrame, feature_names: list[str]) -> pd.DataFrame:
    """Validate a raw DataFrame and return only the model's numeric features.

    Raises FeatureValidationError if columns are missing or contain text that
    cannot be interpreted as a number.
    """
    df = clean_column_names(df)

    missing = [name for name in feature_names if name not in df.columns]
    if missing:
        raise FeatureValidationError(
            f"CSV is missing {len(missing)} required feature column(s).", missing
        )

    features = pd.DataFrame(index=df.index)
    bad_columns = []
    for name in feature_names:
        column = df[name]
        numeric = pd.to_numeric(column, errors="coerce")
        # A value is "bad" if it was present in the file but is not a number.
        if (numeric.isna() & column.notna() & (column.astype(str).str.strip() !=
"")).any():
```

```
            bad_columns.append(name)
        features[name] = numeric

    if bad_columns:
        raise FeatureValidationError(
            f"{len(bad_columns)} feature column(s) contain non-numeric values.",
bad_columns
        )

    features = features.astype(np.float64).replace([np.inf, -np.inf], np.nan)
    return features
```

6.1.2 Splitting protocols (ml/train.py)

_split_frame() makes the stratified 62/8/10/20 split; build_splits() applies it under the random or temporal protocol.

```
def _split_frame(frame: pd.DataFrame, seed: int) -> dict[str, pd.DataFrame]:
    """Split one pool into train / validation / calibration / test (62/8/10/20)."""
    y = to_binary_labels(frame[config.LABEL_COLUMN])
    rest, test = train_test_split(
        frame, test_size=0.20, stratify=y, random_state=seed)
    rest_y = to_binary_labels(rest[config.LABEL_COLUMN])
    train, tmp = train_test_split(
        rest, test_size=0.225, stratify=rest_y, random_state=seed)
    tmp_y = to_binary_labels(tmp[config.LABEL_COLUMN])
    val, cal = train_test_split(
        tmp, test_size=0.5556, stratify=tmp_y, random_state=seed)
    return {"train": train, "validation": val, "calibration": cal, "test": test}


def build_splits(data: pd.DataFrame, protocol: str,
                 temporal_test_cap: int = 20000) -> tuple[dict, dict]:
    """Return the four splits plus descriptive metadata for a protocol."""
    seed = config.RANDOM_STATE

    if protocol == "original":
        splits = _split_frame(data, seed)
        meta = {"protocol": "original", "description": "random split over all
capture days"}

    elif protocol == "random_monwed":
        pool = data[data[DAY_COLUMN].isin(_MONWED)]
        splits = _split_frame(pool, seed)
        meta = {"protocol": "random_monwed",
                "description": "random split restricted to Monday-Wednesday"}

    elif protocol == "temporal":
        pool = data[data[DAY_COLUMN].isin(_MONWED)]
        base = _split_frame(pool, seed)
        calibration = data[data[DAY_COLUMN] == "Thursday-Morning"]
        test = data[data[DAY_COLUMN].isin(
            {"Thursday-Afternoon", "Friday-Morning", "Friday-PortScan", "Friday-
DDoS"})]
        splits = {
            "train": base["train"],
            "validation": base["validation"],
            "calibration": calibration,
            "test": cap_per_class(test, temporal_test_cap),
        }
        meta = {
            "protocol": "temporal",
```

```
            "description": ("train Monday-Wednesday, calibrate Thursday morning, "
                            "test Thursday afternoon + Friday"),
            "test_cap_per_class": temporal_test_cap,
        }
    else:
        raise ValueError(f"unknown protocol: {protocol}")

    meta["sizes"] = {k: int(len(v)) for k, v in splits.items()}
    meta["days"] = {
        name: {d: int((part[DAY_COLUMN] == d).sum())
               for d in sorted(part[DAY_COLUMN].unique())}
        for name, part in splits.items()
    }
    return splits, meta
```

6.1.3 Model definitions and selection (ml/train.py)

Each model is a Pipeline, so the imputer (and scaler) is saved with it. select_model() applies the validation-set rule.

```
def build_models() -> dict[str, Pipeline]:
    """Each model is a Pipeline so preprocessing is saved together with it."""
    return {
        "Random Forest": Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("model", RandomForestClassifier(
                n_estimators=100, n_jobs=-1, random_state=config.RANDOM_STATE
            )),
        ]),
        "Decision Tree": Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("model", DecisionTreeClassifier(random_state=config.RANDOM_STATE)),
        ]),
        "Logistic Regression": Pipeline([
            ("imputer", SimpleImputer(strategy="median")),
            ("scaler", StandardScaler()),  # LR needs features on a similar scale
            ("model", LogisticRegression(max_iter=1000)),
        ]),
    }


def evaluate(y_true: np.ndarray, y_pred: np.ndarray) -> dict:
    """Standard binary metrics with Malicious (1) as the positive class."""
    return {
        "accuracy": float(accuracy_score(y_true, y_pred)),
        "precision": float(precision_score(y_true, y_pred, zero_division=0)),
        "recall": float(recall_score(y_true, y_pred, zero_division=0)),
        "f1_score": float(f1_score(y_true, y_pred, zero_division=0)),
        # [[TN, FP], [FN, TP]] - rows = actual, columns = predicted
        "confusion_matrix": confusion_matrix(y_true, y_pred, labels=[0,
1]).tolist(),
    }


def select_model(results: list[dict]) -> dict:
    """Pick the model to deploy using the validation set (never the test set).

    Random Forest is the primary model. A baseline replaces it only when its
    validation F1 is higher by more than SELECTION_MARGIN, so a difference of
    one or two flows (noise) does not change the deployed model.
    """
    by_name = {r["name"]: r for r in results}
    primary = by_name[config.PRIMARY_MODEL]
    challenger = max(results, key=lambda r: r["validation_f1"])
```

```
    if challenger["validation_f1"] - primary["validation_f1"] >
config.SELECTION_MARGIN:
        return challenger
    return primary
```

6.1.4 Risk-controlled threshold and three-way decision (ml/reliability.py)

crc_threshold() picks the alert threshold for the false-positive budget α; decide() maps calibrated scores to Benign, Review or Malicious.

```
def crc_threshold(probabilities: np.ndarray, labels: np.ndarray, alpha: float) ->
dict:
    """Choose an alert threshold whose expected false-positive rate is <= alpha.

    Only negatives constrain the threshold. Returns the threshold plus the
    diagnostic information needed to report honestly when the budget is too tight
    for the available calibration data.

    The rule is: allow at most ``k = floor(alpha * (n0 + 1))`` false positives on
    the calibration negatives, where ``n0`` is their count. The threshold is then
    just above the ``k``-th largest negative score.
    """
    probabilities = np.asarray(probabilities, dtype=float)
    labels = np.asarray(labels, dtype=int)

    if not 0.0 < alpha < 1.0:
        raise ValueError("alpha must be in (0, 1)")

    negatives = probabilities[labels == 0]
    n0 = int(negatives.size)
    if n0 == 0:
        raise ValueError("calibration set contains no benign flows")

    # CRC condition: (n0/(n0+1)) * FPR_hat + 1/(n0+1) <= alpha
    target = float(alpha - (1.0 - alpha) / n0)

    threshold = float(np.nextafter(1.0, 2.0))
    empirical_fpr = 0.0
    feasible = False

    if target > 0:
        # Ties matter: isotonic calibration maps many raw scores to the same
        # probability, so "the k-th largest score" can admit far more than k
        # false positives. Walk distinct values and take the smallest threshold
        # whose *realised* FPR still satisfies the budget.
        for candidate in np.unique(negatives):  # ascending
            fpr = float(np.count_nonzero(negatives >= candidate) / n0)
            if fpr <= target:
                threshold = float(candidate)
                empirical_fpr = fpr
                feasible = True
                break

    k = int(round(empirical_fpr * n0))
    return {
        "threshold": threshold,
        "alpha": float(alpha),
        "negatives": n0,
        "allowed_false_positives": int(k),
        "empirical_fpr": empirical_fpr,
```

```
        "feasible": bool(feasible),
        "minimum_supportable_alpha": float(2.0 / (n0 + 1)),
    }

def decide(probabilities: np.ndarray, t_benign: float, t_alert: float) ->
np.ndarray:
    """Three-way decision: 0 benign, 1 malicious, 2 review."""
    probabilities = np.asarray(probabilities, dtype=float)
    out = np.full(probabilities.shape, REVIEW, dtype=np.int8)
    out[probabilities < t_benign] = BENIGN
    out[probabilities >= t_alert] = MALICIOUS
    return out
```

6.1.5 Batch shift score (ml/reliability.py)

```
def _normalised_ks(batch_column: np.ndarray, reference_column: np.ndarray) ->
float:
    """Two-sample KS statistic scaled so its null distribution is ~size-invariant.

    Under the null, KS ~ sqrt(-ln(a/2) / (2 * n_eff)) with
    n_eff = n*m/(n+m); multiplying by sqrt(n_eff) removes the sample-size term.
    """
    n, m = batch_column.size, reference_column.size
    if n < 2 or m < 2:
        return 0.0
    stat = float(ks_2samp(batch_column, reference_column,
method="asymp").statistic)
    n_eff = (n * m) / (n + m)
    return stat * float(np.sqrt(n_eff))


def batch_shift_score(features: np.ndarray, profile: dict,
                      max_batch_size: int = 2000, seed: int = 0) -> tuple[float,
np.ndarray]:
    """Aggregate per-feature KS statistics into one batch-shift score.

    Very large batches are downsampled deterministically. Without this, a batch of
    100,000 flows makes the KS test sensitive to differences far too small to
    matter operationally: at that size almost any real batch looks "significant".

    Returns ``(score, per_feature_scores)``. A high aggregate means at least one
    feature's distribution differs materially from the reference.
    """
    features = np.asarray(features, dtype=float)
    reference = np.asarray(profile["reference"], dtype=float)
    if features.ndim != 2 or features.shape[1] != reference.shape[1]:
        raise ValueError(
            f"expected {reference.shape[1]} features, got "
            f"{features.shape[1] if features.ndim == 2 else 'a non-2D array'}"
        )

    if max_batch_size and features.shape[0] > max_batch_size:
        rng = np.random.default_rng(seed)
        take = rng.choice(features.shape[0], size=max_batch_size, replace=False)
        features = features[take]

    per_feature = np.zeros(features.shape[1], dtype=float)
    for j in range(features.shape[1]):
        column = features[:, j]
        column = column[np.isfinite(column)]
        if column.size < 2:
            continue
        reference_column = reference[:, j]
```

```
        reference_column = reference_column[np.isfinite(reference_column)]
        per_feature[j] = _normalised_ks(column, reference_column)

    score = float(np.quantile(per_feature, SHIFT_AGGREGATE_QUANTILE))
    return score, per_feature
```

6.1.6 Loading the model and predicting (backend/app/model_store.py)

```
class ModelStore:
    """Keeps the trained pipeline in memory and reloads it if the file changes."""

    def __init__(self, model_path: Path):
        self.model_path = Path(model_path)
        self._bundle: dict | None = None
        self._loaded_mtime: float | None = None

    def get(self) -> dict:
        if not self.model_path.exists():
            raise ModelNotAvailableError(
                "No trained model found. Run `python -m ml.train` first."
            )
        mtime = self.model_path.stat().st_mtime
        if self._bundle is None or mtime != self._loaded_mtime:
            try:
                bundle = joblib.load(self.model_path)
            except Exception as exc:  # corrupted or incompatible file
                raise ModelNotAvailableError(f"Model file could not be loaded:
{exc}") from exc
            if not {"pipeline", "feature_names", "model_name"} <= set(bundle):
                raise ModelNotAvailableError("Model file has an unexpected
format.")
            self._bundle, self._loaded_mtime = bundle, mtime
        return self._bundle

    def predict(self, raw: pd.DataFrame) -> dict:
        """Return predictions, calibrated probabilities, decisions and a shift
verdict.

        Keys
        ----
        ``predictions``   binary 0/1 (``calibrated >= 0.5``); kept for
compatibility
        ``probabilities`` calibrated probability of Malicious
        ``raw_probabilities`` the ensemble's raw vote fraction
        ``decisions``     0 benign / 1 malicious / 2 review
        ``reliability``   whether the layer is active, and its thresholds
        ``shift``         batch-level distribution verdict, or ``None``
        """
        bundle = self.get()
        features = prepare_features(raw, bundle["feature_names"])
        pipeline = bundle["pipeline"]

        raw_probabilities = np.asarray(
            pipeline.predict_proba(features)[:, 1], dtype=float)

        layer = bundle.get("reliability")
        if layer:
            probabilities = reliability.apply_calibration(
                layer.get("calibrator"), raw_probabilities)
            decisions = reliability.decide(
                probabilities, layer["t_benign"], layer["t_alert"])
            layer_info = {
```

```
                "active": True,
                "kind": layer.get("kind", "isotonic+crc"),
                "alpha": layer.get("alpha"),
                "t_alert": layer.get("t_alert"),
                "t_benign": layer.get("t_benign"),
                "review_capacity": layer.get("review_capacity"),
            }
        else:
            probabilities = np.clip(raw_probabilities, 0.0, 1.0)
            decisions = np.where(probabilities >= 0.5,
                                 reliability.MALICIOUS,
reliability.BENIGN).astype(np.int8)
            layer_info = {"active": False}

        predictions = np.where(probabilities >= 0.5,
                               reliability.MALICIOUS,
reliability.BENIGN).astype(int)

        shift = None
        profile = bundle.get("shift_profile")
        if profile:
            matrix = features.to_numpy(dtype=float)
            # Compare the traffic the model believes is benign against the benign
            # reference. Flagged flows are meant to look different, so including
            # them would make every batch with attacks look "shifted".
            benign_like = decisions != reliability.MALICIOUS
            basis = "non_alerted"
            if np.count_nonzero(benign_like) < 100:
                benign_like = np.ones(matrix.shape[0], dtype=bool)
                basis = "all_flows"
            score, per_feature = reliability.batch_shift_score(
                matrix[benign_like], profile)
            shift = {
                **reliability.shift_verdict(score, profile,
per_feature=per_feature),
                "basis": basis,
                "rows_compared": int(np.count_nonzero(benign_like)),
            }

        return {
            "predictions": predictions,
            "probabilities": probabilities,
            "raw_probabilities": raw_probabilities,
            "decisions": decisions,
            "reliability": layer_info,
            "shift": shift,
        }
```

6.1.7 Upload validation and the prediction endpoint (backend/app/main.py)

```
def read_csv_upload(upload: UploadFile) -> pd.DataFrame:
    """Validate the uploaded file and parse it into a DataFrame."""
    if not upload.filename or not upload.filename.lower().endswith(".csv"):
        raise HTTPException(status_code=400, detail="Please upload a file with a
.csv extension.")

    content = upload.file.read(MAX_UPLOAD_MB * 1024 * 1024 + 1)
    if len(content) > MAX_UPLOAD_MB * 1024 * 1024:
        raise HTTPException(status_code=413, detail=f"File is larger than
{MAX_UPLOAD_MB} MB.")
    if not content.strip():
        raise HTTPException(status_code=400, detail="The uploaded file is empty.")
```

```
    try:
        df = pd.read_csv(io.BytesIO(content), encoding="latin-1", low_memory=False)
    except (pd.errors.ParserError, pd.errors.EmptyDataError, ValueError) as exc:
        raise HTTPException(status_code=400, detail=f"Could not parse CSV: {exc}")
from exc

    if df.empty:
        raise HTTPException(status_code=400, detail="The CSV has a header but no
data rows.")
    return clean_column_names(df)

    @app.post("/api/predict")
    def predict(file: UploadFile = File(...)):
        df = read_csv_upload(file)
        result = predict_or_raise(df)

        predictions = result["predictions"]
        probabilities = result["probabilities"]
        raw_probabilities = result["raw_probabilities"]
        decisions = result["decisions"]

        total = int(len(decisions))
        benign = int(np.count_nonzero(decisions == reliability.BENIGN))
        malicious = int(np.count_nonzero(decisions == reliability.MALICIOUS))
        review = int(np.count_nonzero(decisions == reliability.REVIEW))

        labelled = has_labels(df)
        actual = to_binary_labels(df[config.LABEL_COLUMN]) if labelled else None

        display_columns = [c for c in DISPLAY_COLUMNS if c in df.columns]
        rows = []
        for i in range(min(total, MAX_TABLE_ROWS)):
            pred = int(predictions[i])
            decision = int(decisions[i])
            rows.append({
                "row": i + 1,
                "values": {c: json_value(df[c].iat[i]) for c in display_columns},
                "prediction": config.CLASS_NAMES[pred],
                "decision": reliability.DECISION_NAMES[decision],
                "malicious_probability": round(float(probabilities[i]), 4),
                "raw_vote_fraction": round(float(raw_probabilities[i]), 4),
                "actual": config.CLASS_NAMES[int(actual[i])] if labelled else None,
            })

        evaluation = None
        if labelled:
            correct = int((actual == predictions).sum())
            flagged = decisions == reliability.MALICIOUS
            tp = int(np.count_nonzero(flagged & (actual == 1)))
            fp = int(np.count_nonzero(flagged & (actual == 0)))
            fn = int(np.count_nonzero(~flagged & (actual == 1)))
            evaluation = {
                "accuracy": correct / total,
                "correct": correct,
                "actual_malicious": int(actual.sum()),
                "actual_benign": int(total - actual.sum()),
                # Metrics for the three-way operating point (review counts as no
alert).
                "alert_precision": tp / (tp + fp) if (tp + fp) else 0.0,
                "alert_recall": tp / (tp + fn) if (tp + fn) else 0.0,
                "review": review,
                "review_rate": review / total if total else 0.0,
            }
```

```
        warning = None
        shift = result["shift"]
        if shift and shift["verdict"] == "severely_shifted":
            warning = (
                "This batch is far outside the traffic distribution the model was "
                "trained on. Per-flow predictions are not reliable; treat them as "
                "unverified rather than as findings."
            )
        elif shift and shift["verdict"] == "moderately_shifted":
            warning = (
                "This batch differs noticeably from the training traffic. "
                "Check the shift report before acting on individual predictions."
            )

        return {
            "filename": file.filename,
            "model_name": store.get()["model_name"],
            "summary": {
                "total": total,
                "benign": benign,
                "malicious": malicious,
                "review": review,
                "malicious_ratio": malicious / total if total else 0.0,
                "review_ratio": review / total if total else 0.0,
                "benign_ratio": benign / total if total else 0.0,
            },
            "evaluation": evaluation,
            "reliability": {
                **result["reliability"],
                "counts": {"benign": benign, "malicious": malicious, "review":
review},
            },
            "shift": shift,
            "warning": warning,
            "display_columns": display_columns,
            "rows": rows,
            "rows_truncated": total > MAX_TABLE_ROWS,
        }
```

6.1.8 Frontend API client (frontend/src/api.ts)

The client turns HTTP errors into readable messages and keeps the list of offending columns for the error box.

```
async function request(url: string, init?: RequestInit): Promise<Response> {
  let response: Response;
  try {
    response = await fetch(url, init);
  } catch {
    throw new ApiError("Cannot reach the backend. Is the FastAPI server running on
port 8000?");
  }
  if (!response.ok) {
    let message = `Request failed (${response.status})`;
    let columns: string[] = [];
    try {
      const body = await response.json();
      if (typeof body.detail === "string") message = body.detail;
      else if (body.detail?.message) {
        message = body.detail.message;
        columns = body.detail.columns ?? [];
      }
    } catch {
```

```
      // Not JSON: usually the Vite proxy reporting that the backend is down.
      if (response.status >= 500) message = "Backend unavailable. Start it with:
uvicorn backend.app.main:app";
    }
    throw new ApiError(message, columns);
  }
  return response;
}
```

6.1.9 Automated tests (backend/tests/)

The tests use synthetic data, so they check the API and the reliability layer without needing the real dataset.

```
def test_missing_columns_are_rejected(client):
    flows = synthetic_flows(n=10).drop(columns=["Flow Duration", "Total Fwd
Packets"])
    response = upload(client, to_csv_bytes(flows))
    assert response.status_code == 422
    assert set(response.json()["detail"]["columns"]) == {"Flow Duration", "Total
Fwd Packets"}


def test_shifted_batch_raises_a_warning(reliable_client):
    """A wildly scaled batch must be flagged rather than silently labelled
benign."""
    flows = synthetic_flows(n=400, seed=12)
    for column in FEATURES:
        flows[column] = flows[column].astype(float) * 1000.0 + 1e6

    body = upload(reliable_client, to_csv_bytes(flows)).json()
    assert body["shift"]["verdict"] == "severely_shifted"
    assert body["warning"] is not None


def test_download_neutralises_spreadsheet_formulas(client):
    """A cell starting with '=' must not be written back as an executable
formula."""
    flows = synthetic_flows(n=5, seed=14)
    flows["Note"] = "=cmd|' /C calc'!A0"
    response = upload(client, to_csv_bytes(flows), path="/api/predict/download")
    result = pd.read_csv(io.StringIO(response.text))
    assert result["Note"].iloc[0].startswith("'=")

@pytest.mark.parametrize("alpha", [0.005, 0.01, 0.05])
def test_crc_threshold_holds_budget_on_unseen_data(alpha):
    raw, y = synthetic_scores(20000, seed=1)
    fit = R.fit_reliability(raw, y, alpha=alpha, review_capacity=0.05)

    test_raw, test_y = synthetic_scores(60000, seed=2)
    calibrated = R.apply_calibration(fit["calibrator"], test_raw)
    decisions = R.decide(calibrated, fit["t_benign"], fit["t_alert"])
    metrics = R.binary_metrics(test_y, (decisions == R.MALICIOUS).astype(int))

    # Allow a small slack for sampling noise; the point is that the budget is
    # respected in the right order of magnitude, not that it is exact.
    assert metrics["false_positive_rate"] <= alpha * 1.5
```

6.1.10 Per-attack evaluation (ml/analyze_dataset.py)

The dataset analysis rebuilds the split, verifies it against the saved results and computes the detection rate per attack label.

```
    # Sanity check: the rebuilt split must reproduce the saved evaluation exactly.
    reproduced = {}
    for m in metrics["models"]:
        cm = confusion_matrix(y_test, predictions[m["name"]], labels=[0,
1]).tolist()
        reproduced[m["name"]] = cm == m["confusion_matrix"]
    result["reproduces_saved_metrics"] = reproduced
    print(f"  reproduces saved confusion matrices: {reproduced}")

    # The deployed decision: calibrated probability, risk-controlled alert
threshold.
    layer = bundle["reliability"]
    calibrated = reliability.apply_calibration(layer["calibrator"], scores["Random
Forest"])
    decision = reliability.decide(calibrated, layer["t_benign"], layer["t_alert"])
    predictions["RF deployed alert"] = (decision ==
reliability.MALICIOUS).astype(int)
    result["deployed_review_by_label"] = {
        str(k): int(v) for k, v in pd.Series(labels_test[decision ==
reliability.REVIEW]).value_counts().items()}

    support = {k: int(v) for k, v in pd.Series(labels_test).value_counts().items()}
    per_label = {}
    for name, pred in predictions.items():
        per_label[name] = {}
        for label in support:
            mask = labels_test == label
            expected = 0 if label == "BENIGN" else 1
            per_label[name][label] = float((pred[mask] == expected).mean())
    result["test_support"] = support
    result["per_label_detection"] = per_label
```

6.1.11 Running the project

The complete project is run with the commands below, from the project folder.

```
# 1. Install dependencies
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
cd frontend && npm install && cd ..

# 2. Train the models (dataset in dataset/raw/), run the protocol
#    experiments and analyse the dataset
python -m ml.train
python -m ml.experiments
python -m ml.analyze_dataset

# 3. Start everything with ./start.sh, or the backend (port 8000)
#    and the frontend (port 5173) separately
uvicorn backend.app.main:app --port 8000
cd frontend && npm run dev

# 4. Run the tests
pytest
```

6.2 Sample Screen Shots

When the dashboard opens it calls /api/health and shows that the Random Forest with 69 features is loaded, that its probabilities are calibrated and that the shift gate is active. The upload panel lists the input requirements and accepts a CSV file by clicking or by drag and drop.

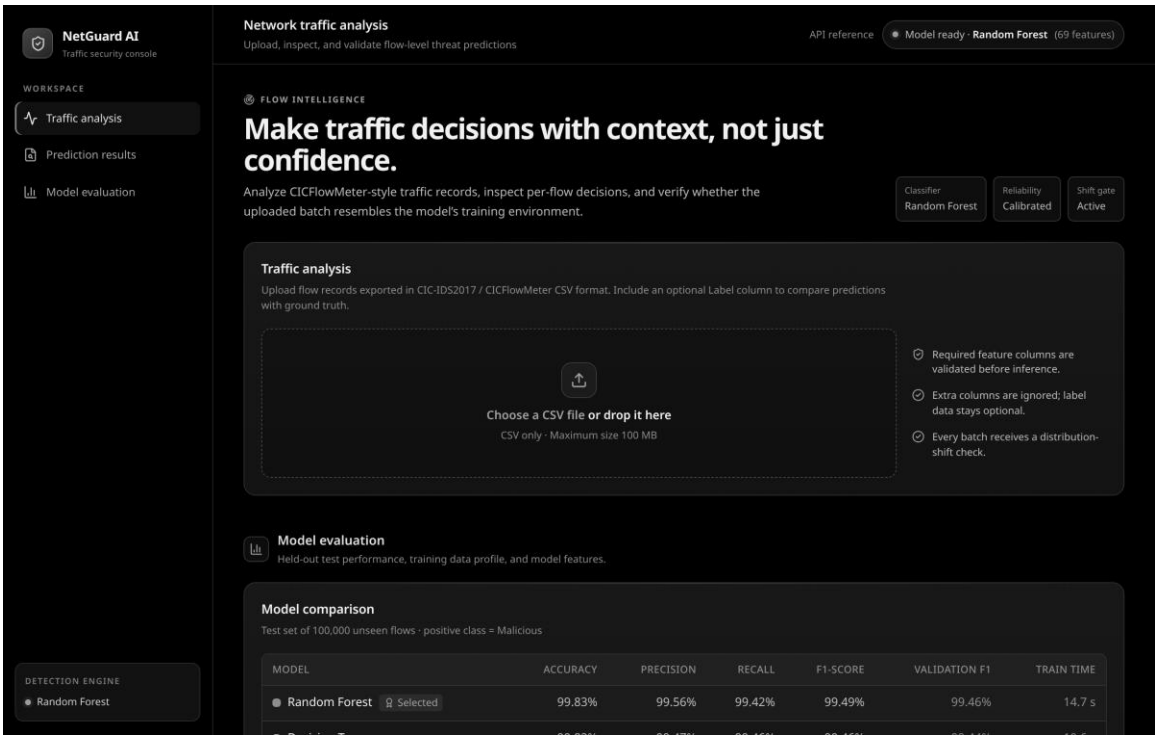


Fig. 6.1 Dashboard on Start-Up: Navigation, Model Status and Upload Panel

After demo_test_flows.csv is analysed, the summary cards show the benign, malicious and review counts and, because the file contains labels, how many predictions agree with them.

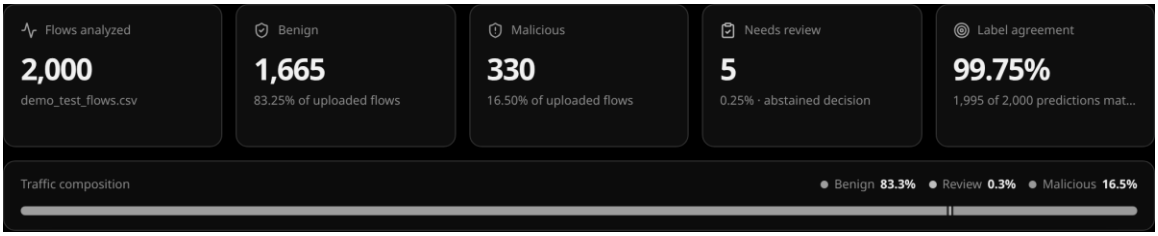


Fig. 6.2 Analysis Summary for the 2,000-Flow Demonstration File



Fig. 6.3 Shift Verdict for an In-Distribution Batch

The results table shows readable flow features, the decision, the calibrated probability and the actual label. The Download CSV button returns the full file with the new columns.

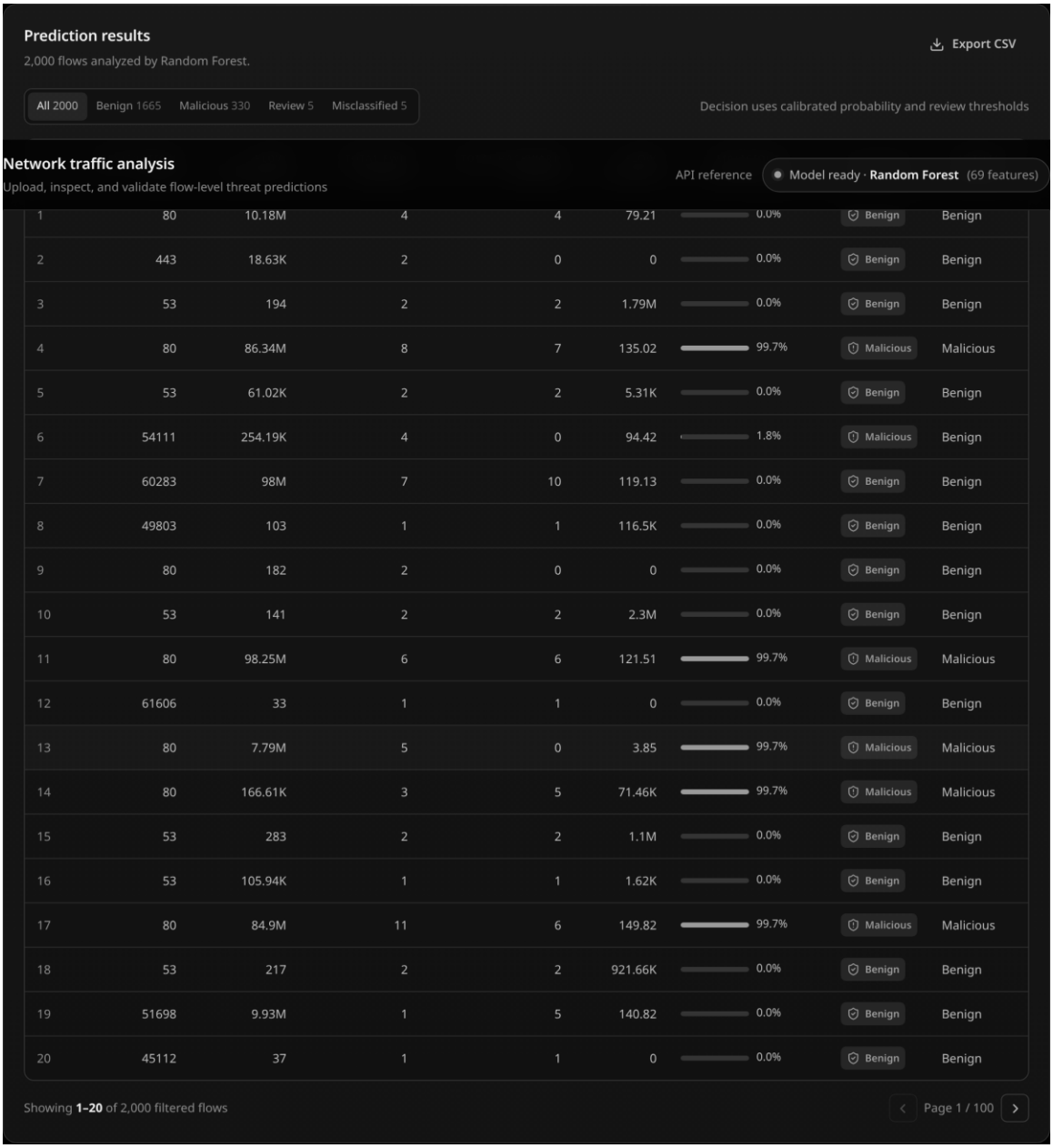


Fig. 6.4 Prediction Results Table With Decisions and Probabilities

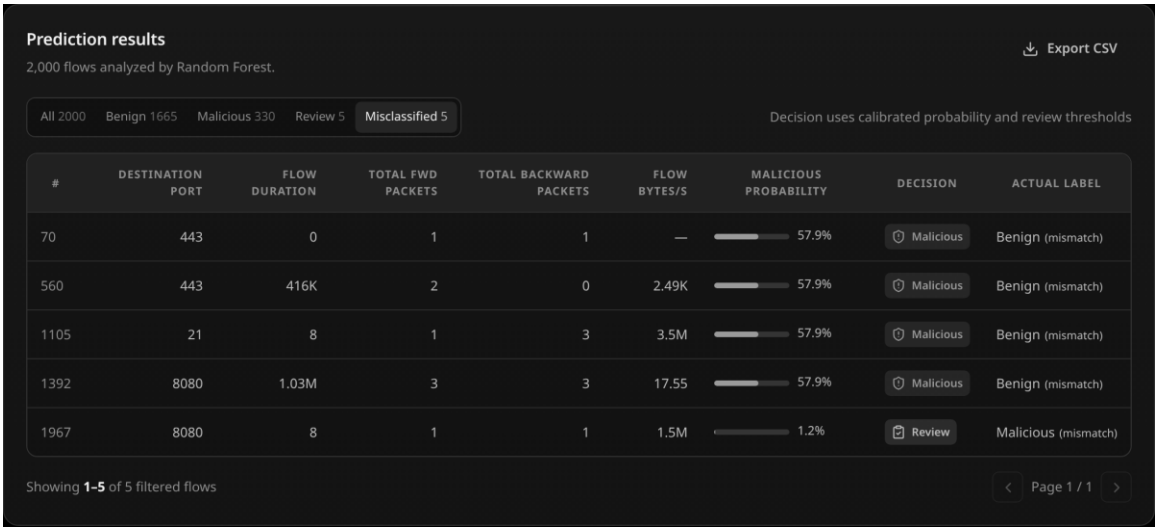


Fig. 6.5 Results Filtered to the Misclassified Flows

Uploading demo_shifted_flows.csv, a copy of the demo flows with every feature scaled far outside its training range, produces a warning that the per-flow predictions should not be trusted, with the features that moved most.

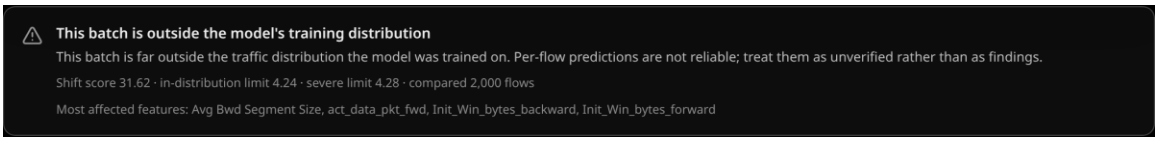


Fig. 6.6 Warning for a Batch Outside the Training Distribution

A file from which three required features were removed is rejected with a list of the missing columns.

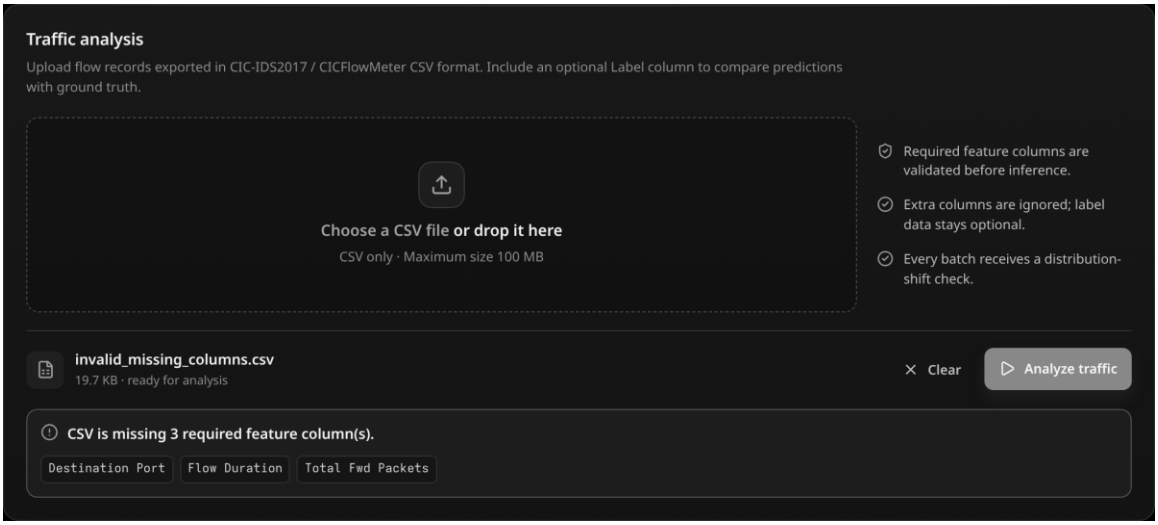


Fig. 6.7 Validation Error for a File With Missing Feature Columns

The evaluation section reads reports/metrics.json and shows the test-set metrics of the three models.

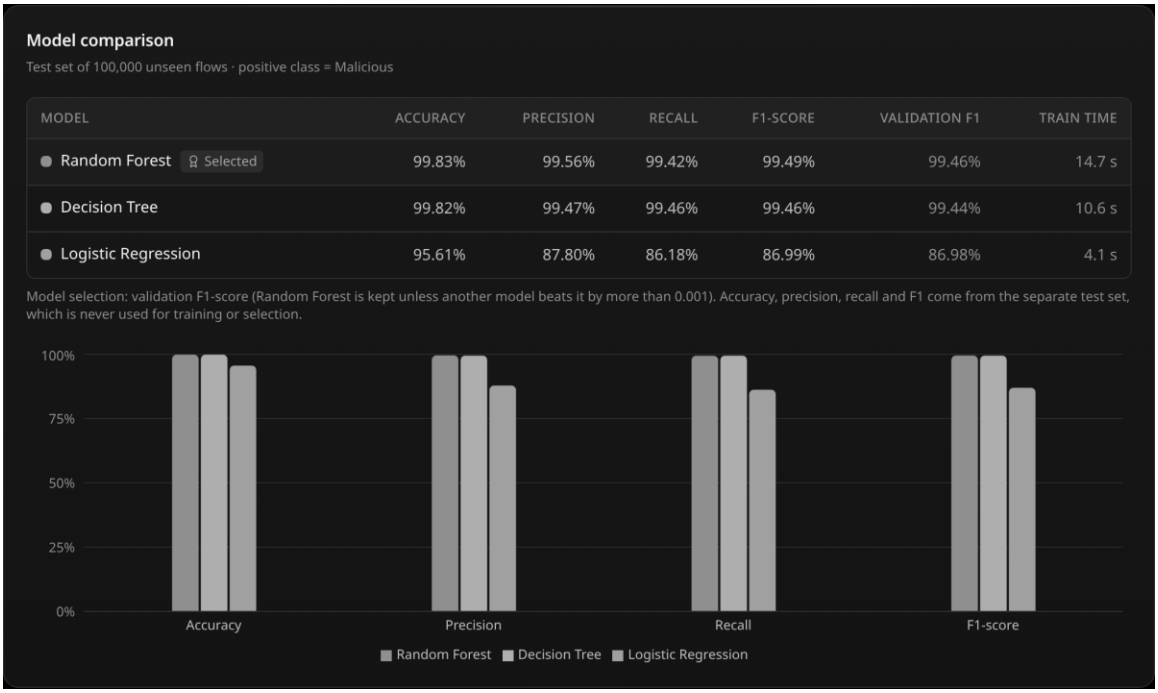


Fig. 6.8 Model Comparison Table and Chart in the Dashboard

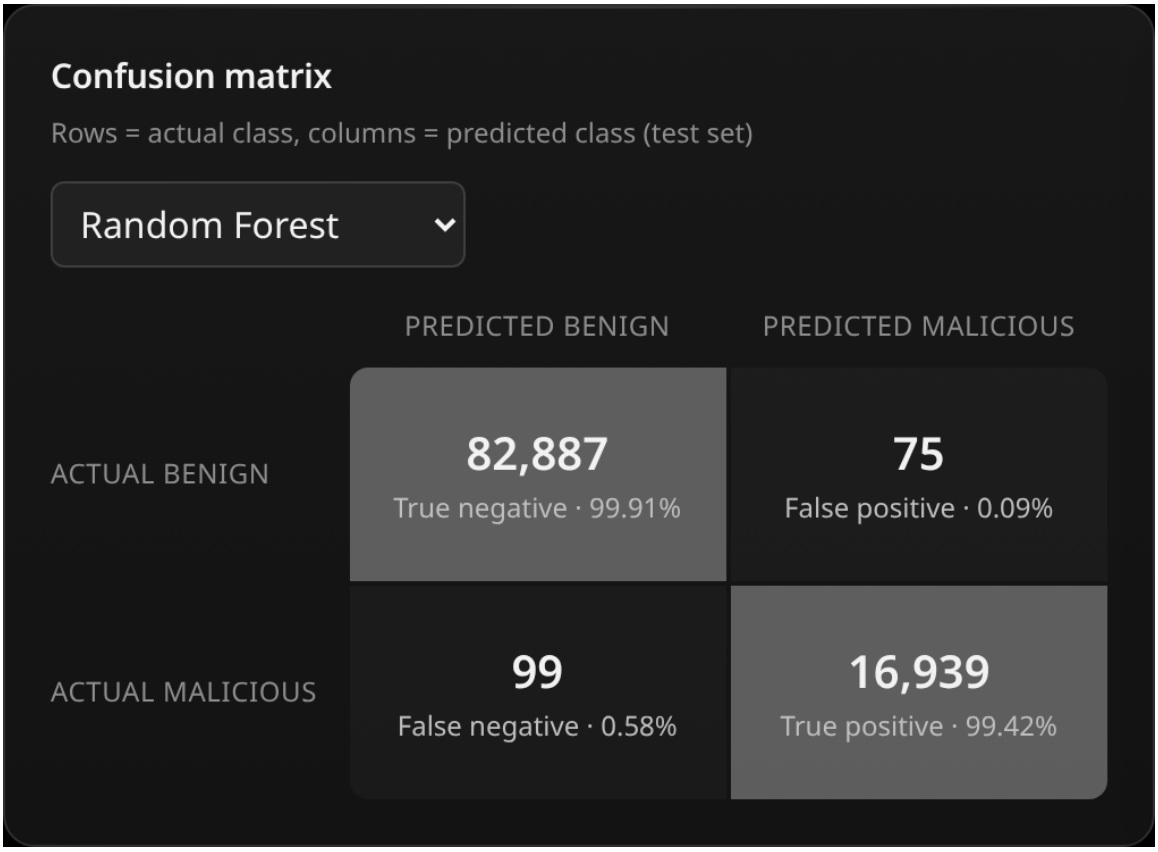


Fig. 6.9 Confusion Matrix in the Dashboard

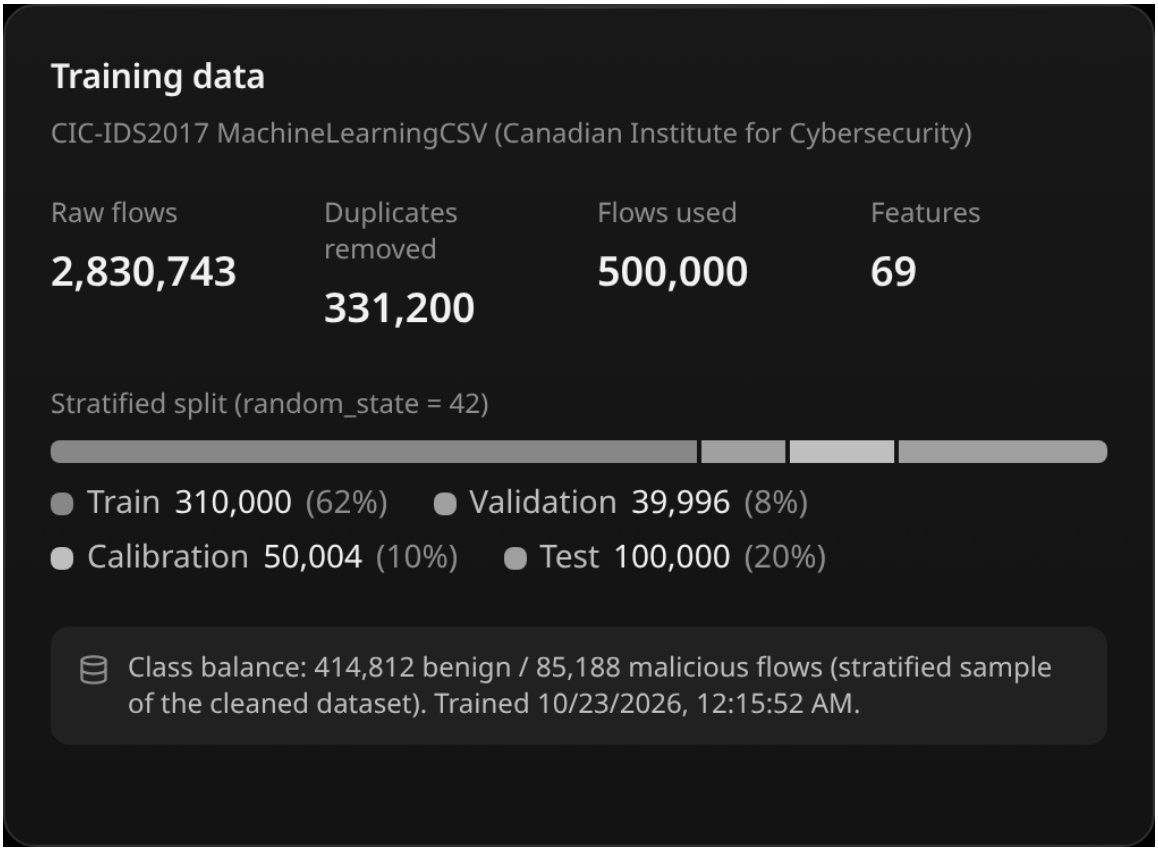


Fig. 6.10 Training Data Summary in the Dashboard

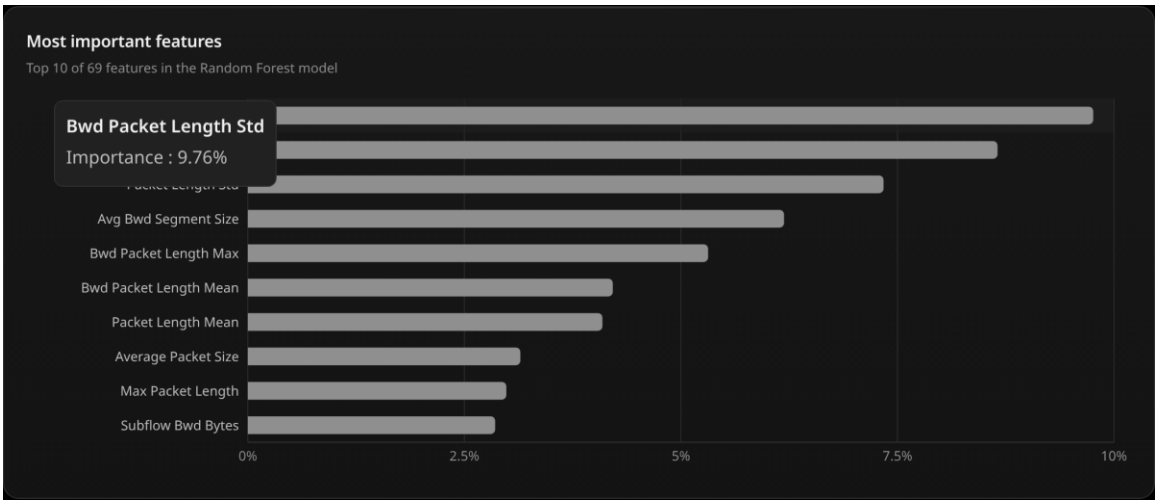


Fig. 6.11 Most Important Features in the Dashboard

FastAPI generates OpenAPI 3.1 documentation at /docs, where every endpoint can be tried from the browser.



Fig. 6.12 Interactive API Documentation Generated by FastAPI

The layout adapts to small screens, as shown for a 390-pixel-wide phone in Fig. 6.13.

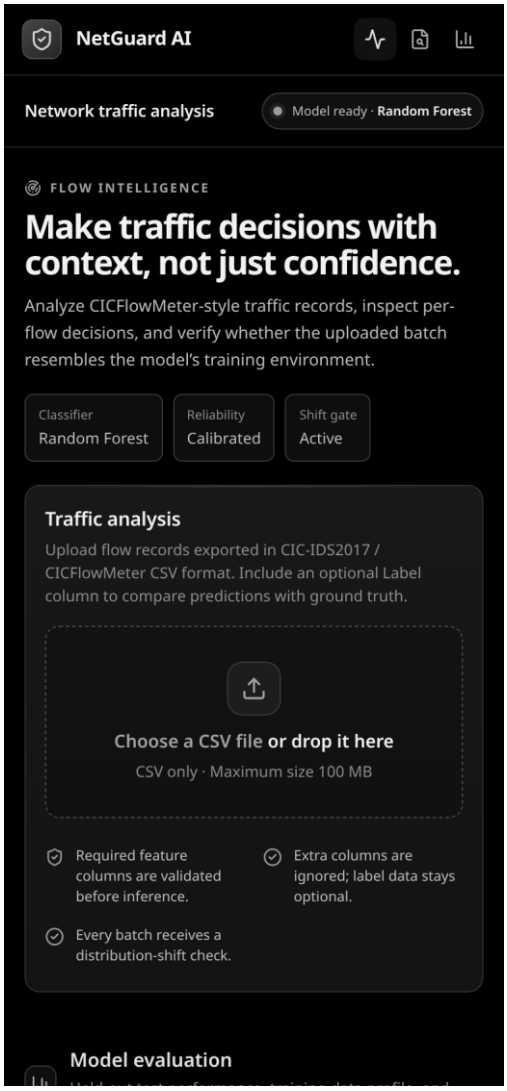


Fig. 6.13 Dashboard on a 390-Pixel-Wide Mobile Screen

CHAPTER 7

RESULT & DISCUSSION

This chapter presents the evaluation of the three models on the held-out test set of 100,000 flows, breaks the results down by attack type, measures the effect of the reliability layer, tests generalisation to unseen capture days, evaluates the shift gate and discusses the limitations that come from the dataset. All numbers are produced by ml/train.py, ml/experiments.py and ml/analyze_dataset.py, or measured through the running application.

7.1 Overall model comparison

Table 7.1 Test-Set Performance of the Three Models (Positive Class = Malicious)

Model	Accuracy	Precision	Recall	F1-score	Validation F1	Train time
Random Forest	99.83%	99.56%	99.42%	99.49%	99.46%	14.7 s
Decision Tree	99.82%	99.47%	99.46%	99.46%	99.44%	10.6 s
Logistic Regression	95.61%	87.80%	86.18%	86.99%	86.98%	4.1 s

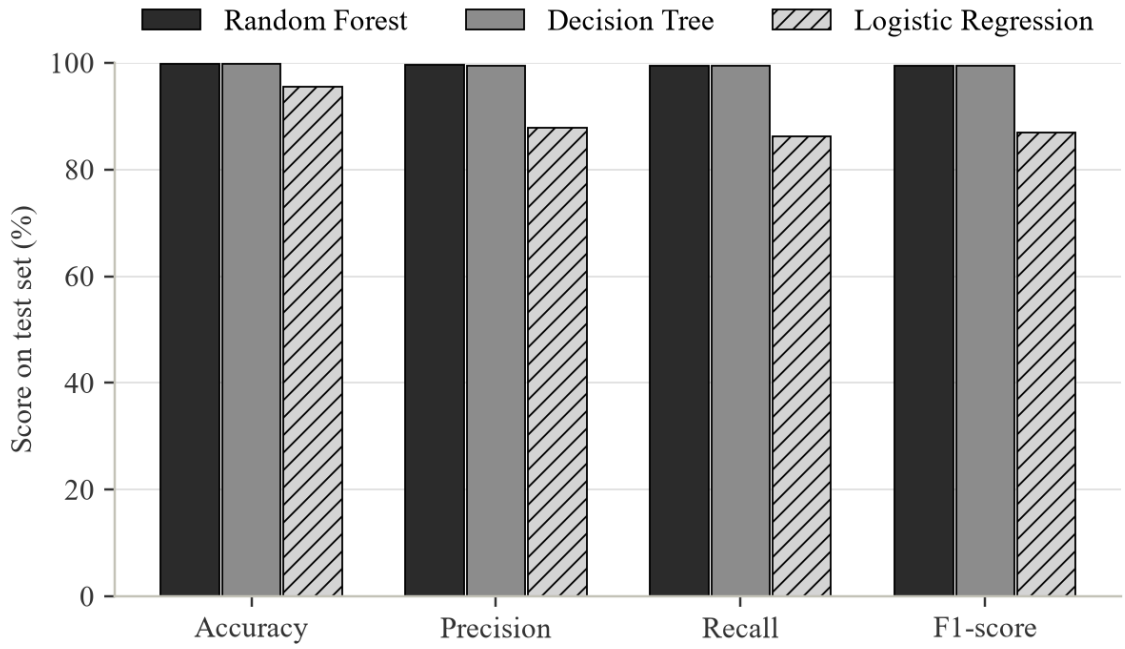


Fig. 7.1 Accuracy, Precision, Recall and F1-Score of the Three Models

Random Forest and Decision Tree are almost tied, with F1-scores of 99.49% and 99.46%. Random Forest has the higher precision, meaning fewer false alarms, and Decision Tree the higher recall. Logistic Regression is clearly weaker, with an F1-score of 86.99%: its single linear boundary cannot capture the threshold-like, non-linear patterns revealed by the exploratory analysis, such as a flow being suspicious when it is very short and targets an unusual port, or very long and almost silent.

Random Forest also had the highest validation F1 (99.46% against 99.44% for Decision Tree), so the selection rule chose it without needing the margin. Accuracy is high for all models partly because 83% of the test flows are benign, which is why the following sections rely on per-class measures.

7.2 Confusion matrices

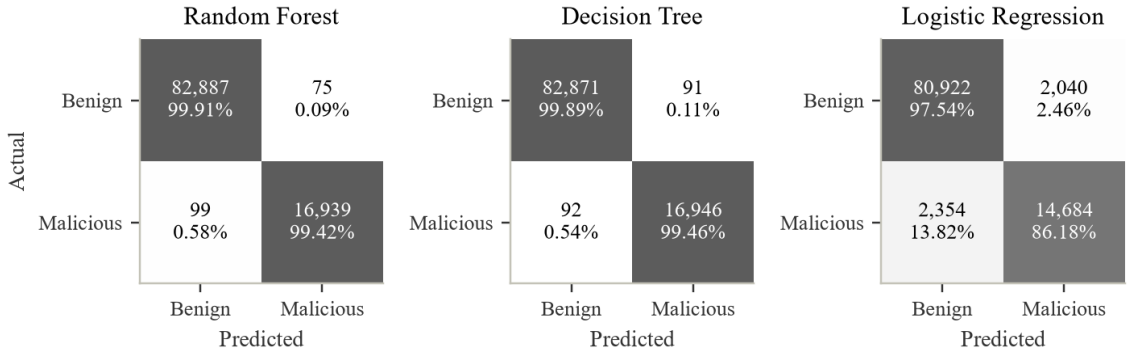


Fig. 7.2 Test-Set Confusion Matrices With Row Percentages

Table 7.2 Confusion Matrix Counts and Error Rates

Model	TN	FP	FN	TP	FPR	Miss rate
Random Forest	82,887	75	99	16,939	0.090%	0.581%
Decision Tree	82,871	91	92	16,946	0.110%	0.540%
Logistic Regression	80,922	2,040	2,354	14,684	2.459%	13.816%

Out of 82,962 benign test flows, the Random Forest at its default 0.5 vote raised 75 false alarms, a false positive rate of 0.090%. Out of 17,038 malicious flows it missed 99. Roughly one alert in 227 is a false alarm and one attack flow in 172 is missed. Logistic Regression would produce 2,040 false alarms and miss 2,354 attack flows on the same traffic.

7.3 Threshold-independent evaluation

Fig. 7.3 shows the ROC and precision–recall curves, which summarise performance over all decision thresholds [27], [28]; the false positive rate is plotted on a logarithmic axis because the interesting region is below 1%.

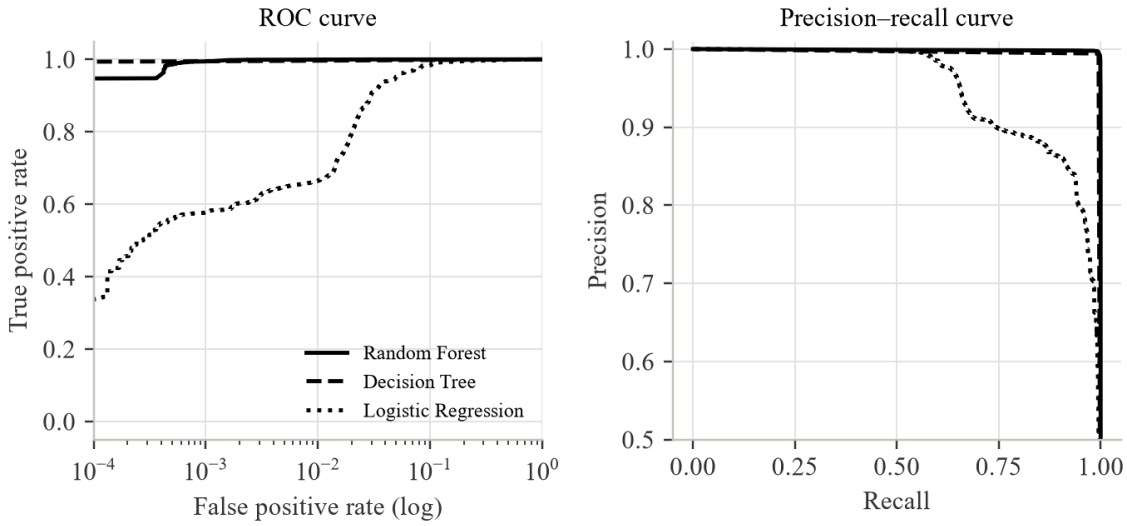


Fig. 7.3 ROC and Precision–Recall Curves on the Test Set

Table 7.3 Area Under the ROC Curve and Average Precision

Model	ROC AUC	Average precision
Random Forest	0.99970	0.99790
Decision Tree	0.99683	0.99005
Logistic Regression	0.98760	0.94810

Random Forest reaches a ROC AUC of 0.9997 and an average precision of 0.9979, and its curve lies above the others at every false positive rate. The Decision Tree's curve has few points because an unpruned tree outputs almost only probabilities of 0 or 1. Fig. 7.4 shows that the Random Forest's raw vote fractions are well separated: most benign flows receive a vote close to 0 and most malicious flows a vote close to 1.

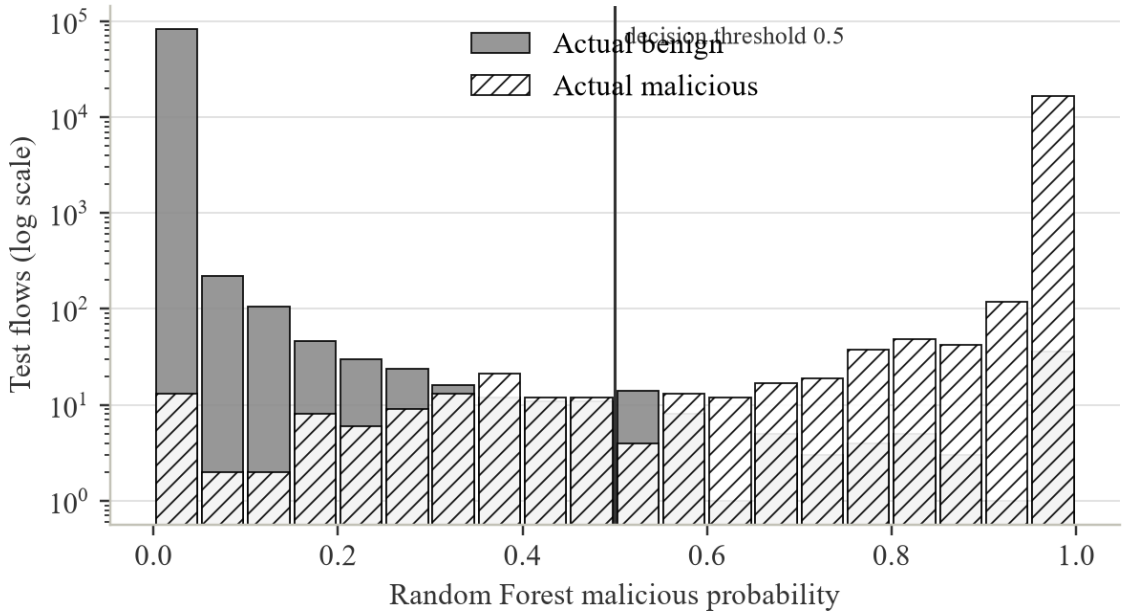


Fig. 7.4 Distribution of the Random Forest Vote Fraction by Actual Class

7.4 Detection rate per attack type

Because the test set keeps the original attack names, recall can be computed for each label separately. Fig. 7.5 compares the three models at their default threshold, and Table 7.4 adds the deployed decision of the Random Forest (calibrated score at the risk-controlled threshold). For BENIGN the value is the share correctly passed.

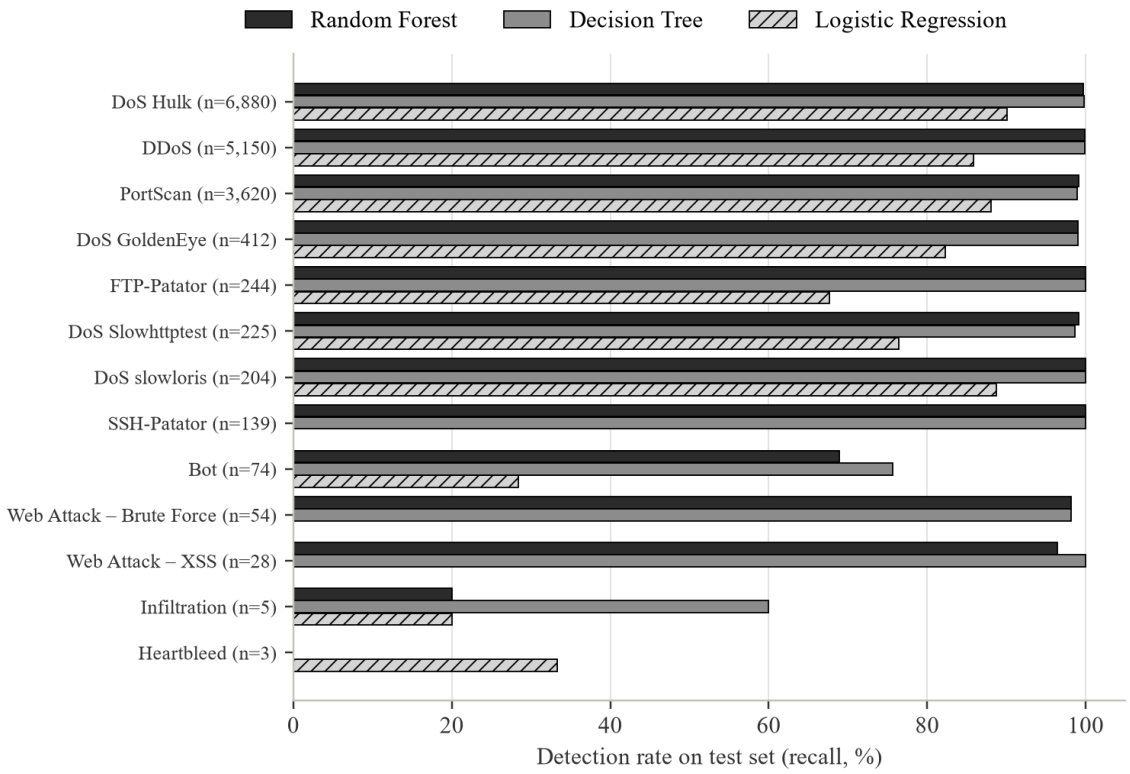


Fig. 7.5 Detection Rate per Attack Label on the Test Set (Default Threshold)

Table 7.4 Detection Rate per Label on the Test Set

Label	Flows	RF 0.5	RF deployed	DT	LR	To review
BENIGN	82,962	99.9%	99.3%	99.9%	97.5%	296
DoS Hulk	6,880	99.7%	99.9%	99.8%	90.1%	1
DDoS	5,150	99.9%	100.0%	99.9%	85.8%	0
PortScan	3,620	99.1%	100.0%	98.9%	88.1%	0
DoS GoldenEye	412	99.0%	100.0%	99.0%	82.3%	0
FTP-Patator	244	100.0%	100.0%	100.0%	67.6%	0
DoS Slowhttptest	225	99.1%	99.1%	98.7%	76.4%	0
DoS slowloris	204	100.0%	100.0%	100.0%	88.7%	0
SSH-Patator	139	100.0%	100.0%	100.0%	0.0%	0
Bot	74	68.9%	97.3%	75.7%	28.4%	1
Web Attack – Brute Force	54	98.1%	100.0%	98.1%	0.0%	0
Web Attack – XSS	28	96.4%	100.0%	100.0%	0.0%	0
Infiltration	5	20.0%	100.0%	60.0%	20.0%	0
Heartbleed	3	0.0%	0.0%	0.0%	33.3%	3

At the default threshold, every attack label with at least 100 test flows is detected at 99.0% or more by Random Forest. The weak points are the rare labels: Bot (68.9% of 74 flows), Infiltration (1 of 5) and Heartbleed (0 of 3). The dataset analysis explains why. The training set held only 4 Infiltration and 0 Heartbleed flows (Table 4.9), and Bot traffic consists of short, low-volume HTTP exchanges (median three packets each way,

Table 4.6) that resemble ordinary web requests, so the forest's averaged votes for many of these flows stay below 0.5.

The deployed decision changes this picture. Its lower, calibrated threshold detects 97.3% of the Bot flows and 5 of 5 Infiltration flows, and reduces the missed attack flows from 99 to 13. The Heartbleed flows, which the model never saw in training, are not passed as benign but sent to review (3 of 3), which is the intended behaviour for flows the model cannot judge. The cost is more false alarms, analysed in the next section. Decision Tree isolates rare training examples in their own leaves and therefore detects more Bot flows than the raw forest; Logistic Regression fails completely on SSH-Patator and the web attacks.

7.5 Effect of the reliability layer

Table 7.5 applies the reliability layer step by step to the same Random Forest on the same test set: B0 is the raw vote at 0.5, B1 the calibrated score at 0.5, B2 the calibrated score at the risk-controlled threshold for $\alpha = 0.01$, and B3 adds the review band (reviewed flows count as not alerted).

Table 7.5 Random Forest With Progressively More of the Reliability Layer (Test Set)

Variant	Precision	Recall	F1-score	FPR	Alerts
B0 raw RF @0.5	99.53%	99.42%	99.48%	0.096%	17,020
B1 calibrated @0.5	99.10%	99.82%	99.46%	0.186%	17,161
B2 CRC threshold	96.83%	99.92%	98.35%	0.673%	17,583
B3 + review band	96.83%	99.92%	98.35%	0.673%	17,583

The risk-controlled threshold trades precision for recall: recall rises from 99.42% to 99.92% while the false positive rate rises from 0.096% to 0.673%, which is still within the budget $\alpha = 1\%$. In other words the operator gives up some precision to miss far fewer attacks, and the rule guarantees how much false-alarm cost that can create. The review band holds 301 test flows (0.30%), far below the 5% capacity. Calibration also improves the probabilities themselves: the expected calibration error falls from 0.00097 to 0.00042 and the Brier score from 0.00135 to 0.00133.

7.6 Generalisation to unseen days

Fig. 7.6 and Table 7.6 compare the three evaluation protocols of Section 4.1.8. This is the most important dataset result of the project.

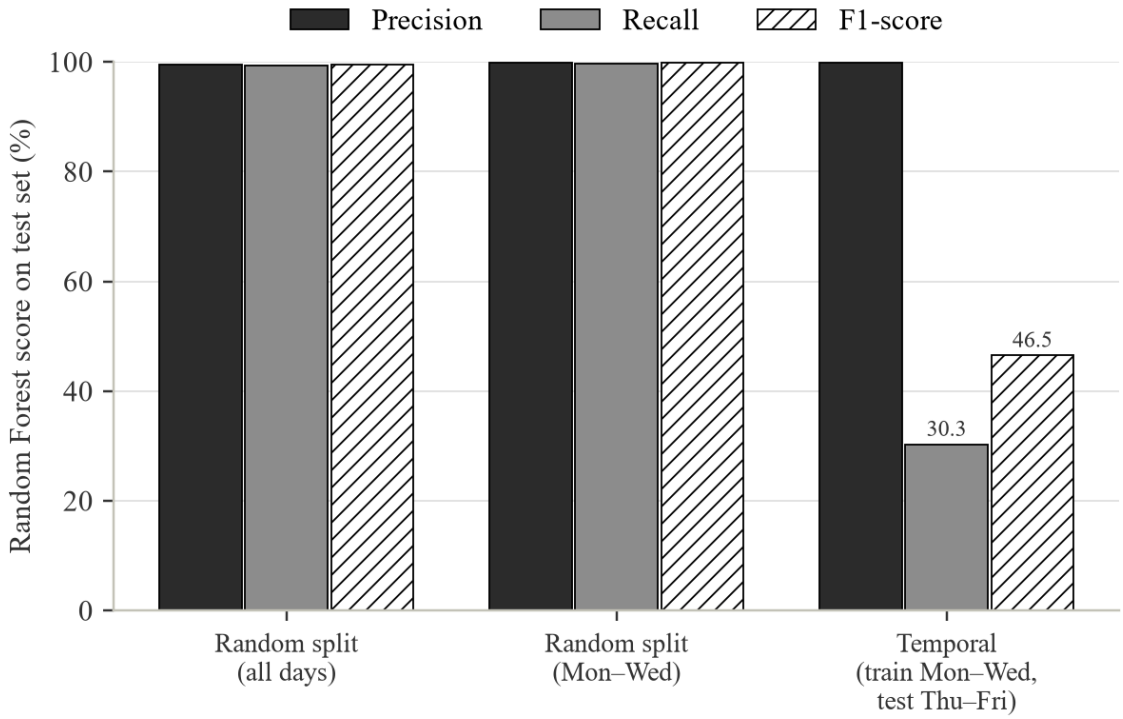


Fig. 7.6 Random Forest Performance Under the Three Evaluation Protocols (Raw Vote at 0.5)

Table 7.6 Results Under the Three Protocols

Protocol	Precision	Recall	F1	Recall (deployed)	FPR (deployed)	Review
Random (all days)	99.5%	99.4%	99.5%	99.9%	0.67%	0.3%
Random (Mon–Wed)	99.8%	99.8%	99.8%	100.0%	0.08%	0.0%
Temporal	99.9%	30.3%	46.5%	52.4%	4.63%	16.2%

With a random split the model looks almost perfect, with an F1-score of 99.5%. When it is trained on Monday to Wednesday and tested on Thursday afternoon and Friday, recall collapses to 30.3% and the F1-score to 46.5%, although precision stays at 99.9%: almost everything the model flags is an attack, but it misses most attacks. Fig. 7.7 shows why, label by label.

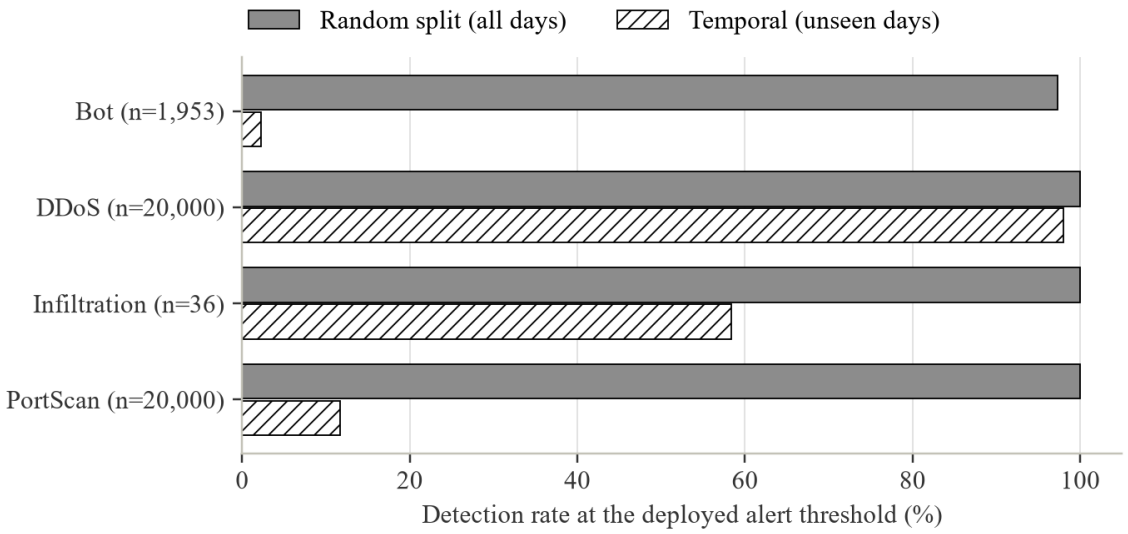


Fig. 7.7 Detection of Each Later-Day Attack: Random Split Versus Unseen Days

Table 7.7 Temporal Protocol: Results per Label on Thursday Afternoon and Friday

Label	Test flows	Value	Measure
BENIGN	20,000	4.6%	false positive rate
Bot	1,953	2.3%	detection rate
DDoS	20,000	98.0%	detection rate
Infiltration	36	58.3%	detection rate
PortScan	20,000	11.7%	detection rate

DDoS is still detected because its HTTP flooding resembles the DoS attacks of Wednesday that the model did see. PortScan and Bot, which never occurred on Monday to Wednesday, are almost entirely missed (11.7% and 2.3%). A supervised model recognises attacks that resemble its training examples; it does not detect attack types it has never seen. A random split hides this because every attack type is present in training, which is why the very high scores commonly reported on CIC-IDS2017 do not predict performance on new traffic [14], [15].

The temporal protocol also tests the reliability layer's assumptions. The threshold was calibrated on Thursday morning for $\alpha = 1\%$, yet the false positive rate on the later days is 4.63%, and the review band grows to 16.2% of the traffic. Risk control is only guaranteed when the calibration and test traffic come from the same distribution; across days they do not. The calibration error rises from 0.0004 on the random split to 0.461. This is precisely the situation the batch shift gate is designed to warn about.

7.7 Batch shift gate

The gate was evaluated on 40 in-distribution batches of benign validation flows and on synthetic shifted batches (Table 7.8).

Table 7.8 Shift Gate on In-Distribution and Synthetic Shifted Batches

Shift type	Mean score	Detection rate	AUC
Every feature × 5	20.16	100%	1.000
Gaussian noise (1 s.d.)	15.70	100%	1.000
Mean + 2 s.d.	31.56	100%	1.000
Columns shuffled independently	0.98	0%	0.487

No in-distribution batch was flagged (mean score 1.00, threshold 4.24), and scaled, noisy and mean-shifted batches were all detected. Shuffling each column independently is a deliberate negative control: it destroys the relationships between features but leaves every single-feature distribution unchanged, so a per-feature Kolmogorov–Smirnov test cannot see it, and the gate's detection rate is 0%. Fig. 7.8 shows the gate applied to each capture day of the test set.

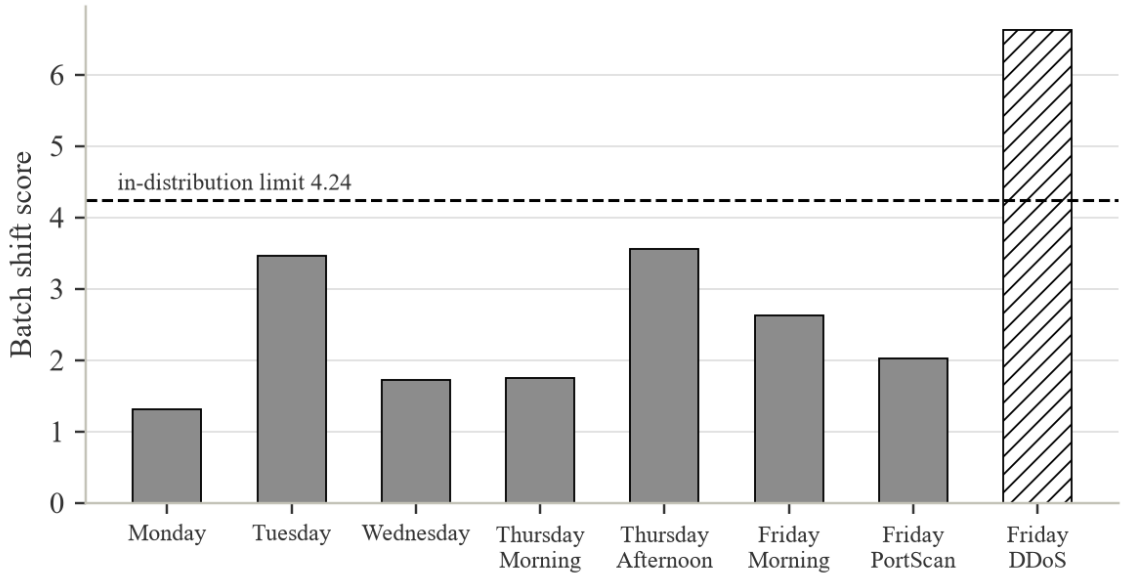


Fig. 7.8 Batch Shift Score of Each Capture Day in the Test Set

Only Friday-DDoS exceeds the limit; the features whose distribution moved most on that day are Bwd IAT Std, Destination Port, min_seg_size_forward. The other days, including those with attacks, stay below it because the gate judges only the traffic the model did not flag. The gate also flagged the deliberately shifted demonstration file

(score 31.6) and, instructively, a contiguous slice of the first 5,000 rows of the raw Wednesday file (score 10.3), although that slice is benign traffic from the training period. A few minutes of traffic from one part of one day does not look like a whole day's mixture, so the verdict should be read as "this batch is unlike the reference", not as "this batch is malicious".

7.8 Feature importance

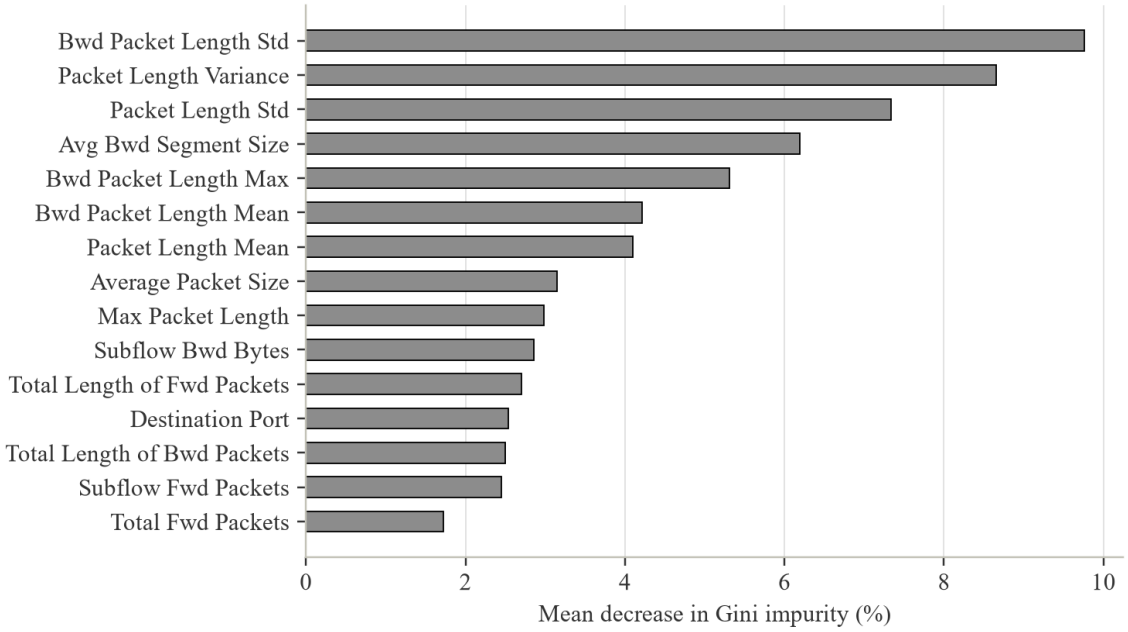


Fig. 7.9 Fifteen Most Important Features of the Random Forest

Fig. 7.9 shows the features with the largest mean decrease in impurity. Packet-size features dominate: Bwd Packet Length Std (9.8%), Packet Length Variance, Packet Length Std and Avg Bwd Segment Size lead the ranking, and size-related features account for 60% of the importance among the top fifteen. Attacked web servers return a mixture of large pages and small error responses, so the size variation of backward packets is high for DoS and DDoS flows and low for typical benign flows.

Because many features are almost perfectly correlated (Table 4.7), the importance of a property is spread over its copies; Packet Length Variance and Packet Length Std, for example, carry the same information. The ranking should therefore be read at the level of properties: packet sizes and byte totals first, the destination port next (rank 12), and timing, flag and TCP window features further down.

7.9 Results on the demonstration files

The demonstration file demo_test_flows.csv contains 2,000 labelled flows from the test set. Through the running application 1,665 were decided benign, 330 malicious and 5 were sent to review. The binary prediction agreed with the label for 1,995 flows (99.75%); the alerts had a precision of 97.6% and caught 99.7% of the 323 malicious flows. The batch was judged in distribution (shift score 1.06). The deliberately shifted copy demo_shifted_flows.csv was flagged as severely shifted with a warning not to trust its per-flow predictions, and the invalid file was rejected with the list of missing columns.

7.10 Limitations and threats to validity

- ✓ **Unseen attack types.** The temporal experiment shows that the model detects attacks similar to those in its training data but misses new attack types. The random-split scores overstate real-world performance.
- ✓ **Single laboratory network.** All traffic comes from one testbed recorded in 2017 [14], [15].
- ✓ **Dataset errors.** Engelen et al. [20] and Lanvin et al. [21] found labelling mistakes and flow-meter artefacts in CIC-IDS2017. This analysis found further artefacts: negative durations, Fwd PSH Flags identical to SYN Flag Count, and bulk features that are always zero. Class imbalance was also noted by Panigrahi and Borah [42].
- ✓ **Rare labels.** Heartbleed, Infiltration and SQL injection have too few flows for reliable learning or evaluation.
- ✓ **Risk-control assumption.** The false-positive guarantee holds only for traffic that resembles the calibration data; under the temporal protocol it was exceeded.
- ✓ **Shift gate scope.** The gate compares single-feature distributions, so it cannot detect changes in the relationships between features, and short contiguous slices of normal traffic can be flagged.
- ✓ **Binary output.** The system says whether a flow is malicious, not which attack it belongs to.

7.11 Cost analysis

The project uses only open-source software and a public dataset, and it runs on an ordinary laptop. Table 7.9 lists the cost items.

Table 7.9 Project Cost

Item	Choice	Cost
Dataset	CIC-IDS2017 (public)	Free
Programming languages and libraries	Python, scikit-learn, SciPy, FastAPI, React, Vite, Tailwind, Recharts	Free (open source)
Hardware	Existing laptop (Intel Core i5, 8 GB RAM)	No additional cost
Hosting for the demonstration	Local machine (single process on one port)	Free
Computation	A few minutes of training and analysis	Negligible electricity

CHAPTER 8

SUMMARY

NetGuard AI is a complete, working system for malware and intrusion detection based on AI-driven analysis of network flows. Its machine learning pipeline turns the eight CIC-IDS2017 files into a clean, deduplicated and correctly split dataset; three classifiers are trained and compared; and the selected Random Forest, wrapped in a reliability layer, is served through a validated FastAPI backend and a React dashboard that shows real predictions and real evaluation results.

The dataset received particular attention. The analysis of all 2,830,743 flows showed that 11.7% of the rows are exact duplicates, concentrated in the attack labels PortScan and SSH-Patator; that rate features contain infinite and missing values because of zero-duration flows; that the -1 values in the window features are protocol sentinels for UDP rather than errors; that eight features are constant and several others are exact copies; and that the attack labels are distributed extremely unevenly, from 231,073 DoS Hulk flows to 11 Heartbleed flows. Each finding led to a specific design decision.

On 100,000 unseen test flows from a random split, the Random Forest achieved an accuracy of 99.83%, precision of 99.56%, recall of 99.42% and an F1-score of 99.49%, with a ROC AUC of 0.9997. The reliability layer raised recall to 99.92% while keeping the false positive rate at 0.67%, within the chosen budget of 1%, and sent the flows it could not judge, including the unseen Heartbleed flows, to review.

The most important finding concerns generalisation. When the model is trained on Monday to Wednesday and tested on later days, recall falls to 30.3%: attacks similar to those in training (DDoS) are still detected, but attack types that never occurred in training (PortScan, Bot) are almost entirely missed, and the false-positive guarantee no longer holds. The very high scores of the random split therefore describe recognition of known attacks, not detection of new ones, and this is a property of how the dataset is split rather than of the model.

Based on these findings, the following recommendations are made:

- ✓ **Evaluate on unseen days or networks.** Report a temporal or cross-dataset result alongside any random split; it is the honest estimate of real-world performance.
- ✓ **Stratify on the attack label.** Stratifying the sample and split on the original attack label instead of the binary class would guarantee that every attack type appears in training, validation and test.
- ✓ **Keep every flow of rare labels.** Including all Heartbleed, Infiltration, SQL injection and Bot flows in the training data, or oversampling them (for example with SMOTE [43]), would give the model enough examples to learn from.
- ✓ **Recalibrate when the shift gate fires.** A shift warning means the alert threshold may no longer meet its budget; the calibration set should then be refreshed with recent traffic.
- ✓ **Use a corrected version of the dataset.** The relabelled CIC-IDS2017 released by Engelen et al. [20] fixes known labelling and flow-meter errors and should be used for further work.
- ✓ **Extend the system.** Multi-class output (naming the attack), live flow capture with CICFlowMeter and an anomaly detector for attack types absent from training are natural next steps.

The project demonstrates that a small, explainable system built from standard tools can detect most known network attacks in a realistic dataset very accurately, that it can be made to state how far its decisions can be trusted, and that understanding the dataset is as important as choosing the model.

CHAPTER 9

REFERENCES

[1] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[2] A. S. Tanenbaum, N. Feamster and D. J. Wetherall, *Computer Networks*, 6th ed. Pearson, 2021.

[3] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022.

[4] J. Postel, "User Datagram Protocol," IETF, RFC 768, Aug. 1980.

[5] K. Ramakrishnan, S. Floyd and D. Black, "The Addition of Explicit Congestion Notification (ECN) to IP," IETF, RFC 3168, Sep. 2001.

[6] B. Claise, Ed., "Cisco Systems NetFlow Services Export Version 9," IETF, RFC 3954, Oct. 2004.

[7] B. Claise, B. Trammell and P. Aitken, Eds., "Specification of the IP Flow Information Export (IPFIX) Protocol for the Exchange of Flow Information," IETF, RFC 7011, Sep. 2013.

[8] G. Draper-Gil, A. H. Lashkari, M. S. I. Mamun and A. A. Ghorbani, "Characterization of encrypted and VPN traffic using time-related features," in *Proc. 2nd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2016, pp. 407–414.

[9] A. H. Lashkari, G. Draper-Gil, M. S. I. Mamun and A. A. Ghorbani, "Characterization of Tor traffic using time based features," in *Proc. 3rd Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2017, pp. 253–262.

[10] D. E. Denning, "An intrusion-detection model," *IEEE Transactions on Software Engineering*, vol. SE-13, no. 2, pp. 222–232, Feb. 1987.

[11] A. L. Buczak and E. Guven, "A survey of data mining and machine learning methods for cyber security intrusion detection," *IEEE Communications Surveys & Tutorials*, vol. 18, no. 2, pp. 1153–1176, 2016.

[12] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[13] L. Breiman, J. H. Friedman, R. A. Olshen and C. J. Stone, *Classification and Regression Trees*. Belmont, CA, USA: Wadsworth, 1984.

[14] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *Proc. IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[15] D. Arp et al., "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[16] M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho, "A survey of network-based intrusion detection data sets," *Computers & Security*, vol. 86, pp. 147–167, 2019.

[17] M. Tavallaee, E. Bagheri, W. Lu and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proc. IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009, pp. 1–6.

[18] N. Moustafa and J. Slay, "UNSW-NB15: A comprehensive data set for network intrusion detection systems," in *Proc. Military Communications and Information Systems Conference (MilCIS)*, 2015, pp. 1–6.

[19] I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116. Dataset: https://www.unb.ca/cic/datasets/ids-2017.html

[20] G. Engelen, V. Rimmer and W. Joosen, "Troubleshooting an intrusion detection dataset: The CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[21] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, LNCS, vol. 13857. Springer, 2023, pp. 18–33.

[22] Z. Durumeric et al., "The matter of Heartbleed," in *Proc. ACM Internet Measurement Conference (IMC)*, 2014, pp. 475–488.

[23] H. He and E. A. Garcia, "Learning from imbalanced data," *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.

[24] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[25] J. R. Quinlan, "Induction of decision trees," *Machine Learning*, vol. 1, no. 1, pp. 81–106, 1986.

[26] D. R. Cox, "The regression analysis of binary sequences," *Journal of the Royal Statistical Society: Series B*, vol. 20, no. 2, pp. 215–242, 1958.

[27] T. Fawcett, "An introduction to ROC analysis," *Pattern Recognition Letters*, vol. 27, no. 8, pp. 861–874, 2006.

[28] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, e0118432, 2015.

[29] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proc. 8th ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2002, pp. 694–699.

[30] A. N. Angelopoulos, S. Bates, A. Fisch, L. Lei and T. Schuster, "Conformal risk control," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. arXiv:2208.02814.

[31] F. J. Massey, "The Kolmogorov-Smirnov test for goodness of fit," *Journal of the American Statistical Association*, vol. 46, no. 253, pp. 68–78, 1951.

[32] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com

[33] Meta Open Source, "React documentation." [Online]. Available: https://react.dev

[34] J. Postel, "Internet Protocol," IETF, RFC 791, Sep. 1981.

[35] R. Fielding, M. Nottingham and J. Reschke, Eds., "HTTP Semantics," IETF, RFC 9110, Jun. 2022.

[36] L. Masinter, "Returning Values from Forms: multipart/form-data," IETF, RFC 7578, Jul. 2015.

[37] Y. Shafranovich, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," IETF, RFC 4180, Oct. 2005.

[38] T. Bray, Ed., "The JavaScript Object Notation (JSON) Data Interchange Format," IETF, RFC 8259, Dec. 2017.

[39] OpenAPI Initiative, "OpenAPI Specification v3.1.0," Feb. 2021. [Online]. Available: https://spec.openapis.org/oas/v3.1.0

[40] G. van Rossum, B. Warsaw and N. Coghlan, "PEP 8 – Style Guide for Python Code," Python Software Foundation, 2001. [Online]. Available: https://peps.python.org/pep-0008/

[41] W3C, "Web Content Accessibility Guidelines (WCAG) 2.2," W3C Recommendation, Oct. 2023.

[42] R. Panigrahi and S. Borah, "A detailed analysis of CICIDS2017 dataset for designing intrusion detection systems," *International Journal of Engineering & Technology*, vol. 7, no. 3.24, pp. 479–482, 2018.

[43] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

APPENDIX A

PROJECT FILES, API EXAMPLE AND MODEL FEATURES

A.1 Repository structure

```
NetGuardAI/
├── ml/                        config.py, preprocess.py, train.py,
reliability.py,
│                              experiments.py, analyze_dataset.py
├── backend/app/               main.py (FastAPI), model_store.py
├── backend/tests/             conftest.py, test_api.py, test_reliability.py,
test_training.py
├── frontend/src/              App.tsx, api.ts, format.ts, index.css,
components/
├── dataset/raw/               CIC-IDS2017 CSV files (not in git, ≈844 MB)
├── dataset/samples/           demo_test_flows.csv, demo_unlabeled_flows.csv,
│                              invalid_missing_columns.csv,
demo_shifted_flows.csv
├── dataset/faculty_tests/     14 test files for evaluators
├── models/                    netguard_model.joblib
├── reports/                   metrics.json, experiments.json,
dataset_analysis.json, figures/
├── report/                    scripts that generate this document
├── docs/                      screenshots/, demo/, report/ (this document)
├── start.sh, stop.sh          start and stop the whole application
├── requirements.txt
└── README.md
```

A.2 Example API response

Response of POST /api/predict for demo_test_flows.csv, recorded from the running application (only the first row of the table is shown):


```
{
  "filename": "demo_test_flows.csv",
  "model_name": "Random Forest",
  "summary": {
    "total": 2000,
    "benign": 1665,
    "malicious": 330,
    "review": 5,
    "malicious_ratio": 0.165,
    "review_ratio": 0.0025,
    "benign_ratio": 0.8325
  },
  "evaluation": {
    "accuracy": 0.9975,
    "correct": 1995,
    "actual_malicious": 323,
    "actual_benign": 1677,
    "alert_precision": 0.9757575757575757,
    "alert_recall": 0.9969040247678018,
    "review": 5,
    "review_rate": 0.0025
  },
  "reliability": {
    "active": true,
    "kind": "isotonic+crc",
```

```
    "alpha": 0.01,
    "t_alert": 0.018181818181818181,
    "t_benign": 0.011627906976744186,
    "review_capacity": 0.05,
    "counts": {
      "benign": 1665,
      "malicious": 330,
      "review": 5
    }
  },
  "shift": {
    "verdict": "in_distribution",
    "score": 1.063812978621859,
    "threshold_in": 4.237768292391645,
    "threshold_severe": 4.281739763256287,
    "top_shifted_features": [],
    "basis": "non_alerted",
    "rows_compared": 1670
  },
  "warning": null,
  "display_columns": [
    "Destination Port",
    "Flow Duration",
    "Total Fwd Packets",
    "Total Backward Packets",
    "Flow Bytes/s"
  ],
  "rows": [
    {
      "row": 1,
      "values": {
        "Destination Port": 80.0,
        "Flow Duration": 10175832.0,
        "Total Fwd Packets": 4.0,
        "Total Backward Packets": 4.0,
        "Flow Bytes/s": 79.20728
      },
      "prediction": "Benign",
      "decision": "Benign",
      "malicious_probability": 0.0,
      "raw_vote_fraction": 0.0,
      "actual": "Benign"
    }
  ],
  "rows_truncated": false
}
```


A.3 Features required by the model

An uploaded CSV must contain the following 69 columns (names are matched after stripping spaces):

Table A.1 Model Input Features

Features 1–23	Features 24–46	Features 47–69
1. Destination Port	24. Fwd IAT Max	47. URG Flag Count
2. Flow Duration	25. Fwd IAT Min	48. CWE Flag Count
3. Total Fwd Packets	26. Bwd IAT Total	49. ECE Flag Count
4. Total Backward Packets	27. Bwd IAT Mean	50. Down/Up Ratio
5. Total Length of Fwd Packets	28. Bwd IAT Std	51. Average Packet Size
6. Total Length of Bwd Packets	29. Bwd IAT Max	52. Avg Fwd Segment Size
7. Fwd Packet Length Max	30. Bwd IAT Min	53. Avg Bwd Segment Size
8. Fwd Packet Length Min	31. Fwd PSH Flags	54. Subflow Fwd Packets

Features 1–23	Features 24–46	Features 47–69
9. Fwd Packet Length Mean	32. Fwd URG Flags	55. Subflow Fwd Bytes
10. Fwd Packet Length Std	33. Fwd Header Length	56. Subflow Bwd Packets
11. Bwd Packet Length Max	34. Bwd Header Length	57. Subflow Bwd Bytes
12. Bwd Packet Length Min	35. Fwd Packets/s	58. Init_Win_bytes_forward
13. Bwd Packet Length Mean	36. Bwd Packets/s	59. Init_Win_bytes_backward
14. Bwd Packet Length Std	37. Min Packet Length	60. act_data_pkt_fwd
15. Flow Bytes/s	38. Max Packet Length	61. min_seg_size_forward
16. Flow Packets/s	39. Packet Length Mean	62. Active Mean
17. Flow IAT Mean	40. Packet Length Std	63. Active Std
18. Flow IAT Std	41. Packet Length Variance	64. Active Max
19. Flow IAT Max	42. FIN Flag Count	65. Active Min
20. Flow IAT Min	43. SYN Flag Count	66. Idle Mean
21. Fwd IAT Total	44. RST Flag Count	67. Idle Std
22. Fwd IAT Mean	45. PSH Flag Count	68. Idle Max
23. Fwd IAT Std	46. ACK Flag Count	69. Idle Min

A.4 Demonstration and test files

Table A.2 Demonstration and Test Files

File	Content	Purpose
demo_test_flows.csv	2,000 real flows from the held-out test set, with Label	Shows predictions and agreement with labels
demo_unlabeled_flows.csv	The same 2,000 flows without the Label column	Shows prediction on unlabelled traffic
invalid_missing_columns.csv	50 flows with three required columns removed	Shows input validation
demo_shifted_flows.csv	The demo flows with every feature scaled far out of range	Shows the shift warning
faculty_tests/ (14 files)	Small and large mixed, benign-only, attack-only, extra columns, missing values, non-numeric values, missing columns, empty, wrong extension, mildly and severely shifted, one flow per attack, unlabelled	Lets an evaluator test every behaviour